

RTL Design Sherpa

RAPIDS Hardware Architecture Specification

September 30, 2026

Table of Contents

1 Rapids Has Index....22

2 Document Information....23

2.1 Purpose....24

2.2 Scope....25

2.3 How This Book Relates to RAPIDS Beats....26

2.4 Audience....27

2.5 Document Conventions....28

2.5.1 Requirement Levels....28

2.5.2 Signal Directions....28

2.5.3 Byte and Beat Terms....28

2.5.4 Timing Diagrams....28

2.6 References....29

3 Revision History....30

3.1 Change Summary....33

3.1.1 Version 0.2 (2026-09-30)....33

3.1.2 Version 0.1 (2026-09-30)....33

4 Acronyms and Definitions....34

4.1 Acronyms....34

4.2 Definitions....35

4.3 Signal Naming Conventions....36

5 Product Overview....37

5.1 Introduction....37

5.2 System Context....38
5.2.1 Figure 1.1.1: RAPIDS in Its System....38
5.3 Target Applications....39
5.4 What the Byte Design Adds....40
5.4.1 Key Capabilities....40
5.5 Permanent Limitation....41
6 Key Features....42
6.1 Feature Summary....42
6.1.1 Multi-Channel Architecture....42
6.1.2 Byte-Granular Data Movement....42
6.1.3 Descriptor-Based Control....43
6.1.4 High-Performance Data Paths....43
6.1.5 Monitoring and Debug....43
6.2 Feature Details....44
6.2.1 SRAM Buffering....45
6.2.2 AXI Burst Optimization....45
6.2.3 Error Handling....46
7 System Context....47
7.1 Integration Overview....47
7.2 System Block Diagram....48
7.2.1 Figure 1.3.1: System Integration....48
7.3 Interface Connections....49
7.3.1 Memory Interface....49
7.3.2 Network Interface....49

7.3.3 Configuration Interface....50
7.4 Clock and Reset....51
7.4.1 Clock Domains....51
7.4.2 Reset Requirements....51
8 Block Diagram....52
8.1 Top-Level Architecture....52
8.1.1 Figure 2.1.1: RAPIDS Block Diagram....52
8.2 Top-Level Integration (rapids_top, rapids_cxs_top and the 8- and 16-channel tops)....53
8.3 Component Summary....54
8.3.1 Scheduler Group Array....54
8.3.2 Sink Data Path....55
8.3.3 Source Data Path....56
8.4 Hierarchy....57
8.5 Internal Buses....58
9 Data Flow....59
9.1 Overview....59
9.2 Byte Placement Rules....60
9.2.1 Figure 2.2.1: Sink Byte Placement....61
9.3 The Packet Record....62
9.3.1 Figure 2.2.2: Packet Record Flow....62
9.4 Concurrent Operation....64
9.4.1 Deadlock Prevention....64
10 Channel Architecture....65

10.1 Multi-Channel Design..... 65
10.2 Channel Resource Allocation..... 66
10.2.1 Figure 2.3.1: Per-Channel and Shared Resources..... 66
10.3 Per-Channel Resources..... 67
10.4 Shared Resources..... 68
10.5 Channel Scheduling..... 69
10.5.1 Arbitration Policy..... 69
10.5.2 Figure 2.3.2: Channel Arbitration onto a Shared AXI Port..... 69
10.6 Channel State Machine..... 70
10.7 Channel Isolation..... 72
10.7.1 Per-Channel Error Handling..... 72
11 Interface Summary..... 73
11.1 External Interface Overview..... 73
11.1.1 Figure 3.1.1: External Interfaces..... 73
11.2 Interface Port Summary..... 74
11.3 Clock and Reset..... 75
11.4 Interface Feature Matrix..... 76
11.5 Signal Count Summary..... 77
12 AXI4 Interface Specification..... 78
12.1 Overview..... 78
12.1.1 Byte Granularity on the Memory Side..... 78
12.2 Descriptor AXI Master..... 79
12.3 Sink AXI Master (Write)..... 80
12.3.1 Purpose..... 80

12.3.2 Configuration....80
12.3.3 Signal List....80
12.3.4 WSTRB Contract....81
12.4 Source AXI Master (Read)....83
12.4.1 Purpose....83
12.4.2 Configuration....83
12.4.3 Signal List....83
12.4.4 Read Coverage....84
12.5 AXI Protocol Constraints....85
12.5.1 Address Alignment....85
12.5.2 4 KB Boundary Handling....85
12.5.3 Response Handling....85
13 AXIS Interface Specification....87
13.1 Overview....87
13.1.1 Byte Semantics....87
13.2 Sink AXIS Slave....88
13.2.1 Purpose....88
13.2.2 Configuration....88
13.2.3 Signal List....88
13.2.4 Packet Contract....89
13.2.5 Descriptor-First Ordering....89
13.2.6 Backpressure Behavior....90
13.3 Source AXIS Master....91
13.3.1 Purpose....91

13.3.2 Configuration....91
13.3.3 Signal List....91
13.3.4 Egress Packet Contract....92
13.3.5 Timing....92
13.3.6 Data Availability....92
13.4 AXIS Protocol Notes....93
13.4.1 Handshake Rules....93
13.4.2 TLAST Semantics....93
13.4.3 TSTRB Usage....93
13.4.4 Channel Multiplexing via TID....94
14 APB Configuration Interface....95
14.1 Overview....95
14.2 APB Slave and Address Routing....96
14.2.1 APB Slave Signal List....96
14.2.2 Descriptor Kick-Off....97
14.3 Status Interface....98
14.3.1 Per-Channel Status Signals....98
14.3.2 Channel State Encoding....98
14.4 Configuration Registers....99
14.4.1 Address Range Validation....99
14.4.2 Configuration Registers....99
14.5 Programming Sequence....100
14.5.1 Initialization....100
14.5.2 Descriptor Kick-Off....100

15 Monitor Bus Interface..... 102
15.1 Overview..... 102
15.2 MonBus Packet Format..... 103
15.2.1 64-bit Packet Structure..... 103
15.2.2 Signal List..... 103
15.3 Packet Types..... 104
15.4 Agent IDs..... 105
15.5 Event Codes..... 107
15.5.1 Scheduler Events (Agent 0x30-0x37)..... 107
15.5.2 Descriptor Engine Events (Agent 0x10-0x17)..... 107
15.6 Event Data Encoding..... 108
15.6.1 Completion Event Data..... 108
15.6.2 Error Event Data..... 108
15.6.3 State Transition Event Data..... 108
15.7 MonBus Arbitration..... 109
15.7.1 Figure 3.5.1: MonBus Arbitration Tree..... 109
15.7.2 Arbitration Policy..... 109
15.8 Top-Level MonBus Delivery (monbus_axil4_axil4_group)..... 110
15.9 Integration Notes..... 111
15.9.1 Downstream Connection..... 111
15.9.2 Packet Rate..... 111
16 CXS Network Interface..... 112
16.1 Overview..... 112
16.1.1 Figure 3.6.1: Network Planes..... 112

16.2 Link Profile....113
16.3 Credits....115
16.4 Signal List....116
16.5 Flow Control and the Sink Contract....118
16.6 Monitoring....119
16.7 Routing and Payload Type (Proposed)....120
17 Use Case: Network-to-Memory Transfer....121
17.1 Scenario....121
17.2 Sequence....122
17.2.1 Ordering Rule....123
17.3 Worked Example 1: 77 Bytes at Offset 5....124
17.4 Worked Example 2: 4035 Bytes Crossing a 4 KB Boundary....125
17.5 Errors....126
17.6 Related Chapters....127
18 Use Case: Memory-to-Network Transfer....128
18.1 Scenario....128
18.2 Sequence....129
18.3 Worked Example 1: 77 Bytes at Offset 5....130
18.4 Worked Example 2: 4035 Bytes Crossing a 4 KB Boundary....131
18.5 Errors....132
18.6 Related Chapters....133
19 Use Case: Descriptor Chaining....134
19.1 Overview....134
19.2 Chain Structure....135

19.2.1 Figure 4.3.1: Descriptor Chain Structure....135
19.3 Descriptor Fields for Chaining....138
19.3.1 Relevant Fields....138
19.3.2 Chain Termination....138
19.4 Prefetch Optimization....140
19.4.1 Descriptor Prefetching....140
19.5 Multi-Channel Chaining....141
19.5.1 Figure 4.3.2: Independent Per-Channel Chains....141
19.6 Error During Chain....142
19.6.1 Error Handling....142
19.6.2 Recovery....142
20 Use Case: Multi-Channel Operation....143
20.1 Overview....143
20.2 Channel Independence....144
20.2.1 Figure 4.4.1: Independent Channels....144
20.3 Resource Arbitration....145
20.3.1 Descriptor Port Arbitration....145
20.3.2 Sink/Source Arbitration....145
20.3.3 Figure 4.4.2: Data-Path AXI Arbitration....146
20.4 Channel Isolation....147
20.4.1 Error Isolation....147
20.4.2 Per-Channel Soft Reset....147
20.5 Use Case Examples....148
20.5.1 Mixed Traffic Pattern....148

20.5.2 Bidirectional Transfer....148
20.5.3 Figure 4.4.3: Bidirectional Transfer....149
20.5.4 Load Balancing....149
20.5.5 Figure 4.4.4: Load Balancing by tid....150
20.6 Performance Considerations....151
20.6.1 Bandwidth Sharing....151
20.6.2 Latency Impact....151
20.7 MonBus Events....152
21 Descriptor Format....153
21.1 Overview....153
21.2 Descriptor Layout....154
21.2.1 256-Bit Structure....154
21.3 Field Descriptions....156
21.3.1 Control Bits [195:192]....156
21.3.2 Direction....156
21.3.3 Transfer Length [159:128]....156
21.3.4 Next Pointer [191:160]....156
21.3.5 Address Fields [127:0]....157
21.4 Alignment and Offsets....158
21.4.1 How the offset reaches the bus....158
21.4.2 4 KB boundaries....159
21.4.3 Zero-length descriptors....159
21.4.4 Packet record....159
21.5 Descriptor Opcodes (DATA / CTRL_READ / CTRL_WRITE)....160

21.6 Extended Descriptors (row/col-major addressing)....161
21.6.1 EXT descriptors stay beat-aligned (permanent limitation)....161
21.7 Descriptor Fetch....163
21.8 Alignment Requirements....164
21.9 What Changed from RAPIDS Beats....165
21.10 Descriptor Examples....166
21.10.1 Sink Descriptor (Network to Memory)....166
21.10.2 Source Descriptor (Memory to Network)....166
21.11 Software Construction....167
21.11.1 C Structure Example....167
21.11.2 Descriptor Ring Setup....168
22 Register Map....169
22.1 Overview....169
22.2 Address Space Layout....170
22.3 Base Registers (0x100-0x3FF)....171
22.4 Monitor Registers (+0x800 within each half)....173
22.5 Key Register Fields....174
22.5.1 GLOBAL_CTRL (0x100) - Read/Write....174
22.5.2 SCHED_TIMEOUT_LIMIT (0x208) - Read/Write....174
22.5.3 DESCENG_CONFIG (0x220) - Read/Write....174
22.5.4 Tile Header Status (0x2D0-0x2D8) - byte-granular RAPIDS with TILE_HDR = 1....175
22.6 Channel Kick Sequence....177
22.7 Address Decoding....178

23 16-Channel Register Map....179
24 Initialization Sequence....181
24.1 Overview....181
24.2 Boot Sequence....182
24.2.1 Figure 5.3.1: Bring-Up Sequence....182
24.3 What the Hardware Requires....183
24.4 Detailed Steps....184
24.4.1 Step 1: Verify Hardware Presence....184
24.4.2 Step 2: Optional Reset....184
24.4.3 Step 3: Configure the Engine....184
24.4.4 Step 4: Optional Monitor and Interrupt Egress....185
24.4.5 Step 5: Build Descriptors....185
24.4.6 Step 6: Enable....186
24.5 Starting Transfers....187
24.5.1 Kick a Channel....187
24.6 Completion and Interrupt Handling....188
24.7 Reset and Recovery....189
24.7.1 Per-Channel Reset....189
24.7.2 Global Reset....189
25 Error Handling....190
25.1 Overview....190
25.2 Error Classes....191
25.3 The Byte-Granular Error: Packet Length Mismatch....194
25.3.1 Contract....194

25.3.2 Exposure....194
25.3.3 How it Clears....195
25.3.4 Prevention....195
25.4 Shared Error Classes....196
25.4.1 Invalid or Unfetchable Descriptor....196
25.4.2 Data-Path AXI Errors....196
25.4.3 Control Engine Errors....196
25.4.4 Write-Progress Timeout....196
25.5 Not Errors....198
25.5.1 Packet Record Queue Full....198
25.5.2 Sink Beat With No Packet Record....198
25.5.3 The 4 KB Burst Cap....198
25.6 Error Reporting....199
25.6.1 Registers....199
25.6.2 MonBus....199
25.7 Recovery....200
25.7.1 Channel Reset....200
25.7.2 What the Reset Does to Work in Flight....200
25.8 Software Checklist....203
25.9 Related Chapters....204
26 Throughput....205
26.1 Theoretical Maximum....206
26.2 Byte Efficiency....207
26.2.1 4 KB Boundaries....207

26.2.2 Head-of-Line Delay on the Sink Stream....208
26.3 Measured Results....209
26.3.1 Byte Campaign (Correctness)....209
26.3.2 Shared Engines on the Board....210
26.3.3 RAPIDS Beats Performance Report....211
26.3.4 Using the Beats Numbers....213
26.4 Throughput Limiting Factors....214
26.5 Related Chapters....215
27 Latency Characteristics....216
27.1 Latency Components....217
27.1.1 Kick-to-First-Beat Breakdown....217
27.1.2 Total Latency Formula....217
27.2 First-Beat Latency....219
27.3 Control-Descriptor Latency....220
27.4 Latency Variability....221
27.4.1 Sources of Variability....221
27.4.2 Worst-Case Latency....221
27.5 Latency Optimization....222
27.5.1 Software Recommendations....222
27.5.2 Hardware Configuration....222
28 Resource Estimates....223
28.1 Build Configuration....224
28.2 FPGA Resource Summary....225
28.3 Timing....226

28.4 Board Result..... 227
28.5 Shared Engines: Before and After..... 228
28.6 Comparison with the Beats Build..... 230
28.7 What Depends on What..... 231
28.8 Related Chapters..... 232

List of Figures

Figure 1.1.1: RAPIDS in Its System....38
Figure 1.3.1: System Integration....48
Figure 2.1.1: RAPIDS Block Diagram....52
Figure 2.2.1: Sink Byte Placement....61
Figure 2.2.2: Packet Record Flow....62
Figure 2.3.1: Per-Channel and Shared Resources....66
Figure 2.3.2: Channel Arbitration onto a Shared AXI Port....69
Figure 3.1.1: External Interfaces....73
Figure 3.5.1: MonBus Arbitration Tree....109
Figure 3.6.1: Network Planes....112
Figure 4.3.1: Descriptor Chain Structure....134
Figure 4.3.2: Independent Per-Channel Chains....140
Figure 4.4.1: Independent Channels....143
Figure 4.4.2: Data-Path AXI Arbitration....145
Figure 4.4.3: Bidirectional Transfer....148
Figure 4.4.4: Load Balancing by tid....149
Figure 5.3.1: Bring-Up Sequence....181

List of Tables

Table 1: Target Audience....27
Table 2: Byte and Beat Terms....28
Table 3: Reference Documents....29
Table 4: Document Revision History....30
Table 5: Acronym Definitions....34
Table 6: Term Definitions....35
Table 7: Signal Naming Conventions....36
Table 8: Target Applications....39
Table 9: Byte Design Additions....40
Table 10: Data Path Summary....43
Table 11: SRAM Buffer Parameters....45
Table 12: AXI Optimization Features....45
Table 13: Error Handling Summary....46
Table 14: AXI Memory Interface Requirements....49
Table 15: AXIS Network Interface....49
Table 16: Configuration Methods....50
Table 17: Clock Domains....51
Table 18: Reset Signals....51
Table 19: Scheduler Array Components....54
Table 20: Sink Path Components....55
Table 21: Source Path Components....56
Table 22: Internal Bus Summary....58
Table 23: Byte Placement Rules....60
Table 24: Resource Isolation....64
Table 25: Per-Channel Resources....67
Table 26: Shared Resources....68
Table 27: Channel Isolation Features....72
Table 28: External Interface Summary....74
Table 29: Clock and Reset Signals....75
Table 30: Interface Feature Matrix....76
Table 31: Signal Count Summary....77
Table 32: Where byte granularity is carried on AXI4....78
Table 33: Sink AXI Configuration....80
Table 34: Sink AXI Signals....80
Table 35: WSTRB per beat....81
Table 36: Source AXI Configuration....83

Table 37: Source AXI Signals....83
Table 38: Address Alignment Requirements....85
Table 39: AXI Response Handling....85
Table 40: Byte Semantics of the RAPIDS Streams....87
Table 41: Sink AXIS Configuration....88
Table 42: Sink AXIS Signals....88
Table 43: Reasons the Sink Deasserts TREADY....90
Table 44: Source AXIS Configuration....91
Table 45: Source AXIS Signals....91
Table 46: Egress Packet Contract....92
Table 47: TLAST Usage....93
Table 48: TSTRB Examples....93
Table 49: APB Address Routing....96
Table 50: APB Slave Signals....96
Table 51: Status Signals....98
Table 52: Scheduler State Encoding....98
Table 53: Key Configuration Registers....99
Table 54: MonBus Signals....103
Table 55: MonBus Packet Types....104
Table 56: RAPIDS Agent IDs....105
Table 57: Scheduler Event Codes....107
Table 58: Descriptor Engine Event Codes....107
Table 59: Top-Level MonBus AXI-Lite Group....110
Table 60: MonBus Packet Rates....111
Table 61: CXS Link Profile....113
Table 62: CXS Top Parameters....114
Table 63: CXS Link Signals....115
Table 64: CXS Status Signals....116
Table 65: Network-to-Memory Sequence....121
Table 66: Example 1 Quantities....123
Table 67: Example 1 Write Beats....123
Table 68: Example 2 Quantities....124
Table 69: Example 2 Write Bursts....124
Table 70: Network-to-Memory Error Cases....125
Table 71: Memory-to-Network Sequence....128
Table 72: Example 1 Quantities....129
Table 73: Example 1 Egress Steps....129
Table 74: Example 2 Quantities....130

Table 75: Memory-to-Network Error Cases....131
Table 76: Chaining-Related Descriptor Fields....137
Table 77: Mixed Traffic Channel Assignment....147
Table 78: Bandwidth Distribution....150
Table 79: Arbitration Latency....150
Table 80: Control Bits....155
Table 81: Address Field Usage....156
Table 82: Beat Count Examples (BYTE_LANES = 32)....157
Table 83: Descriptor Opcodes....159
Table 84: Address and Length Alignment....163
Table 85: Descriptor Differences, RAPIDS Beats to RAPIDS....164
Table 86: Base Registers....170
Table 87: Monitor Registers (+0x800 within each half; SRC at 0x0800, SNK at 0x1800)....172
Table 88: GLOBAL_CTRL Register....173
Table 89: SCHED_TIMEOUT_LIMIT Register....173
Table 90: DESCENG_CONFIG Register....173
Table 91: Tile Header Status Registers....174
Table 92: Register Differences, 8- and 16-Channel Maps....178
Table 93: Conditions for a Channel to Run....182
Table 94: Error Classes....190
Table 95: Length Mismatch Cases....193
Table 96: Where the Length Mismatch Appears....193
Table 97: Timeout Configuration....195
Table 98: Error Registers....198
Table 99: Channel Reset: Handling of In-Flight Work....199
Table 100: Software Checklist....202
Table 101: Theoretical Maximum (256-bit Build)....205
Table 102: Beats and Efficiency for Small and Unaligned Transfers....206
Table 103: Sources of Measured Figures....208
Table 104: Byte Campaign on the Board (7 of 7 Pass)....208
Table 105: Board Results, SHARED_ENGINES 0 and 1....209
Table 106: Per-Point Comparison....209
Table 107: RAPIDS Beats Build, 8 Channels at 128 KB per Channel....211
Table 108: RAPIDS Beats Build, Transfer Size Sweep at 8 Channels....211
Table 109: RAPIDS Beats Build, Channel Scaling at 4096 Beats per Channel....211
Table 110: RAPIDS Beats Build, Burst Length Sweep at 8 Channels and 1024 Beats 212

Table 111: Throughput Limiting Factors....213
Table 112: Board Build Configuration....223
Table 113: Post-Route Utilization, 8 Channels, 256 Bits....224
Table 114: Timing at 100 MHz....225
Table 115: Post-Route Utilization, SHARED_ENGINES 0 and 1....227
Table 116: Beats and Byte-Granular Builds....229
Table 117: Scaling Notes....230

1 Rapids Has Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

Document Title: RAPIDS Hardware Architecture Specification (HAS) **Document Number:** RAPIDS-HAS-002 **Version:** 0.9 **Date:** 2026-10-20 **Status:** Draft

2.1 Purpose

This Hardware Architecture Specification (HAS) defines the external interfaces, system integration requirements, and high-level architecture of RAPIDS, the byte-granular descriptor-driven DMA engine. RAPIDS counts bytes, not beats: a descriptor names a byte length and byte addresses, the write master carries byte enables, and the AXI-Stream ports carry packed bytes with a partial last beat.

RAPIDS Beats, the beat-granular design that came first, remains in the repository and remains supported. Its specification is the RAPIDS Beats HAS. The two designs share most of their external behavior, so this document links the RAPIDS Beats chapters that apply unchanged and rewrites only the chapters where the byte-granular design differs.

2.2 Scope

This document covers:

- External interface specifications (AXI4, AXIS, APB, MonBus) as they apply to the byte-granular design
- System-level block diagrams and data flows, including the byte placement rules
- Programming model and descriptor format (length in bytes)
- Use cases and operational sequences with unaligned transfers
- Performance characteristics and constraints

This document does NOT cover:

- Internal module architecture (see the RAPIDS MAS)
- RTL implementation details (see the RAPIDS MAS)
- The RAPIDS Beats design (see the RAPIDS Beats HAS)
- Verification test plans
- Physical design constraints

2.3 How This Book Relates to RAPIDS Beats

Chapters that are identical for both designs are not copied. The index links them from the RAPIDS Beats HAS, and the table of contents marks each one “(shared with RAPIDS Beats)”. When such a chapter names a module, the byte-granular module is the same name without the `_beats` suffix, for example `rapids_top` for `rapids_beats_top`.

Where a shared chapter says a descriptor length is a number of beats, read it as a number of bytes for RAPIDS. Every chapter that carries that distinction is rewritten in this book, so a reader following the RAPIDS chapters in order will not meet the beat unit except in the comparison tables.

2.4 Audience

Target Audience

Role	Relevance
System Architect	Primary - system integration
Hardware Integration Engineer	Primary - interface connection
Software Engineer	Primary - driver development, descriptor construction
Verification Engineer	Reference - interface protocols
RTL Designer	Reference - external constraints

2.5 Document Conventions

2.5.1 Requirement Levels

Term	Meaning
SHALL	Mandatory requirement
SHOULD	Recommended but not mandatory
MAY	Optional feature

2.5.2 Signal Directions

- **Input:** Signal driven by external logic into RAPIDS
- **Output:** Signal driven by RAPIDS to external logic

2.5.3 Byte and Beat Terms

Byte and Beat Terms

Term	Meaning
Beat	One transfer on a data bus, `DATA_WIDTH/8` bytes wide (`BYTE_LANES`)
Offset	The low `log2(BYTE_LANES)` bits of a byte address: the lane where a transfer starts
Packet	The bytes of one descriptor as they appear on an AXI-Stream port, delimited by TLAST
Packet record	The `{bytes, offset}` pair the scheduler hands each data path for one descriptor

2.5.4 Timing Diagrams

Timing diagrams use WaveDrom format with the following conventions:

- `p` = Positive clock edge
- `0/1` = Logic low/high
- `x` = Unknown/don't care
- `=` = Data value (with label)
- `.` = Previous value continues

2.6 References

Reference Documents

Document	Description
RAPIDS MAS	Micro-architecture Specification for the byte-granular design
RAPIDS Beats HAS	Architecture specification for the beat-granular design; source of every shared chapter
ARM AMBA AXI4 Specification	AXI4 protocol reference
ARM AMBA AXI-Stream Specification	AXIS protocol reference
Arm AMBA CXS Protocol Specification (IHI 0079, Issue D)	CXS protocol reference (CXS tops)
RAPIDS PRD	Product Requirements Document

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Revision History

Document Revision History

Version	Date	Author	Changes
0.7	2026-10-17	RTL Design Sherpa	Shared engines (`SHARED_ENGINES`, default 1): key features, block diagram components and hierarchy, and the whole-unit diagram; 8- and 16-channel tops; Resource Estimates gains the measured before-and-after builds and Throughput the board comparison of the two
0.6	2026-10-17	RTL Design Sherpa	Tile packet header (`TILE_HDR`): descriptor `ptype` / `op` / destination fields implemented; new 16-Channel Register Map section (`rapids_16ch_top`, `KICK_ENABLE` at 0x080); tile status registers (shared Register Map)
0.5	2026-10-16	RTL Design Sherpa	Routing and payload-type PROPOSAL (not implemented): the tile packet header is

Version	Date	Author	Changes
			defined in the compute-engine tile's TILE_SPEC Section 11; CXS only transports it. Descriptor Format points at the proposed fields
0.4	2026-10-16	RTL Design Sherpa	Network planes and CXS: links the shared CXS Network Interface chapter; block diagram, hierarchy, system context and key features updated for `rapids_top` / `rapids_cxs_top`; all block diagrams updated
0.3	2026-10-16	RTL Design Sherpa	`SRAM_DEPTH` stated as beats per channel, per direction
0.2	2026-09-30	RTL Design Sherpa	Channel reset recovers every error class; the aresetn-only caveats are removed from Error Handling
0.1	2026-09-30	RTL Design Sherpa	Initial release: the byte-granular RAPIDS, split from the RAPIDS Beats HAS
0.8	2026-10-20	RTL Design Sherpa	Block diagrams redrawn to the repo

Version	Date	Author	Changes
			element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
0.9	2026-10-20	RTL Design Sherpa	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

3.1 Change Summary

3.1.1 Version 0.2 (2026-09-30)

Channel reset closes the data-path error gap

- The per-channel reset now reaches the AXI read and write engines and both byte data paths. The data read and write response errors and the sink packet length error clear on a channel reset, and the channel takes a good descriptor afterwards. Error Handling no longer tells software to use `aresetn` for these.
- Error Handling gains the list of what a channel reset does to work in flight, including the sink tail discard and the unterminated source packet.
- Head-of-line blocking on a shared sink stream is unchanged and stays documented.

3.1.2 Version 0.1 (2026-09-30)

Initial Release

- The byte-granular RAPIDS (rapids TASK-019) is the product design. The RAPIDS Beats design was its stepping stone and keeps its own book, the RAPIDS Beats HAS.
- New chapters where the byte design differs: Product Overview, Key Features, System Context, Block Diagram, Data Flow, AXI4 Master Interface, AXIS Interface, both use cases, Descriptor Format, Error Handling, Throughput and Resource Estimates.
- Chapters unchanged from RAPIDS Beats are linked from the RAPIDS Beats HAS rather than copied, so a correction there reaches both books.
- Permanent limitation recorded: TYPE=EXT (row and column striding) descriptors stay beat-aligned. Linear descriptors may start at any byte.

4 Acronyms and Definitions

4.1 Acronyms

Acronym Definitions

Acronym	Definition
AXI	Advanced eXtensible Interface
AXIS	AXI-Stream
APB	Advanced Peripheral Bus
DMA	Direct Memory Access
FIFO	First-In First-Out
FSM	Finite State Machine
HAS	Hardware Architecture Specification
MAS	Module Architecture Specification
MonBus	Monitor Bus
RAPIDS	Rapid AXI Programmable In-band Descriptor System
SRAM	Static Random Access Memory

4.2 Definitions

Term Definitions

Term	Definition
Beat	Single data transfer unit (512 bits = 64 bytes in default configuration)
Channel	Independent DMA context with dedicated descriptor processing
Descriptor	256-bit control structure defining a single DMA transfer
Descriptor Chain	Linked list of descriptors for multi-transfer operations
Sink Path	Data flow from network (AXIS slave) to memory (AXI write)
Source Path	Data flow from memory (AXI read) to network (AXIS master)
Fill	Operation writing data into SRAM buffer (sink path input)
Drain	Operation reading data from SRAM buffer (source path output)

4.3 Signal Naming Conventions

Signal Naming Conventions

Prefix/Suffix	Meaning
`m_axi_*`	AXI Master signals
`s_axis_*`	AXIS Slave signals
`m_axis_*`	AXIS Master signals
`cfg_*`	Configuration signals
`*_valid`	Handshake valid signal
`*_ready`	Handshake ready signal
`*_n`	Active-low signal

5 Product Overview

5.1 Introduction

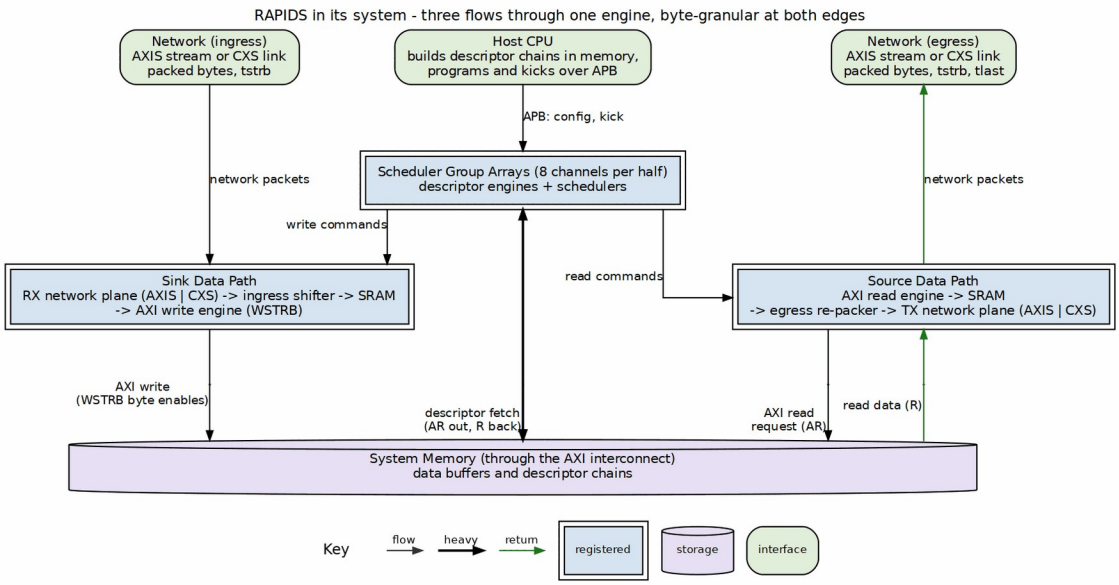
RAPIDS is a high-performance network-to-memory accelerator for descriptor-based data movement. It bridges network interfaces (AXI-Stream) with system memory (AXI4), moving data without CPU intervention.

RAPIDS is byte-granular. A descriptor gives a length in bytes and byte addresses. The sink writes exactly the bytes the network sent, using byte enables on the AXI4 write data channel. The source emits exactly the bytes the descriptor names, packed from lane 0 of the AXI-Stream, with a partial last beat when the length is not a multiple of the bus width. A transfer may begin at any byte address.

The earlier RAPIDS Beats design moved whole beats only. It was the stepping stone to this design, it stays supported, and its specification is the RAPIDS Beats HAS.

5.2 System Context

5.2.1 Figure 1.1.1: RAPIDS in Its System



RAPIDS in Its System

Source: ch01_product_context.dot

5.3 Target Applications

Target Applications

Application	Description	Key Requirements
Network Offload	Receive packets from the network, store to memory	Low latency, arbitrary packet lengths
Scatter-Gather DMA	Multi-descriptor data movement	Descriptor chaining
Protocol Processing	Header and payload separation at byte boundaries	Any byte offset, one packet per descriptor
Streaming Accelerator	Front-end for compute engines	Continuous data flow

5.4 What the Byte Design Adds

Byte Design Additions

Capability	RAPIDS Beats	RAPIDS
Transfer unit	Full beats only	Bytes
Address alignment	Pre-aligned required	Any byte for linear descriptors
Sink write	Every lane written	Byte enables mark the bytes written
Source stream	Full beats	Packed bytes, partial last beat, TLAST per descriptor
Flow control	Backpressure-based	Backpressure-based

Flow control is unchanged: both designs rely on backpressure, and neither has credit management.

5.4.1 Key Capabilities

1. **8 Independent Channels** - Concurrent transfers with dedicated resources
2. **Byte-Granular Transfers** - Length in bytes, start at any byte address
3. **Descriptor Chaining** - Automatic next-descriptor fetch
4. **Dual Data Paths** - Simultaneous sink (write) and source (read)
5. **Monitor Integration** - Real-time event reporting via MonBus
6. **Error Detection** - AXI error propagation, and a packet-length check on the sink
7. **Swappable Network Plane** - AXI-Stream or CXS (Arm IHI 0079 Issue D) at the top, same core

5.5 Permanent Limitation

TYPE=EXT descriptors, which stride across rows and columns, stay beat-aligned by design. Their addresses SHALL be beat-aligned and their lengths SHALL be a multiple of the beat size. This is a permanent property of the design, not a phase of a schedule. Linear descriptors carry no such restriction. See the Descriptor Format chapter.

6 Key Features

6.1 Feature Summary

6.1.1 Multi-Channel Architecture

- **8 or 16 Independent Channels** per direction, each with its own scheduler. The 16-channel tops use the 16-channel register map.
- **Per-Channel State Machines** for concurrent operation
- **Shared Engines:** each direction has one descriptor engine, one control-read engine and one control-write engine for all its channels (`SHARED_ENGINES = 1`, the default). Each engine picks channels round-robin and owns its AXI master. `SHARED_ENGINES = 0` builds one of each per channel instead. On the Genesys 2 board build, sharing saves 20% of the RAPIDS LUTs and 37% of its flip-flops, and the 117-point board profile measures the same throughput on both builds (geometric-mean change under 0.2%).
- **Channel Isolation** prevents cross-channel interference

6.1.2 Byte-Granular Data Movement

- **Length in Bytes.** The descriptor length field counts bytes. A length of zero moves nothing.
- **Any Start Byte.** Source and destination byte addresses may have any offset within a beat (linear descriptors).
- **Byte Enables on the Write Master.** WSTRB is exactly the set of bytes the descriptor writes. The first beat of a transfer strobes from the start offset, the last beat strobes up to the final byte, and every beat between strobes all lanes.
- **Packed AXI-Stream.** The sink accepts packed bytes with a contiguous TSTRB on the last beat. The source emits packed bytes, contiguous TSTRB, and TLAST on the last beat of each descriptor.
- **One Packet per Descriptor.** The source ends every descriptor with TLAST. The sink expects one TLAST-delimited packet per descriptor and checks its byte count against the descriptor length.
- **Beat-Aligned Bus Traffic.** AXI addresses are beat-aligned and no burst crosses a 4 KB boundary. The byte offset never reaches an AXI address: the sink carries it in WSTRB and the source drops it at the egress.

6.1.3 Descriptor-Based Control

- **256-bit Descriptor Format** with all transfer parameters
- **Automatic Chaining** follows next_descriptor_ptr links
- **Last Descriptor Flag** terminates chain processing
- **IRQ Generation** via gen_irq descriptor flag
- **Control Descriptors** (`CTRL_READ` consumer gate, `CTRL_WRITE` producer doorbell) for in-memory producer and consumer synchronization without moving payload. They are unchanged from RAPIDS Beats; see the Descriptor Format chapter and the shared Descriptor Chaining chapter.

6.1.4 High-Performance Data Paths

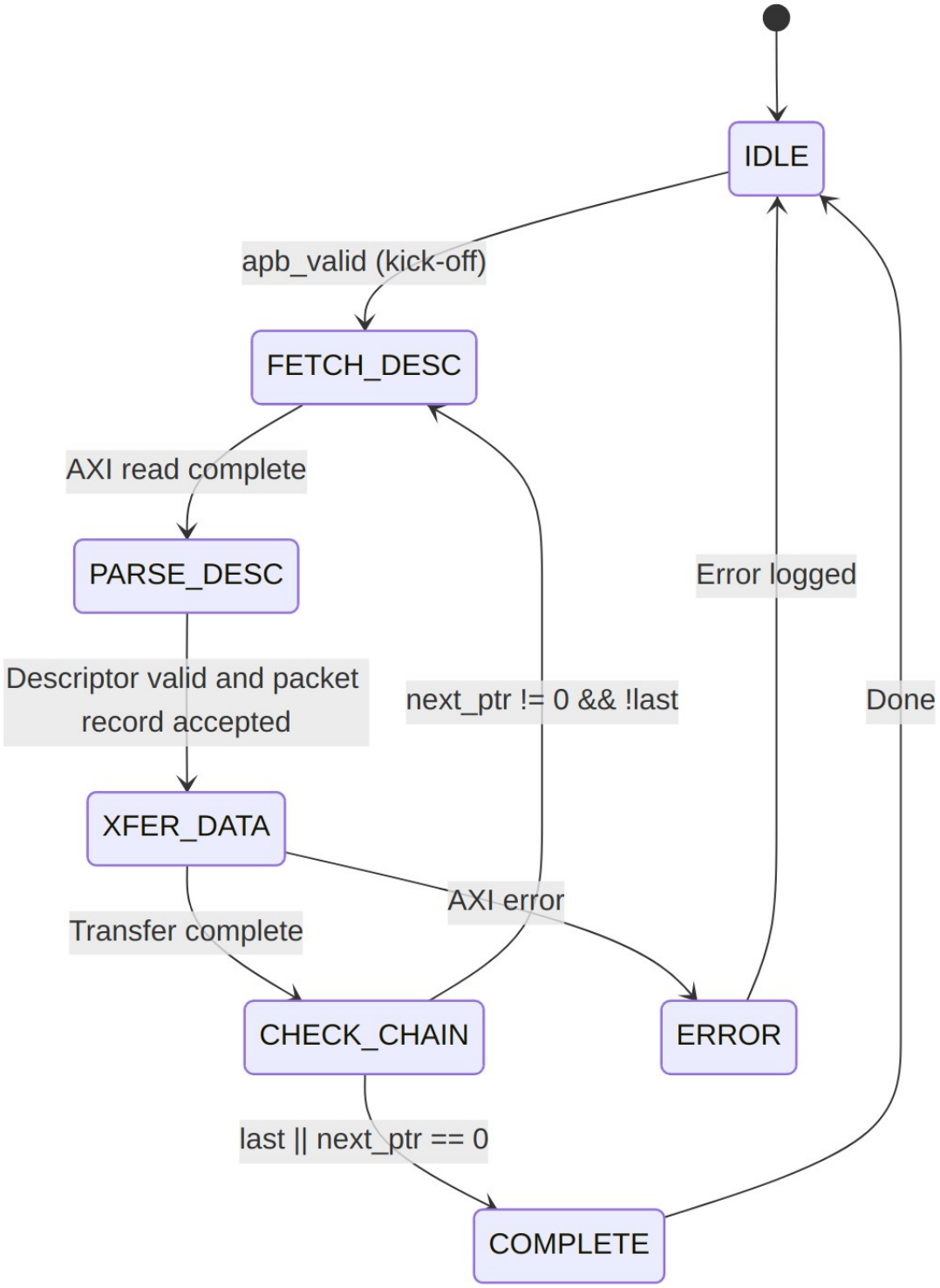
Data Path Summary

Path	Direction	Bandwidth	Features
Sink	Network to Memory	`DATA_WIDTH` per clock	Ingress byte shifter, SRAM buffering, AXI burst writes with byte enables
Source	Memory to Network	`DATA_WIDTH` per clock	AXI burst reads, SRAM buffering, egress byte re-packer

6.1.5 Monitoring and Debug

- **64-bit MonBus Packets** for all significant events
- **State Transition Reports** track FSM activity
- **Error Event Codes** for fault diagnosis
- **Performance Metrics** for throughput analysis

6.2 Feature Details



Descriptor Processing

The scheduler leaves the descriptor-fetch state only when every data path the descriptor uses has room for its packet record. A full record queue holds the channel there; it is backpressure, not an error.

6.2.1 SRAM Buffering

Each channel has its own buffer in each data path (sink and source), `SRAM_DEPTH` beats deep:

SRAM Buffer Parameters

Parameter	Default	Description
`SRAM_DEPTH`	4096	Beats per channel, per direction (parameter of `rapids_top`); one beat is one `DATA_WIDTH` entry
Source entry	`DATA_WIDTH` bits	Data only
Sink entry	`DATA_WIDTH + DATA_WIDTH/8` bits	Data plus one byte-enable bit per lane

The buffer holds `SRAM_DEPTH x DATA_WIDTH/8` bytes per channel per direction. At the defaults (4096 beats, 512 bits) that is 256 KB, a simulation size. The Genesys 2 builds use 128 beats of 32 bytes, 4 KB per channel.

Buffer Operation:

- **Sink Path:** Fill from AXIS (from the RX network plane, AXIS or CXS) through the ingress shifter, drain to the AXI write engine
- **Source Path:** Fill from the AXI read engine, drain through the egress re-packer to AXIS (to the TX network plane, AXIS or CXS)
- **Flow Control:** Backpressure when the buffer is full or empty

6.2.2 AXI Burst Optimization

AXI Optimization Features

Feature	Implementation
Burst Length	Up to 256 beats (AXI4 maximum)
Outstanding Transactions	Configurable
Address Alignment	Beat-aligned AxADDR; the byte offset rides in WSTRB or is dropped at the egress
4KB Boundary	Every burst is capped at the boundary

6.2.3 Error Handling

Error Handling Summary

Error Type	Detection	Response
AXI SLVERR	Response check	Transfer aborted
AXI DECERR	Response check	Transfer aborted
Timeout	Watchdog timer	Recoverable error
Packet length mismatch	Sink counts strobed bytes at TLAST	Sticky per-channel write error

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

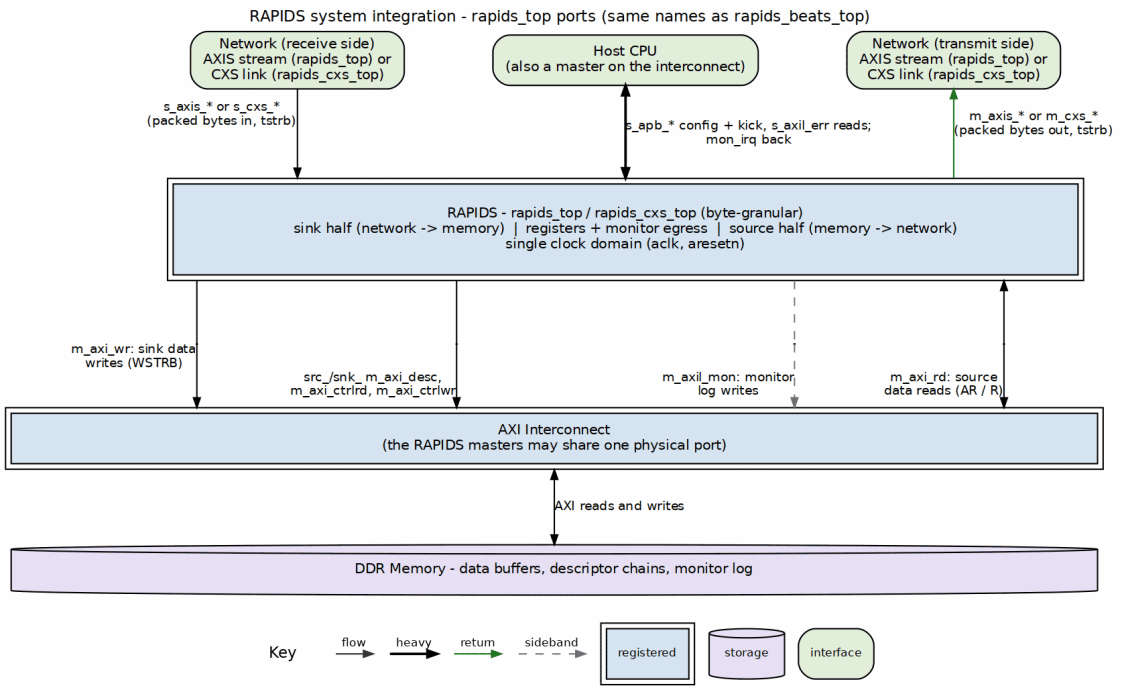
7 System Context

7.1 Integration Overview

RAPIDS integrates into a system as a DMA engine bridging network interfaces and system memory. This chapter describes the external connections and the system-level requirements that differ from RAPIDS Beats. Everything not mentioned here matches the RAPIDS Beats System Context chapter.

7.2 System Block Diagram

7.2.1 Figure 1.3.1: System Integration



System Integration

Source: ch01_system_integration.dot

7.3 Interface Connections

7.3.1 Memory Interface

RAPIDS reaches system memory through the AXI4 masters below. Each half (`src_`, `snk_`) has its own descriptor master and its own two control masters (`m_axi_ctrlrd`, `m_axi_ctrlwr`). With monitors built, an AXI-Lite master (`m_axil_mon`) also writes the monitor log:

AXI Memory Interface Requirements

Port	Access Pattern	Typical Bandwidth
Descriptor AXI	Random read (256-bit)	Low (<100 MB/s)
Sink AXI	Sequential write with byte enables	High (up to interface max)
Source AXI	Sequential read	High (up to interface max)

Interconnect Requirements:

- All of these ports MAY share a single physical AXI port with arbitration
- The descriptor port has the lowest bandwidth requirement
- Sink and Source ports should have balanced bandwidth allocation
- The memory system SHALL honor WSTRB on the sink write master. A slave that ignores byte enables corrupts the bytes outside a partial beat. Every AXI4-conformant slave honors them.

7.3.2 Network Interface

The network side is a swappable plane. The AXIS tops use standard AXI-Stream, below; the CXS tops (`*_cxs_top`) replace the two streams with one CXS link per direction, with `CXSSRCID` carrying the channel (see the CXS Network Interface chapter):

AXIS Network Interface

Interface	Role	Signals
Sink AXIS	Receives data from the network	TDATA, TSTRB, TLAST, TID, TVALID, TREADY
Source AXIS	Sends data to the network	TDATA, TSTRB, TLAST, TID, TVALID, TREADY

Protocol Notes:

- Byte-granular: the byte lanes are strobed by TSTRB, and the strobed lanes of a beat SHALL be contiguous from lane 0. RAPIDS does not use TKEEP. Only the last beat of a packet may have a partial TSTRB.
- A packet is the bytes of one descriptor, delimited by TLAST. The source ends every descriptor with TLAST. The sink expects its packet to match the descriptor's byte length.
- TID selects the channel. TDEST and TUSER are available for sideband information.
- The sink holds TREADY low for a beat until the packet record of that beat's channel (TID) exists, which is after that channel's descriptor has been fetched. TREADY is one signal, so a waiting beat also holds up the beats behind it on the same stream. Software SHOULD issue a channel's descriptor before its packet arrives, or stream one channel at a time.

7.3.3 Configuration Interface

Software configures RAPIDS through register writes:

Configuration Methods

Method	Interface	Usage
Direct Write	Memory-mapped	Register configuration
Descriptor Kick	APB-style	Start descriptor processing

7.4 Clock and Reset

7.4.1 Clock Domains

Clock Domains

Clock	Frequency	Usage
`aclk`	100-500 MHz	All RAPIDS logic

Note: RAPIDS uses a single clock domain. Crossing into other frequency domains is handled externally.

7.4.2 Reset Requirements

Reset Signals

Signal	Type	Description
`aresetn`	Active-low	Async assert, sync deassert

Reset Sequence:

1. Assert `aresetn` low for minimum 4 clock cycles
2. Ensure all AXI interfaces are idle before reset
3. Release reset synchronously to `aclk` rising edge
4. Wait 16 clock cycles before initiating transfers

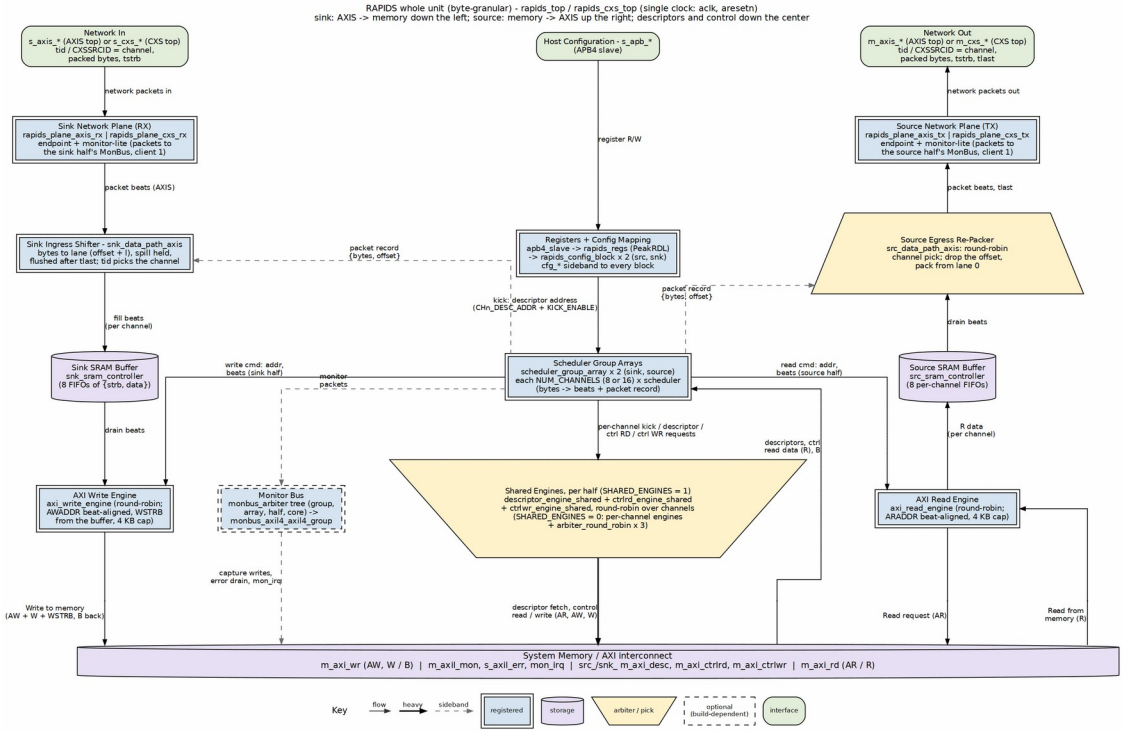
Reset clears every packet record, every ingress spill, and every sticky write error.

8 Block Diagram

8.1 Top-Level Architecture

RAPIDS has the same block structure as RAPIDS Beats: a scheduler group array per direction, a data path per direction, and a top level that adds the registers, configuration mapping and monitoring. What the byte design adds sits at the edges of the two data paths: an ingress shifter on the sink, an egress re-packer on the source, and a packet record passed from each scheduler to its data path.

8.1.1 Figure 2.1.1: RAPIDS Block Diagram



RAPIDS block diagram

Source: 01_byte_block_diagram.dot

8.2 Top-Level Integration (rapids_top, rapids_cxs_top and the 8- and 16-channel tops)

`rapids_core` is wrapped by `rapids_top`, which adds the software register interface, configuration mapping, and monitoring infrastructure. The integration is identical to `rapids_beats_top`: a single APB slave feeds a command/response router into the `rapids_regs` register block, `rapids_config_block` translates the register outputs into the `cfg_*` signals, and a `monbus_axil4_axil4_group` delivers the merged monitor stream to an AXI-Lite error-drain slave, an AXI-Lite capture master, and a `mon_irq` interrupt. The register map, the APB interface and the MonBus interface are shared with RAPIDS Beats; see the linked chapters.

The top-level ports have the same names as `rapids_beats_top`. The only port-level differences are the byte enables: the sink write master's `m_axi_wr_wstrb` is driven from the buffered byte enables instead of all ones, and the two AXI-Stream ports use `tstrb` as a real byte mask.

The network endpoints sit in two network planes outside the core, exactly as in RAPIDS Beats: `rapids_top` uses the AXIS planes and `rapids_cxs_top` the CXS planes, both generated from `rapids_top_base`. The generator also emits `rapids_8ch_top`, `rapids_16ch_top` and their CXS twins, which fix `NUM_CHANNELS` and compile the matching register map. The CXS planes carry `tstrb` in `CXSUSER`, so partial last beats and unaligned starts cross the CXS link intact. See the shared CXS Network Interface chapter.

8.3 Component Summary

8.3.1 Scheduler Group Array

The scheduler group array manages `NUM_CHANNELS` (8 or 16) independent channels per direction. `SHARED_ENGINES` selects how its engines are built:

Scheduler Array Components

Component	`SHARED_ENGINES = 1` (default)	`SHARED_ENGINES = 0`	Purpose
`scheduler`	`NUM_CHANNELS`	`NUM_CHANNELS`	Transfer coordination per channel; converts bytes to beats, issues the packet record
Descriptor engine	1 `descriptor_engine_shared`	`NUM_CHANNELS` `descriptor_engine`	Descriptor fetch and parse
Control-read engine	1 `ctrlrd_engine_shared`	`NUM_CHANNELS` `ctrlrd_engine`	`CTRL_READ` consumer gate
Control-write engine	1 `ctrlwr_engine_shared`	`NUM_CHANNELS` `ctrlwr_engine`	`CTRL_WRITE` producer doorbel

Component	SHARED_ENGINES = 1 (default)	SHARED_ENGINES = 0	Purpose
			l
AXI-side arbiters	none (each engine owns its master)	3 round-robin	Merge the per-channel masters onto the descriptor, control-read and control-write ports

A shared engine keeps each channel's behaviour: the same kick policy, chaining, prefetch, errors and channel reset. Fetches carry the channel in ARID, so channels overlap on the bus while each stays in order. The only visible difference is that a polling `CTRL_READ` holds the control-read engine, so other channels' control reads wait behind it; data transfers do not. The MAS Shared Engines chapter has the detail.

8.3.2 Sink Data Path

Network-to-memory data flow:

Sink Path Components

Component	Purpose
`snk_data_path_axis`	Ingress: accepts packed bytes, shifts them to `offset + lane`, holds the spill, flushes after TLAST, counts the packet
`snk_sram_controller`	Multi-channel SRAM management; each entry is data plus byte enables
`axi_write_engine`	AXI4 burst write generation; beat-aligned AWADDR, 4 KB cap, WSTRB from the buffer

Component	Purpose
`alloc_ctrl`, `drain_ctrl`	Space and data availability tracking (unchanged)

8.3.3 Source Data Path

Memory-to-network data flow:

Source Path Components

Component	Purpose
`axi_read_engine`	AXI4 burst read generation; beat-aligned ARADDR, 4 KB cap
`src_sram_controller`	Multi-channel SRAM management (data only)
`src_data_path_axis`	Egress: drops the start offset, re-packs from lane 0, sets TSTRB and TLAST
`alloc_ctrl`, `drain_ctrl`	Space and data availability tracking (unchanged)

8.4 Hierarchy

```
rapids_top | rapids_cxs_top   (generated by bin/gen_plane_tops.py)
├── u_base: rapids_top_base   (regs, rapids_config_block x2, monbus
group)
│   └── u_core: rapids_core
├── u_snk_plane: rapids_plane_axis_rx | rapids_plane_cxs_rx
│   └── axis4_slave_monlite | cxs_slave_monlite + cxs_to_axis
└── u_src_plane: rapids_plane_axis_tx | rapids_plane_cxs_tx
    └── axis4_master_monlite | axis_to_cxs + cxs_master_monlite

rapids_core
├── monbus_arbiter
├── rapids_src
│   ├── scheduler_group_array
│   │   ├── scheduler [0..NC-1]                    (SHARED_ENGINES = 1,
default)
│   │   ├── descriptor_engine_shared
│   │   ├── ctrlrd_engine_shared
│   │   ├── ctrlwr_engine_shared
│   │   └── scheduler_group [0..NC-1]              (SHARED_ENGINES = 0
instead)
│   │       ├── scheduler
│   │       ├── descriptor_engine
│   │       ├── ctrlrd_engine
│   │       └── ctrlwr_engine
│   ├── monbus_arbiter (3:1; client 1 = src_plane_mon_*)
│   └── src_data_path_axis
│       └── src_data_path
│           ├── axi_read_engine
│           └── src_sram_controller
└── rapids_snk
    ├── scheduler_group_array (same shape)
    ├── monbus_arbiter (3:1; client 1 = snk_plane_mon_*)
    └── snk_data_path_axis
        └── snk_data_path
            ├── axi_write_engine
            └── snk_sram_controller
```

8.5 Internal Buses

Internal Bus Summary

Bus	Width	Description
Descriptor Bus	256-bit	Parsed descriptor to scheduler
Scheduler Command	Variable	Beat count and byte address to the engines
Packet Record	Bytes count plus offset	One per descriptor and direction, scheduler to data path
Source SRAM Data	`DATA_WIDTH`	Data to and from the source SRAM
Sink SRAM Data	`DATA_WIDTH + DATA_WIDTH/8`	Data plus byte enables
MonBus	64-bit	Monitor packets (aggregated)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

9 Data Flow

9.1 Overview

RAPIDS implements two independent data paths that can operate concurrently:

1. **Sink Path:** Network to Memory (AXIS Slave -> AXI Write)
2. **Source Path:** Memory to Network (AXI Read -> AXIS Master)

The flow is the RAPIDS Beats flow with three additions: a packet record that carries the byte length and offset from the scheduler to the data path, an ingress shifter on the sink, and an egress re-packer on the source.

9.2 Byte Placement Rules

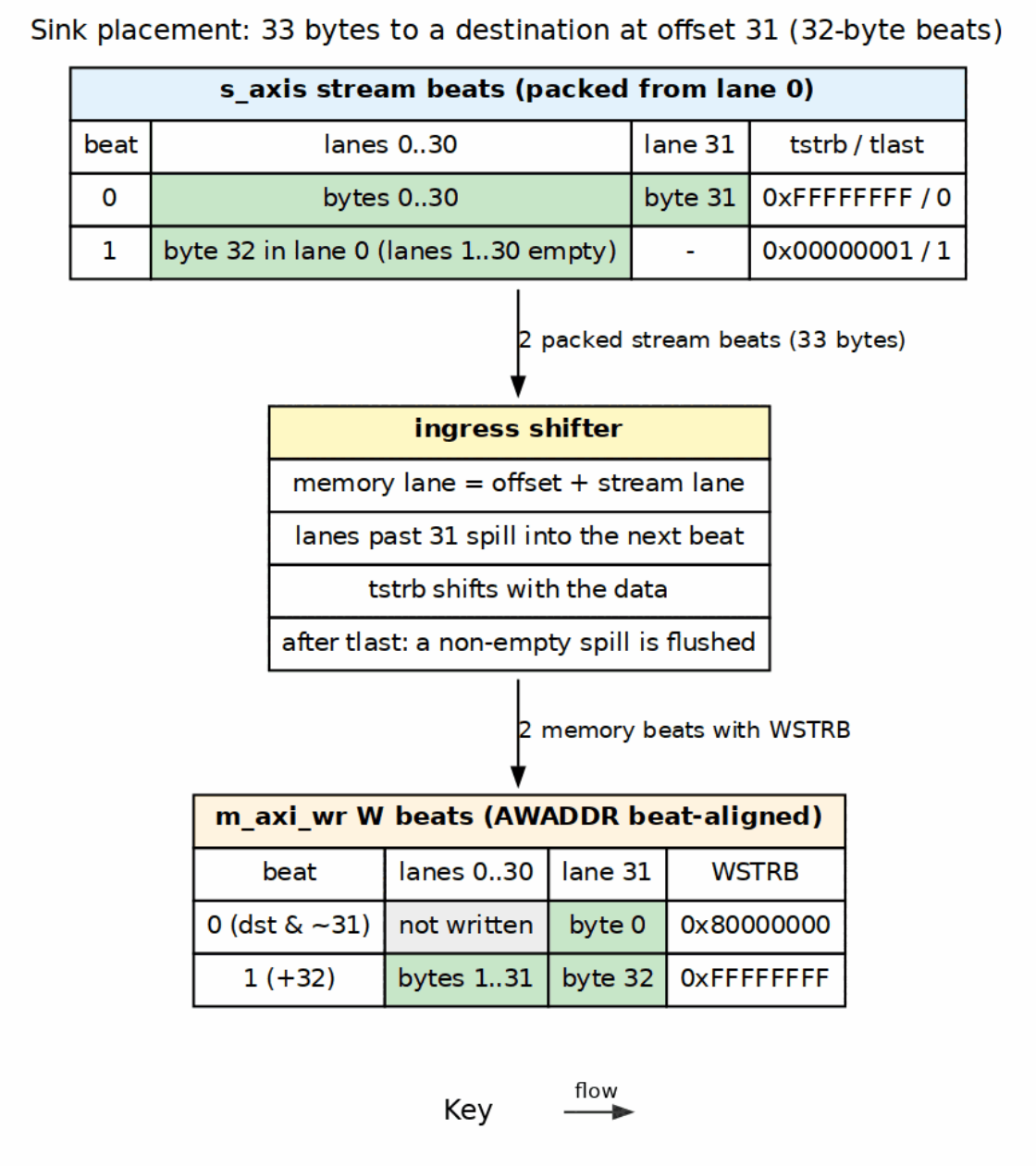
A descriptor names a byte length `L` and a byte address `A`. With `BYTE_LANES` bytes per beat and `OFF = A mod BYTE_LANES`:

Byte Placement Rules

Quantity	Value
Start offset	`OFF` = the low `log2(BYTE_LANES)` bits of `A`
Memory beats	`0` if `L` = 0, else `ceil((OFF + L) / BYTE_LANES)`
First memory beat	address `A` rounded down to a beat boundary; lanes below `OFF` are not written (sink) or dropped (source)
Last memory beat	lanes above the final byte are not written (sink) or dropped (source)
Beats between	all lanes carry data
Later addresses	rounded-down address plus the beats already moved, times `BYTE_LANES`

The stream side is packed: a packet of `L` bytes occupies `ceil(L / BYTE_LANES)` stream beats, all full except possibly the last. The number of memory beats and stream beats can differ by one, because the offset shifts the boundary. The data paths absorb the difference.

9.2.1 Figure 2.2.1: Sink Byte Placement



Sink byte placement

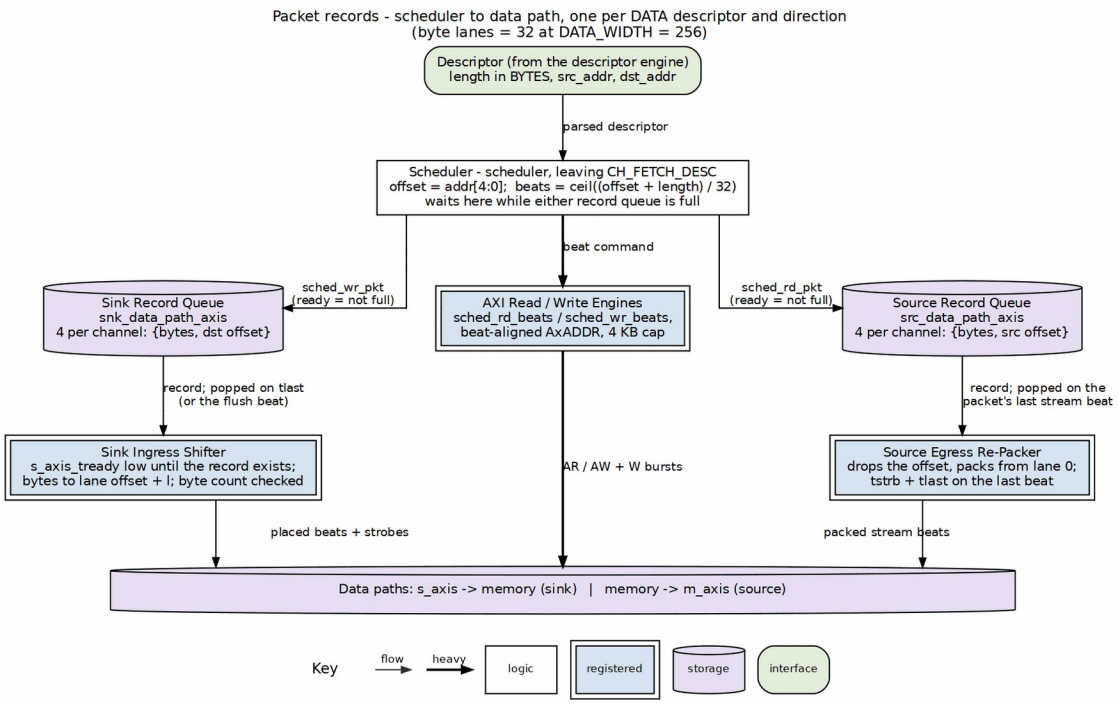
Source: 02_byte_placement.dot

9.3 The Packet Record

For every DATA descriptor, the scheduler hands each data path it uses a record: the byte count and the start offset. The scheduler pulses the record once per descriptor and direction as it leaves the descriptor-fetch state. Each data path keeps a small per-channel queue of records (four deep). If a queue is full, the scheduler stays in descriptor fetch until there is room, so a full queue holds the channel and never drops a record. A zero-length descriptor carries no record.

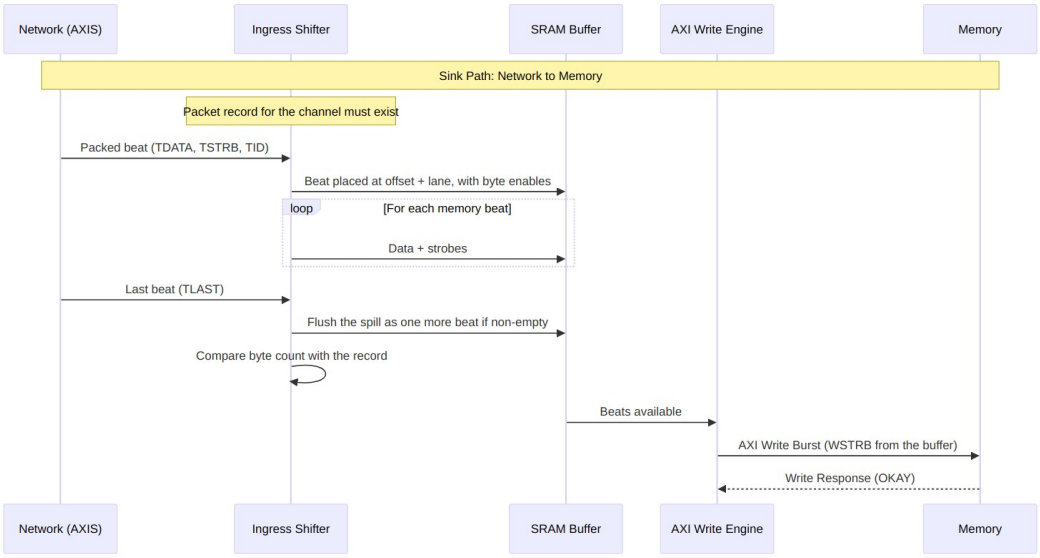
The record is what lets the data path work in bytes while the engines work in beats. The sink ingress uses it to place bytes and to check the packet length. The source egress uses it to drop the offset and to set TSTRB and TLAST.

9.3.1 Figure 2.2.2: Packet Record Flow



Packet record flow

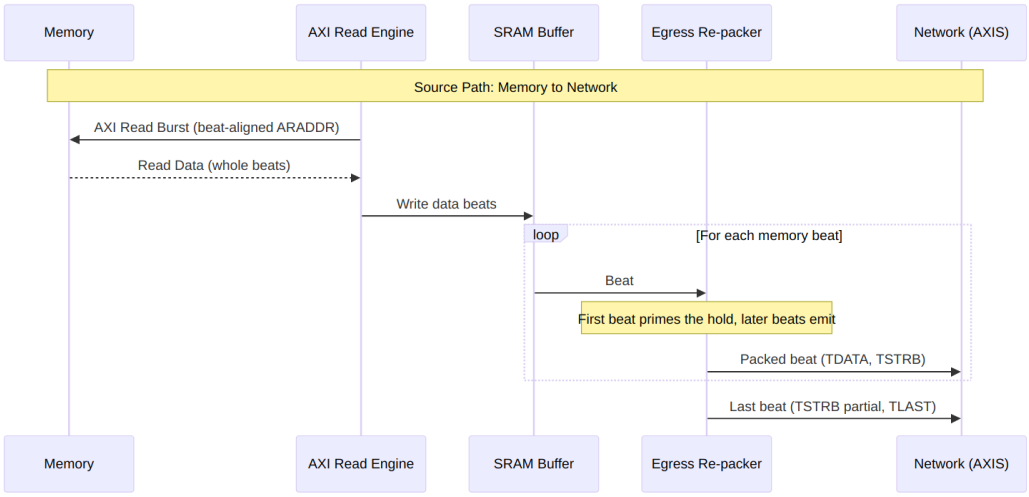
Source: 03_packet_record_flow.dot



Sink Path Data Flow

The ingress accepts a beat only when the channel's packet record exists. Until then TREADY stays low for that channel. After TLAST, if the offset pushed bytes into a further memory beat, the ingress flushes it as one more beat and holds TREADY low for that cycle.

The byte-enable timing on the write master is in the AXI4 Master Interface chapter.



Source Path Data Flow

The source always reads whole beats and lets the egress discard the bytes outside the descriptor. The first memory beat only primes the egress hold when the offset is non-zero; every later beat emits one stream beat, and a final flush emits the bytes left in the hold. The result is one AXI-Stream packet per descriptor, TLAST on its last beat.

The AXI-Stream chapter carries the waveform for a packet whose last beat is partial.

9.4 Concurrent Operation

Both data paths operate independently and can run simultaneously. Neither shares a resource with the other.

9.4.1 Deadlock Prevention

RAPIDS prevents deadlock through independent resource allocation:

Resource Isolation

Resource	Sink Path	Source Path	Shared
SRAM Buffer	Dedicated	Dedicated	No
AXI Port	Write only	Read only	No
Scheduler	Per-channel	Per-channel	No
Packet record queue	Per-channel, sink	Per-channel, source	No

The sink's TREADY is one signal, qualified by the TID of the beat on the bus. A beat for a channel whose descriptor has not been issued holds TREADY low, and it holds up any beats queued behind it on the same AXI-Stream, whatever their channel. A network source that interleaves channels on one stream SHOULD therefore issue every channel's descriptor before it streams, or present one channel at a time. The record queues and the ingress hold are per channel, so a stalled channel never corrupts another; it can only delay it.

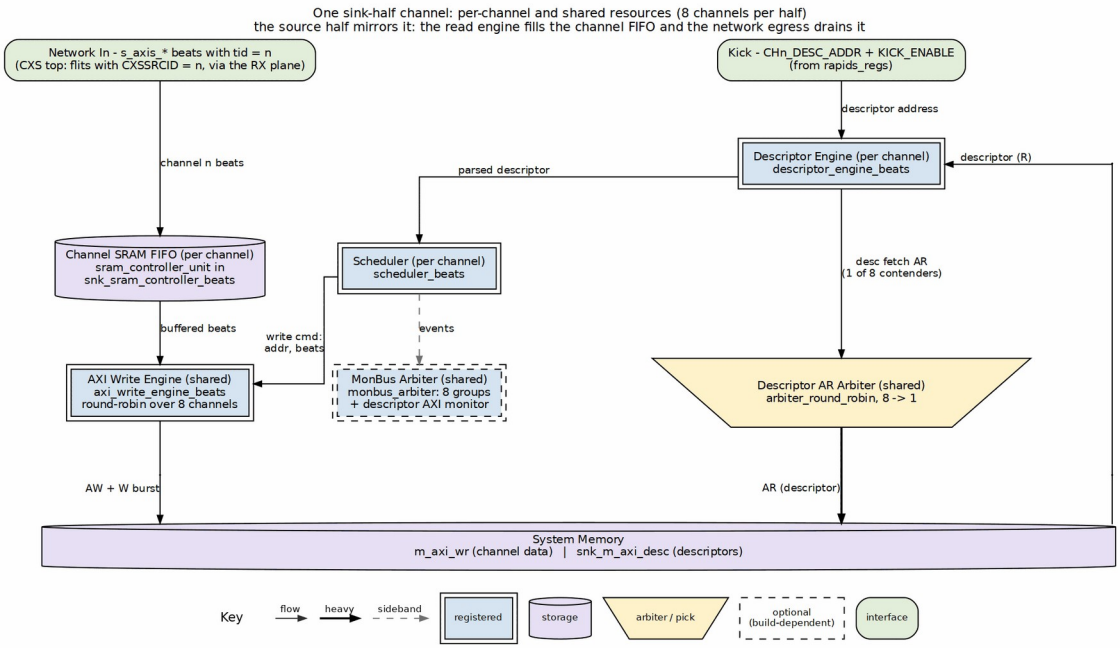
10 Channel Architecture

10.1 Multi-Channel Design

RAPIDS Beats supports 8 independent DMA channels, each with dedicated resources for concurrent operation.

10.2 Channel Resource Allocation

10.2.1 Figure 2.3.1: Per-Channel and Shared Resources



Per-Channel and Shared Resources

Source: ch02_channel_resources.dot

10.3 Per-Channel Resources

Each channel has dedicated:

Per-Channel Resources

Resource	Description	Size/Capacity
Scheduler	Transfer coordination FSM	1 instance
Descriptor Engine	Descriptor fetch/parse	8-deep FIFO
Sink SRAM Unit	Sink buffer, its own `sram_controller`	`SRAM_DEPTH` beats
Source SRAM Unit	Source buffer, its own `sram_controller`	`SRAM_DEPTH` beats
Configuration Registers	Per-channel config	~16 registers

10.4 Shared Resources

Resources shared across all channels:

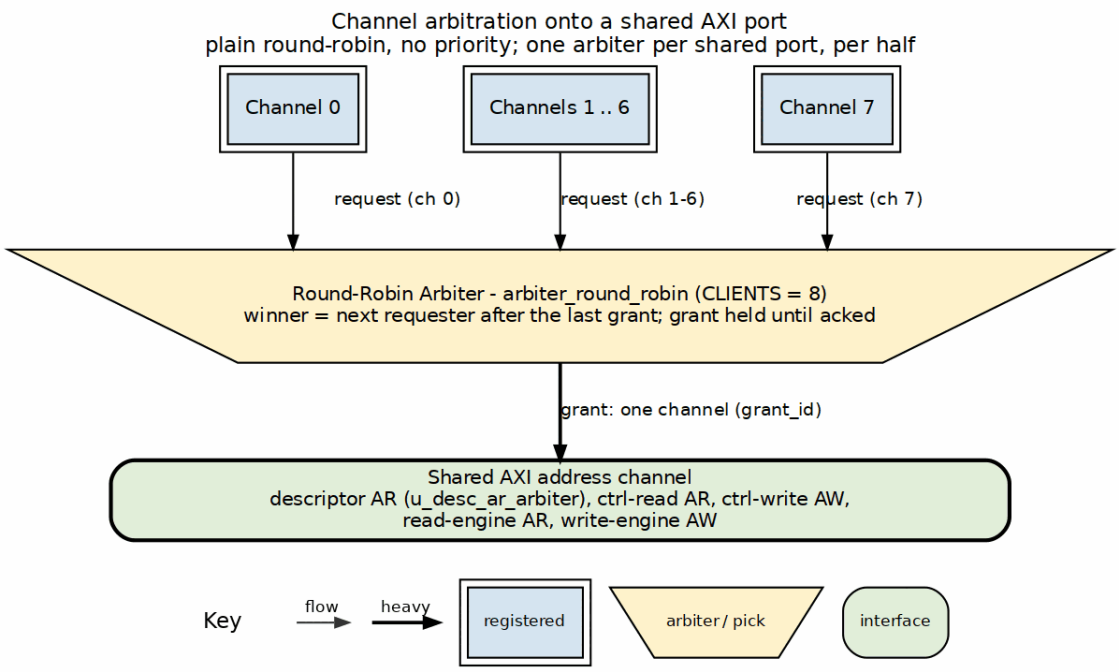
Shared Resources

Resource	Arbitration	Max Outstanding
Descriptor AXI	Round-robin	8 (1 per channel)
Sink AXI Write	Round-robin	8 configurable
Source AXI Read	Round-robin	8 configurable
MonBus Output	Round-robin (`monbus_arbiter` tree)	1 (serialized)

10.5 Channel Scheduling

10.5.1 Arbitration Policy

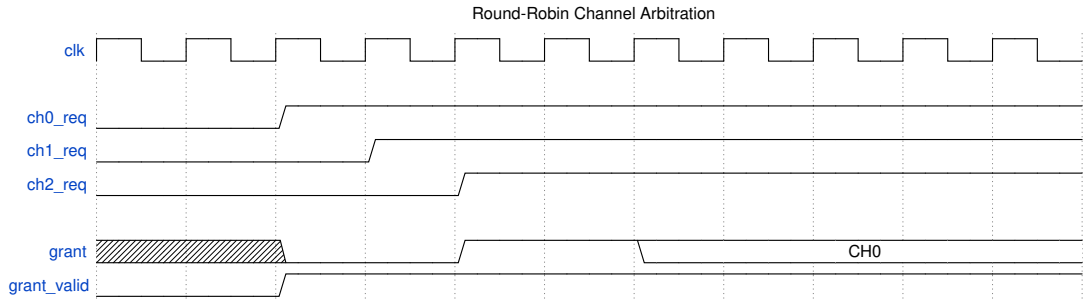
10.5.2 Figure 2.3.2: Channel Arbitration onto a Shared AXI Port



Channel Arbitration onto a Shared AXI Port

Source: ch02_channel_arbitration.dot

Arbitration Timing:



Waveform 1

10.6 Channel State Machine

Each channel scheduler follows this state machine:

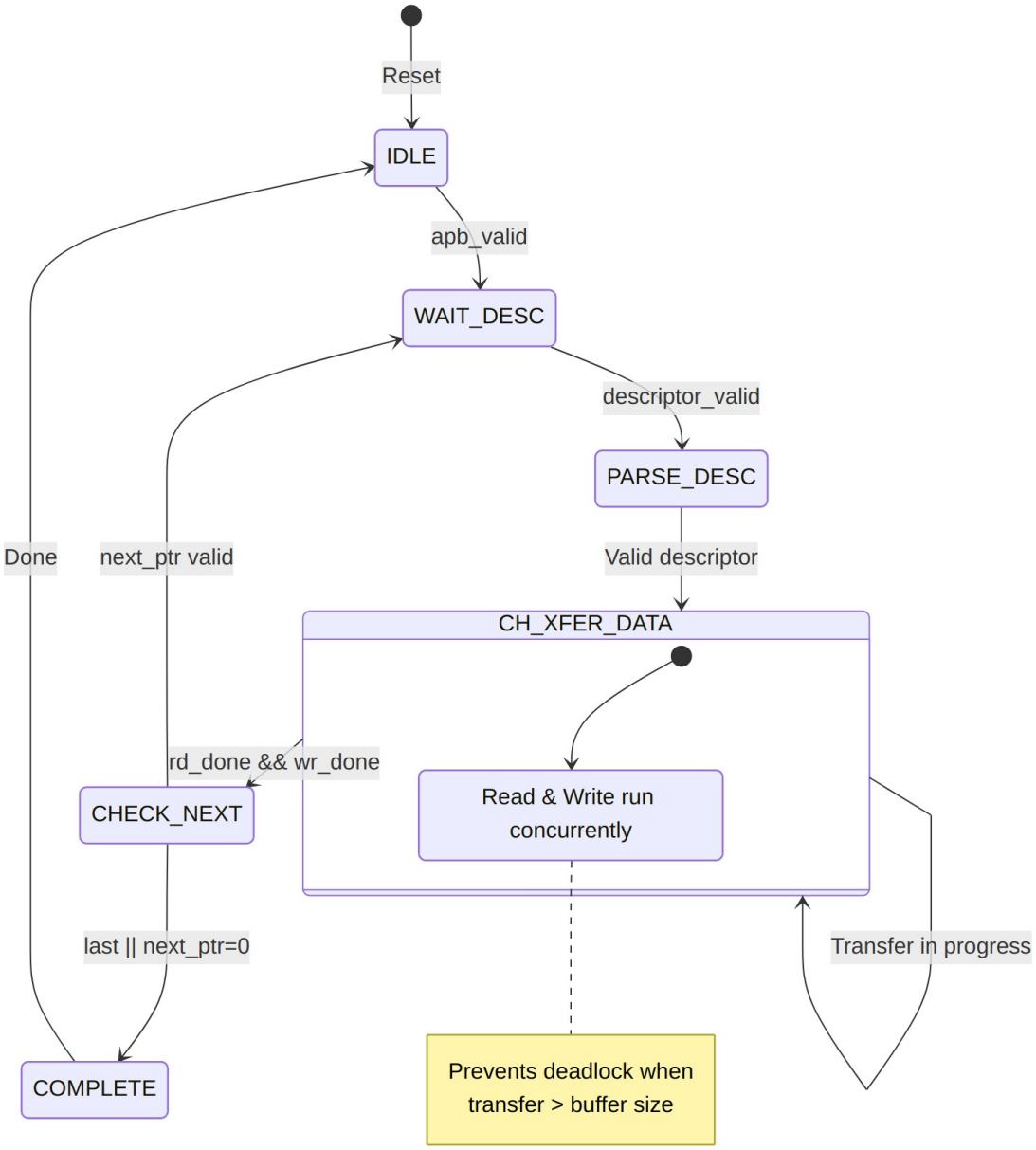
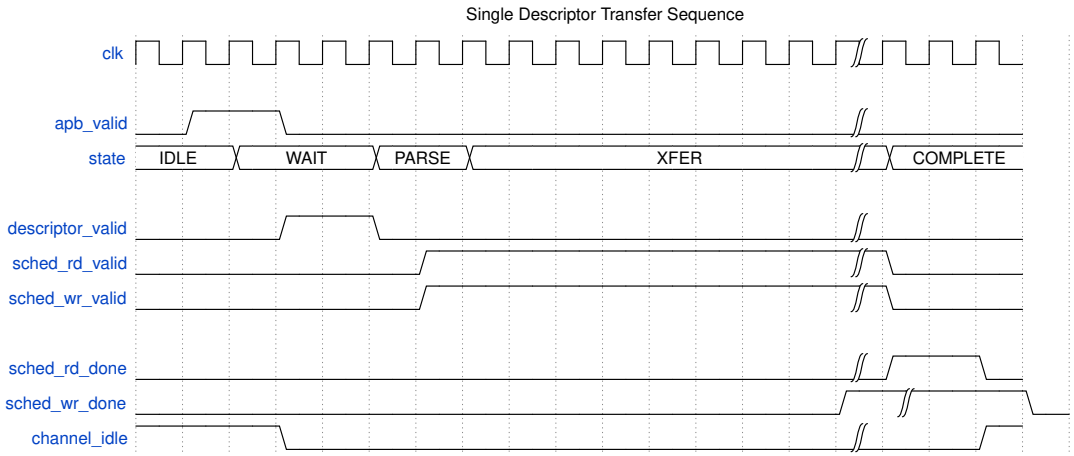


Diagram 4



Channel State Timing

10.7 Channel Isolation

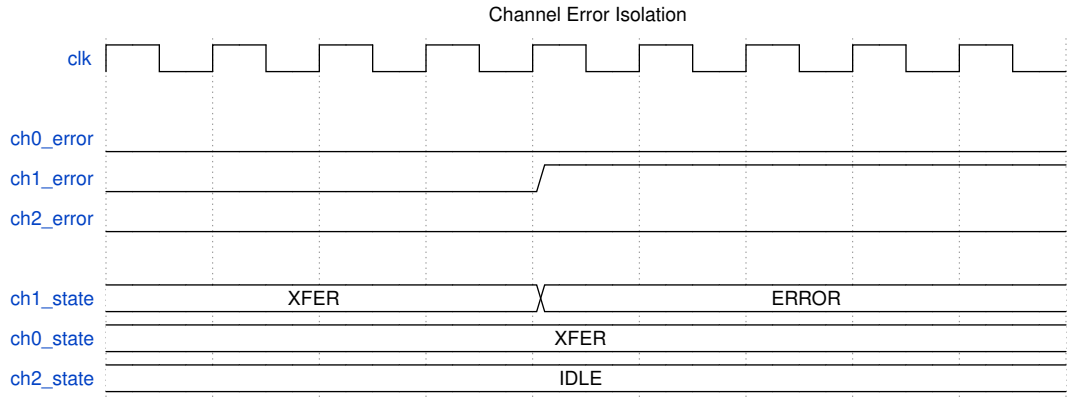
Channels are isolated to prevent interference:

Channel Isolation Features

Isolation Type	Implementation
State Isolation	Separate FSM per channel
Buffer Isolation	Partitioned SRAM per channel
Error Isolation	Per-channel error flags
Reset Isolation	Per-channel soft reset

10.7.1 Per-Channel Error Handling

Each channel maintains independent error state:



Waveform 3

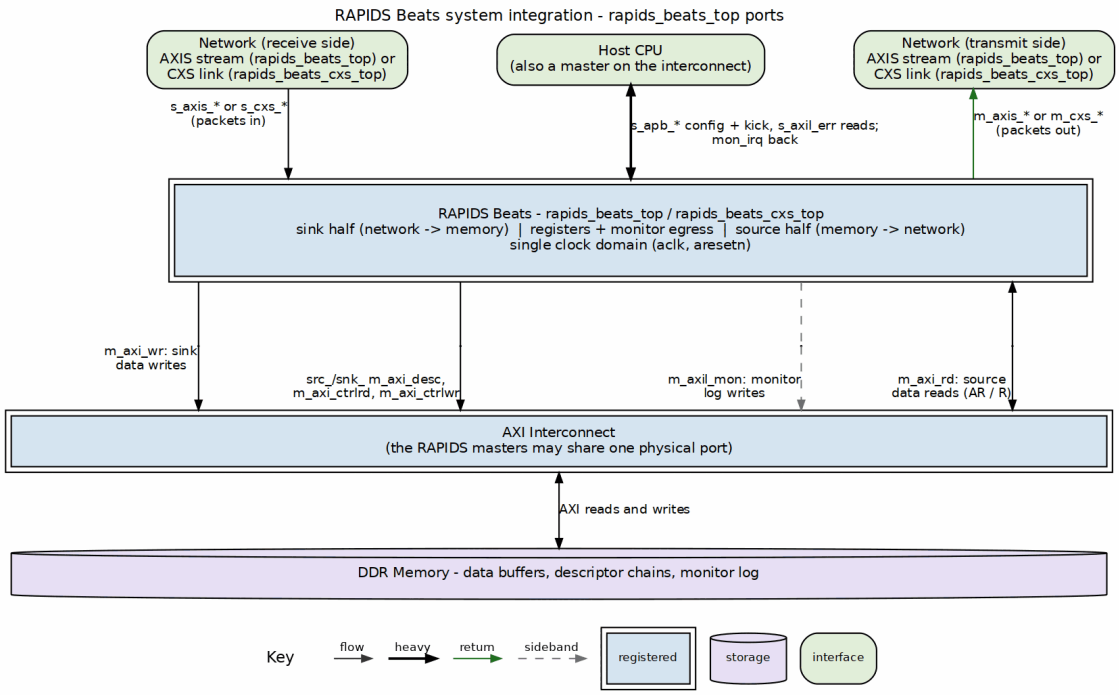
Channel 1 error does not affect Channel 0 or Channel 2 operation.

11 Interface Summary

11.1 External Interface Overview

RAPIDS Beats exposes the following external interfaces. The network side is a swappable plane: the AXIS tops (`rapids_beats_top`, `rapids_top`) carry the `s_axis_*` / `m_axis_*` streams, and the CXS tops (`rapids_beats_cxs_top`, `rapids_cxs_top`) carry one CXS link per direction in their place (see the CXS Network Interface chapter). Everything else is identical.

11.1.1 Figure 3.1.1: External Interfaces



External Interfaces

Source: ch01_system_integration.dot

11.2 Interface Port Summary

External Interface Summary

Interface	Type	Direction	Data Width	Address Width
Descriptor AXI	AXI4 Master	Read	256-bit	64-bit
Sink AXI	AXI4 Master	Write	512-bit	64-bit
Source AXI	AXI4 Master	Read	512-bit	64-bit
Sink AXIS	AXIS Slave	Input	512-bit	-
Source AXIS	AXIS Master	Output	512-bit	-
Sink CXS (CXS tops only, replaces Sink AXIS)	CXS Receiver	Input	`DATA_WIDTH` (up to 1024-bit)	-
Source CXS (CXS tops only, replaces Source AXIS)	CXS Transmitter	Output	`DATA_WIDTH` (up to 1024-bit)	-
APB Config	APB-like	Input	32-bit	12-bit
MonBus	Custom	Output	64-bit	-

11.3 Clock and Reset

All interfaces operate in the same clock domain:

Clock and Reset Signals

Signal	Type	Description
`aclk`	input	System clock (100-500 MHz typical)
`aresetn`	input	Active-low reset (async assert, sync deassert)

11.4 Interface Feature Matrix

Interface Feature Matrix

Feature	Desc AXI	Sink AXI	Src AXI	Sink AXIS	Src AXIS
Protocol	AXI4	AXI4	AXI4	AXIS	AXIS
Direction	Read	Write	Read	Slave	Master
Burst Support	INCR	INCR	INCR	N/A	N/A
Max Burst	1	256	256	N/A	N/A
Outstanding	8	8	8	N/A	N/A
ID Width	8-bit	8-bit	8-bit	Optional	Optional

11.5 Signal Count Summary

Signal Count Summary

Interface	Input Signals	Output Signals	Total
Descriptor AXI	~20	~30	~50
Sink AXI	~15	~40	~55
Source AXI	~25	~25	~50
Sink AXIS	~20	~5	~25
Source AXIS	~5	~20	~25
Sink CXS (CXS tops)	~8	~7	~15
Source CXS (CXS tops)	~3	~10	~13
APB Config	~15	~5	~20
MonBus	~2	~5	~7
Total	**~102**	**~130**	**~232**

Note: Signal counts are approximate and vary with configuration parameters. The total counts an AXIS top; a CXS top replaces the two AXIS rows with the two CXS rows (including the link-status pins).

12 AXI4 Interface Specification

12.1 Overview

RAPIDS uses AXI4 master interfaces for all memory access:

1. **Descriptor AXI** - Fetches 256-bit descriptors (read-only), one master per half (`src_m_axi_desc_*`, `snk_m_axi_desc_*`)
2. **Sink AXI** - Writes network data to memory (`m_axi_wr_*`)
3. **Source AXI** - Reads memory data for the network (`m_axi_rd_*`)
4. **Control AXI** - Single-word control reads and writes (`*_m_axi_ctrlrd_*`, `*_m_axi_ctrlwr_*`), unchanged from RAPIDS Beats

The descriptor and control masters behave exactly as in RAPIDS Beats; see the RAPIDS Beats AXI4 chapter for their timing. This chapter specifies what changed: the data masters carry byte-granular transfers.

12.1.1 Byte Granularity on the Memory Side

The descriptor length is a byte count and the source and destination addresses are byte addresses. The data masters keep AXI4 conventional by splitting the two concerns:

Where byte granularity is carried on AXI4

Concern	Where it is carried
Which beats are transferred	`AxADDR` (beat-aligned) and `AxLEN`
Which bytes within a beat are transferred	`WSTRB` on the write master; the egress drops unwanted bytes on the read side

`AxSIZE` is always the full beat (`log2(DATA_WIDTH/8)`), so the masters remain ordinary full-width INCR masters that any interconnect accepts.

12.2 Descriptor AXI Master

Unchanged from RAPIDS Beats. One 256-bit single-beat INCR read per descriptor, `ARLEN` = 0, `ARSIZE` = 5, `ARID` carries the channel. The source half and the sink half each own a master, and the descriptor address is 32-byte aligned. The descriptor's contents (Chapter 5) differ: its length field is in bytes.

12.3 Sink AXI Master (Write)

12.3.1 Purpose

Writes packed network bytes from the SRAM buffer to system memory. The SRAM holds each beat as data plus a strobe (`{strb, data}`), placed by the sink ingress at the byte offset of the destination address. The write master emits that strobe on `WSTRB` unchanged.

12.3.2 Configuration

Sink AXI Configuration

Parameter	Default	Range	Description
`DATA_WIDTH`	512	32-1024 (power of two)	Beat width; 32-byte beats (256-bit) in the Genesys 2 build
`ADDR_WIDTH`	64	Configurable	System address space
`AXI_ID_WIDTH`	8	Configurable	Transaction tracking (channel in low bits)
Max burst	`WR_XFER_BEATS` register	1-256	Per-channel burst cap, further capped at 4 KB
Outstanding	8	1-16	Concurrent AW requests

12.3.3 Signal List

Sink AXI Signals

Signal	Width	Direction	Description
`m_axi_wr_awid`	`AXI_ID_WIDTH`	output	Write ID
`m_axi_wr_awaddr`	`ADDR_WIDTH`	output	Write address, **beat-aligned**
`m_axi_wr_awlen`	8	output	Burst length - 1
`m_axi_wr_awsize`	3	output	Full beat, `log2(DATA_WIDTH/8)`
`m_axi_wr_awburst`	2	output	INCR
`m_axi_wr_awvalid` / `awready`	1	output / input	Address handshake

Signal	Width	Direction	Description
m_axi_wr_wdata	DATA_WIDTH	output	Write data
m_axi_wr_wstrb	DATA_WIDTH/8	output	**Byte enables: the bytes this beat writes**
m_axi_wr_wlast	1	output	Last beat of the burst
m_axi_wr_wvalid / wready	1	output / input	Data handshake
m_axi_wr_bid	AXI_ID_WIDTH	input	Response ID
m_axi_wr_bresp	2	input	Write response
m_axi_wr_bvalid / bready	1	input / output	Response handshake

12.3.4 WSTRB Contract

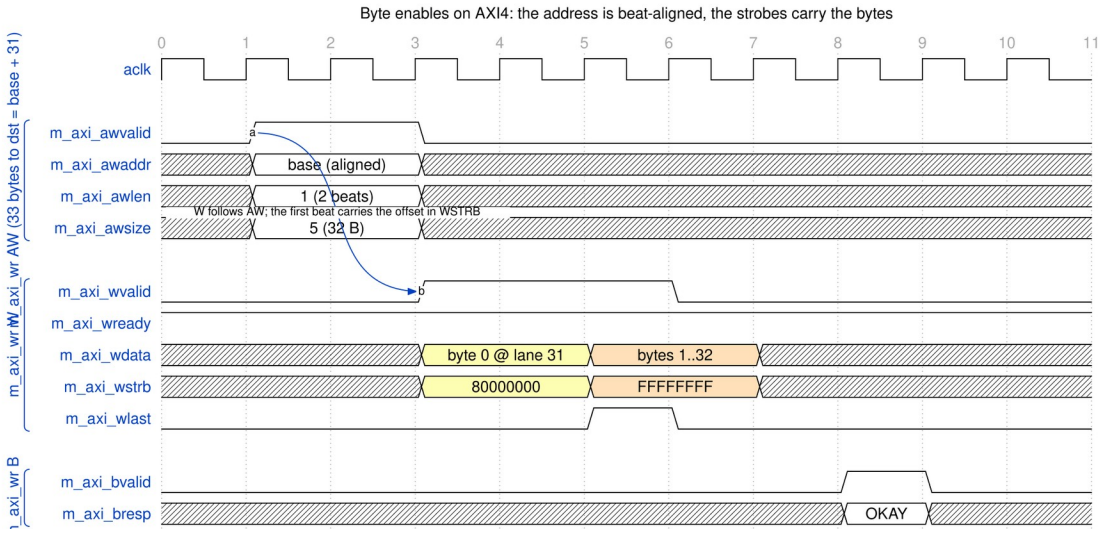
For a descriptor that writes `length` bytes starting at byte address `dst`, let `offset = dst mod (DATA_WIDTH/8)`. The transfer covers `ceil((offset + length) / (DATA_WIDTH/8))` beats.

WSTRB per beat

Beat	WSTRB
First beat	Lanes `offset` to the end of the beat, or to the last byte if the transfer ends inside this beat
Middle beats	All ones
Last beat	Lanes 0 to the last byte, or up to the last lane if the transfer ends on a beat boundary
Single-beat transfer	Lanes `offset` to `offset + length - 1`

Lanes outside the run have `WSTRB` = 0 and their `WDATA` bytes are not significant. RAPIDS never writes a byte the descriptor did not name, so a destination memory that honors byte enables is left untouched outside `[dst, dst + length)`. Every beat in the transfer carries at least one strobed byte.

12.3.4.1 Waveform 3.1: Unaligned Sink Write, Byte Enables on AXI4



Byte enables on AXI4

Source: 02_axi_byte_enables.json

A 33-byte transfer to `base + 31` occupies two beats of a 32-byte-beat interface. `AWADDR` is `base`, not `base + 31`. The first beat strobes only lane 31 (`0x80000000`) and carries byte 0 there; the second beat strobes all 32 lanes and carries bytes 1 to 32.

12.4 Source AXI Master (Read)

12.4.1 Purpose

Reads whole beats from system memory into the SRAM buffer. The read master does not use byte enables (AXI has none on the read side). It fetches every beat that overlaps `[src, src + length)`, and the source egress (Chapter 3.2) drops the leading and trailing bytes that were fetched but not requested.

12.4.2 Configuration

Source AXI Configuration

Parameter	Default	Range	Description
`DATA_WIDTH`	512	32-1024 (power of two)	Beat width
`ADDR_WIDTH`	64	Configurable	System address space
`AXI_ID_WIDTH`	8	Configurable	Transaction tracking
Max burst	`RD_XFER_BEATS` register	1-256	Per-channel burst cap, further capped at 4 KB
Outstanding	8	1-16	Concurrent AR requests

12.4.3 Signal List

Source AXI Signals

Signal	Width	Direction	Description
`m_axi_rd_arid`	`AXI_ID_WIDTH`	output	Read ID
`m_axi_rd_araddr`	`ADDR_WIDTH`	output	Read address, **beat-aligned**
`m_axi_rd_arlen`	8	output	Burst length - 1
`m_axi_rd_arsize`	3	output	Full beat
`m_axi_rd_arburst`	2	output	INCR
`m_axi_rd_arvalid` / `arready`	1	output / input	Address handshake

Signal	Width	Direction	Description
m_axi_rd_rid	AXI_ID_WIDTH	input	Response ID
m_axi_rd_rdata	DATA_WIDTH	input	Read data
m_axi_rd_rresp	2	input	Read response
m_axi_rd_rlast	1	input	Last beat
m_axi_rd_rvalid / rready	1	input / output	Data handshake

12.4.4 Read Coverage

A read of `length` bytes from byte address `src` fetches `ceil((offset + length) / (DATA_WIDTH/8))` beats starting at `src` rounded down to a beat boundary. Bytes fetched outside `[src, src + length)` are never delivered to the network.

12.5 AXI Protocol Constraints

12.5.1 Address Alignment

Address Alignment Requirements

Transfer Type	Alignment Requirement
Descriptor	32-byte aligned
Sink data, linear descriptor	**None**: any byte address
Source data, linear descriptor	**None**: any byte address
Sink or source data, TYPE=EXT descriptor	Beat-aligned address, beat-multiple length (permanent)

The AXI address on the wire is always beat-aligned, whatever the descriptor holds. Only EXT (row and column striding) descriptors restrict the descriptor's own addresses; that is a permanent design decision, not a phase limitation.

12.5.2 4 KB Boundary Handling

AXI4 forbids a burst crossing a 4 KB boundary. RAPIDS caps every burst at the next 4 KB boundary, computed from the beat-aligned address, on both data masters. The cap combines with the configured burst limit: the burst is the smaller of the two.

```
Example: 4035 bytes to dst = 0x0CB (32-byte beats, offset = 0x0B)
  Beats total = ceil((11 + 4035) / 32) = 127
  Aligned start = 0x0C0; beats from there to the 4 KB boundary = 122
  With WR_XFER_BEATS = 16 the bursts are 16-beat, the last one before
the boundary
  ends at 0x1000 (a 122-beat run is 7 bursts of 16 and one of 10);
  the remaining 5 beats start at AWADDR = 0x1000.
```

The first burst's `WSTRB` carries the offset (`0xFFFFF800`, lanes 11 to 31). After the first burst the working address is the aligned address plus the beats completed, so every later burst is beat-aligned.

12.5.3 Response Handling

AXI Response Handling

Response	Code	RAPIDS Behavior
OKAY	2'b00	Normal completion
EXOKAY	2'b01	Treated as OKAY
SLVERR	2'b10	Error, transfer aborted

Response	Code	RAPIDS Behavior
DECERR	2’b11	Error, transfer aborted

13 AXIS Interface Specification

13.1 Overview

RAPIDS uses AXI-Stream interfaces for network data transfer:

1. **Sink AXIS Slave** - Receives packed bytes from the network (ingress)
2. **Source AXIS Master** - Sends packed bytes to the network (egress)

Both streams carry **packed bytes**: a packet is a contiguous run of bytes starting at lane 0 of its first beat, with every beat full except possibly the last. The stream is byte-granular through `TSTRB`, and `TLAST` delimits exactly one descriptor's worth of data.

13.1.1 Byte Semantics

Byte Semantics of the RAPIDS Streams

Property	Rule
Byte qualifier	`TSTRB` (there is no `TKEEP` on RAPIDS)
Packing	Bytes are packed from lane 0; no gaps inside a packet
Full beats	`TSTRB` all ones
Last beat	`TSTRB` is a contiguous run from lane 0 (`2^n - 1`), `n` = bytes in the beat
Packet	One descriptor's `length` bytes, ending with `TLAST`

The destination address offset never appears on the stream. A descriptor that writes to byte address 5 of a beat still receives its packet packed from lane 0; the sink places the bytes at the destination offset internally (Chapter 2).

13.2 Sink AXIS Slave

13.2.1 Purpose

Receives a packet, places its bytes at the descriptor's destination offset, and writes them to the SRAM buffer for the sink AXI master.

13.2.2 Configuration

Sink AXIS Configuration

Parameter	Default	Range	Description
DATA_WIDTH	512	32-1024 (power of two)	TDATA width
AXIS_ID_WIDTH	8	1-8	TID width; TID selects the channel
AXIS_DEST_WIDTH	8	0-8	TDEST width (not used to route)
AXIS_USER_WIDTH	1	0-16	TUSER width (not used)

13.2.3 Signal List

Sink AXIS Signals

Signal	Width	Direction	Description
s_axis_tdata	DATA_WIDTH	input	Data payload
s_axis_tstrb	DATA_WIDTH/8	input	Byte qualifier, contiguous from lane 0
s_axis_tlast	1	input	Last beat of the packet
s_axis_tid	AXIS_ID_WIDTH	input	Channel select
s_axis_tdest	AXIS_DEST_WIDTH	input	Destination (accepted, not used)
s_axis_tuser	AXIS_USER_WIDTH	input	User sideband (accepted, not used)
s_axis_tvalid	1	input	Data valid

Signal	Width	Direction	Description
s_axis_tready	1	output	Ready to accept

13.2.4 Packet Contract

A sink packet SHALL:

1. Be delimited by TLAST, with one TID for all its beats.
2. Be packed: every beat but the last has TSTRB all ones, and the last beat's TSTRB is a contiguous run from lane 0.
3. Contain exactly as many strobed bytes as the descriptor's length for that channel.

The byte count is checked at TLAST. A packet whose count differs from the descriptor length is not silently padded or dropped: the channel's sticky sched_wr_error bit is set and stays set until reset.

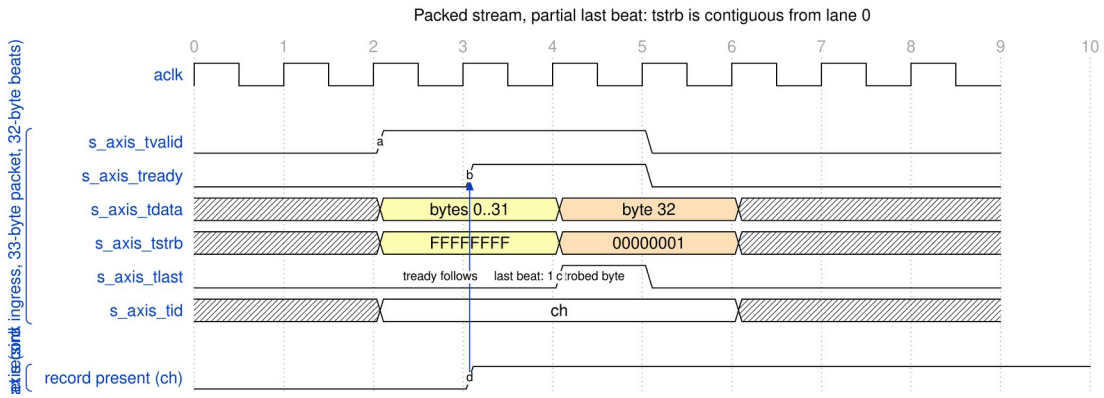
13.2.5 Descriptor-First Ordering

The sink cannot place bytes until it knows the destination offset, and the offset comes from the channel's descriptor. s_axis_tready is a single signal. It is qualified by the TID of the beat currently on the bus: it is low whenever that channel has no packet record, in addition to the output register being busy or a flush beat pending.

A beat for a channel whose descriptor has not been fetched therefore stalls the whole stream, including beats for other channels queued behind it. There is no per-channel backpressure. Software SHOULD issue a channel's descriptor before its packet arrives, or stream one channel at a time until its descriptor is known.

This is the one flow-control difference from RAPIDS Beats, whose sink buffered beats before any descriptor existed. A transmitter that blocks its own descriptor kick behind the stalled stream will deadlock; kick first.

13.2.5.1 Waveform 3.1: Sink Packet With a Partial Last Beat



Packed stream, partial last beat

Source: 01_axis_partial_last_beat.json

A 33-byte packet occupies two 32-byte beats. The first beat is full (`TSTRB = FFFFFFFF`); the second carries one byte (`TSTRB = 00000001`) and `TLAST`. `s_axis_tready` rises only after the channel's record exists, so `TVALID` waits for it.

13.2.6 Backpressure Behavior

`s_axis_tready` deasserts when:

Reasons the Sink Deasserts TREADY

Cause	Meaning
No record for the beat's channel	The descriptor for the `TID` on the bus has not been fetched; the whole stream waits
SRAM space	The channel's SRAM partition is full
Ingress output register busy	The write into the SRAM controller has not completed
Flush cycle	`TREADY` drops for one cycle while the spill beat that follows `TLAST` is emitted

The flush cycle occurs when a packet's last bytes spill into a memory beat beyond the stream's last beat: for example, 33 bytes to destination offset 5 occupy 38 bytes, so the stream's second beat maps to two memory beats.

13.3 Source AXIS Master

13.3.1 Purpose

Reads memory beats through the SRAM buffer and emits the descriptor's bytes as one packed AXIS packet.

13.3.2 Configuration

Source AXIS Configuration

Parameter	Default	Range	Description
DATA_WIDTH	512	32-1024 (power of two)	TDATA width
AXIS_ID_WIDTH	8	1-8	TID width; carries the channel
AXIS_DEST_WIDTH	8	0-8	TDEST width; carries the channel
AXIS_USER_WIDTH	1	0-16	TUSER width; driven 0

13.3.3 Signal List

Source AXIS Signals

Signal	Width	Direction	Description
m_axis_tdata	DATA_WIDTH	output	Data payload
m_axis_tstrb	DATA_WIDTH/8	output	Byte qualifier, contiguous from lane 0
m_axis_tlast	1	output	Last beat of the packet
m_axis_tid	AXIS_ID_WIDTH	output	Channel
m_axis_tdest	AXIS_DEST_WIDTH	output	Channel (zero-extended)
m_axis_tuser	AXIS_USER_WIDTH	output	Driven 0
m_axis_tvalid	1	output	Data valid
m_axis_tready	1	input	Downstream ready

13.3.4 Egress Packet Contract

Egress Packet Contract

Property	Behavior
Packet framing	One AXIS packet per descriptor, however many drain blocks it spans
Bytes	Exactly `length` bytes, from the descriptor's source byte address
First beat	Starts at lane 0: the egress shifts memory beats down by the source offset
Middle beats	`TSTRB` all ones
Last beat	Contiguous `TSTRB` and `TLAST`
Zero-length descriptor	No packet

RAPIDS Beats cut a packet at the end of each drain block and asserted `TLAST` on every one. RAPIDS asserts `TLAST` once, on the last byte of the descriptor.

13.3.5 Timing

The partial last beat is the mirror image of the sink case (Waveform 3.1 above): the final beat carries a contiguous run of `n` bytes and `TLAST`. The re-pack mechanism that produces it, including the beat that only primes the shifter, is specified in the MAS.

13.3.6 Data Availability

The source asserts `TVALID` only when packed data is available. The first memory beat of a packet with a nonzero source offset only primes the egress shifter and produces no output unless the entire packet fits in that beat.

13.4 AXIS Protocol Notes

13.4.1 Handshake Rules

Standard AXI-Stream handshake applies:

1. `TVALID` asserts when data is available
2. `TREADY` asserts when the receiver can accept
3. A transfer occurs on a clock edge when both are high
4. `TVALID` SHALL NOT depend on `TREADY`
5. `TREADY` MAY depend on `TVALID`

13.4.2 TLAST Semantics

TLAST Usage

Scenario	TLAST Behavior
Sink packet	Assert on the last beat of the packet; the byte count must equal the descriptor length
Source packet	Asserted once per descriptor, on its last stream beat
Continuous stream	Not supported: every transfer is a descriptor-sized packet

13.4.3 TSTRB Usage

TSTRB Examples

TSTRB Value (32-byte beat)	Meaning
`FFFFFFFF`	All 32 bytes valid (every beat but the last)
`0000FFFF`	Last beat, lower 16 bytes valid
`00000001`	Last beat, one byte valid
Any non-contiguous pattern	Not produced by RAPIDS; not accepted on the sink

13.4.4 Channel Multiplexing via TID

Channels share each stream through `TID`. A sink packet SHALL keep one `TID` from its first beat to `TLAST`. Each channel is packed independently, so packets of different channels may follow one another, but `s_axis_tready` does not distinguish channels: a channel that is not ready stalls the stream for all of them (see Descriptor-First Ordering). The source stream emits one packet at a time, tagged with the channel in `TID` and `TDEST`.

---

14 APB Configuration Interface

14.1 Overview

At the top level (`rapids_beats_top`), RAPIDS Beats exposes a single **APB4 slave** (`s_apb_*`) for all software access. This interface allows software to:

1. Configure channel and monitor parameters (via the register block)
2. Initiate descriptor processing (kick-off)
3. Read status and error information

14.2 APB Slave and Address Routing

The APB slave is converted to an internal command/response transaction and routed by address to two targets:

APB Address Routing

Address Range	Target	Purpose
0x000-0x03F	`rapids_regs` (staged descriptor addresses)	Per-channel `CH*_DESC_ADDR_{LOW,HIGH}`, stride 0x8
0x040	`rapids_regs` (`KICK_ENABLE`)	Per-channel launch bits (singlepulse)
0x100-0x3FF	`rapids_regs` (base regfile)	Channel/scheduler/descriptor configuration and status
0x1000+	`rapids_regs` (monitor regfile)	AXI-monitor configuration and performance counters

Because the monitor regfile is located at 0x1000, the APB address bus must be at least 13 bits wide to reach it. Configuration that was formerly imagined as discrete `cfg_*` ports is now driven internally by the register block through `rapids_config_block`; see the Register Map for the full address map, and the descriptor address ranges (`DESCENG_ADDR0/1_*` at 0x224-0x230).

14.2.1 APB Slave Signal List

APB Slave Signals

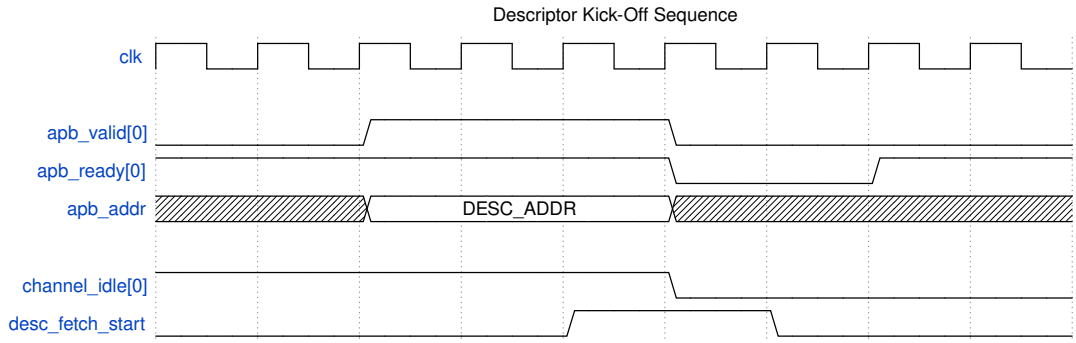
Signal	Width	Direction	Description
`s_apb_paddr`	APB_ADDR_WIDTH (>=13)	input	Address
`s_apb_psel`	1	input	Select
`s_apb_penable`	1	input	Enable phase
`s_apb_pwrite`	1	input	Write / read
`s_apb_pwdata`	32	input	Write data
`s_apb_pstrb`	4	input	Write byte strobes

Signal	Width	Direction	Description
`s_apb_prdata`	32	output	Read data
`s_apb_pready`	1	output	Ready
`s_apb_pslverr`	1	output	Slave error

14.2.2 Descriptor Kick-Off

To start descriptor processing on a channel:

1. Write descriptor to memory
2. Assert `apb_valid[ch]` with descriptor address on `apb_addr`
3. Wait for `apb_ready[ch]` assertion
4. Deassert `apb_valid[ch]`



Timing Diagram

14.3 Status Interface

14.3.1 Per-Channel Status Signals

Status Signals

Signal	Width	Direction	Description
`channel_idle`	NC	output	Channel idle status
`channel_error`	NC	output	Channel error flag
`scheduler_state`	4*NC	output	FSM state per channel

14.3.2 Channel State Encoding

Scheduler State Encoding

State	Value	Description
IDLE	4’h0	Waiting for kick-off
WAIT_DESC	4’h1	Waiting for descriptor
PARSE_DESC	4’h2	Parsing descriptor
CH_XFER_DATA	4’h3	Transfer in progress
CHECK_NEXT	4’h4	Checking next descriptor
COMPLETE	4’h5	Transfer complete
ERROR	4’hF	Error state

14.4 Configuration Registers

14.4.1 Address Range Validation

RAPIDS validates descriptor addresses against configurable ranges:

```
Valid if:
  (addr >= cfg_addr0_base && addr < cfg_addr0_limit) ||
  (addr >= cfg_addr1_base && addr < cfg_addr1_limit)
```

14.4.2 Configuration Registers

Configuration and status registers are provided by the `rapids_regs` register block. See the Register Map for the complete, RTL-accurate address map. The registers most relevant to bring-up are:

Key Configuration Registers

Register	Offset	Description
`GLOBAL_CTRL`	0x100	`GLOBAL_EN`, `GLOBAL_RST` (level; write 0 to release)
`CHANNEL_ENABLE`	0x120	Per-channel enable bitmap
`SCHED_TIMEOUT_CYCLES`	0x200	Write-progress timeout window
`SCHED_TIMEOUT_LIMIT`	0x208	Consecutive-timeout escalation limit (0 = never)
`DESCENG_CONFIG`	0x220	Descriptor-engine enable / prefetch / FIFO threshold
`DESCENG_ADDR0_BASE/_LIMIT`	0x224/0x228	Descriptor address range 0

14.5 Programming Sequence

14.5.1 Initialization

```
// 1. Optional global reset: GLOBAL_RST is a level, not self-clearing
GLOBAL_CTRL = 0x2;           // assert
GLOBAL_CTRL = 0x0;           // release

// 2. Configure descriptor address range(s)
DESCENG_ADDR0_BASE  = 0x8000_0000;
DESCENG_ADDR0_LIMIT = 0x9000_0000;

// 3. Scheduler timeout policy
SCHED_TIMEOUT_CYCLES = 1000;
SCHED_TIMEOUT_LIMIT  = 4;    // escalate after 4 stalled windows; 0 =
never

// 4. Enable channels and global enable
CHANNEL_ENABLE = 0xFF;       // enable all 8 channels
GLOBAL_CTRL    = 0x1;        // GLOBAL_EN
```

14.5.2 Descriptor Kick-Off

```
// Prepare descriptor in memory
desc->src_addr = src;
desc->dst_addr = dst;
desc->length = beats;
desc->last = 1;
desc->valid = 1;

// Kick off channel 0: stage the address, then launch
CH0_DESC_ADDR_LOW  = (uint32_t)desc_addr;
CH0_DESC_ADDR_HIGH = (uint32_t)(desc_addr >> 32);
KICK_ENABLE        = 0x01;   // KICK0, self-clearing

// Wait for completion: idle, or faulted (CH_ERROR is never idle)
while (!(SCHEDULER_IDLE & 0x01) && !(SCHED_ERROR & 0x01));

// Check for errors
if (SCHED_ERROR & 0x01) {
    handle_error(0);
}
```

15 Monitor Bus Interface

15.1 Overview

RAPIDS Beats outputs a 64-bit Monitor Bus (MonBus) for real-time event reporting. The MonBus provides visibility into:

- State machine transitions
- Descriptor processing events
- Error conditions
- Performance metrics

15.2 MonBus Packet Format

15.2.1 64-bit Packet Structure

```
Bit Field          Width   Description
──────────────────────────────────────────────────
[63:60]            4-bit   packet_type (error, completion, etc.)
[59:57]            3-bit   protocol (CORE for RAPIDS)
[56:53]            4-bit   event_code (specific event)
[52:47]            6-bit   channel_id
[46:43]            4-bit   unit_id (subsystem)
[42:35]            8-bit   agent_id (module)
[34:0]             35-bit  event_data (event-specific)
```

15.2.2 Signal List

MonBus Signals

Signal	Width	Direction	Description
monbus_pkt_valid	1	output	Packet valid
monbus_pkt_data	64	output	Packet data
monbus_pkt_ready	1	input	Downstream ready

15.3 Packet Types

MonBus Packet Types

Type	Code	Description
ERROR	4'h0	Error event
COMPLETION	4'h1	Transfer/operation complete
THRESHOLD	4'h2	Threshold crossed
TIMEOUT	4'h3	Timeout event
PERF	4'h4	Performance metric
DEBUG	4'hF	Debug/trace event

15.4 Agent IDs

RAPIDS modules use the following Agent IDs:

RAPIDS Agent IDs

Agent ID	Module	Description
0x08	Descriptor AXI monitor	`axi4_master_rd_monlite` on the descriptor read master (per half, `USE_AXI_MONITORS`)
0x09	Sink-ingress network monitor	The sink RX plane's monitor-lite: `axis4_slave_monlite` on `s_axis_*` (AXIS top) or `cxs_monitor_lite` in `cxs_slave_monlite` on `s_cxs_*` (CXS top) (`USE_AXI_MONITORS`; config `SNK.MON.WRMON_*`)
0x0A	Source-egress network monitor	The source TX plane's monitor-lite: `axis4_master_monlite` on `m_axis_*` (AXIS top) or `cxs_monitor_lite` in `cxs_master_monlite` on `m_cxs_*` (CXS top) (`USE_AXI_MONITORS`; config `SRC.MON.RDMON_*`)
0x10-0x17	Descriptor Engine	Channels 0-7
0x20	Control Read Engine	ctrlrd (one per half)
0x21	Control Write Engine	ctrlwr (one per half)
0x30-0x37	Scheduler	Channels 0-7

These are the values the scheduler group array and the halves pass down (`DESC_AXI_MON_AGENT_ID`, `DESC_MON_BASE_AGENT_ID`, `CTRLRD_MON_AGENT_ID`, `CTRLWR_MON_AGENT_ID`, `SCHED_MON_BASE_AGENT_ID`); the module defaults inside the engines differ and are overridden at instantiation. The data paths carry no MonBus emitters (an earlier revision listed data-path agents at 0x40-0x57, which nothing emits). Agents 0x09 and 0x0A live in the network planes, outside the core; their packets enter each half's 3:1 `monbus_arbiter` as client

1 on snk_plane_mon_* / src_plane_mon_*. A CXS plane's packets carry protocol PROTOCOL_CXS, which the top's monitor group filters through its AXIS mask slot. A CXS PACKET_END completion carries the channel (CXSSRCID) in event_data[55:48]; its channel_id field holds {PRCLTYPE[2:0], TGTID[5:0]}.

15.5 Event Codes

15.5.1 Scheduler Events (Agent 0x30-0x37)

Scheduler Event Codes

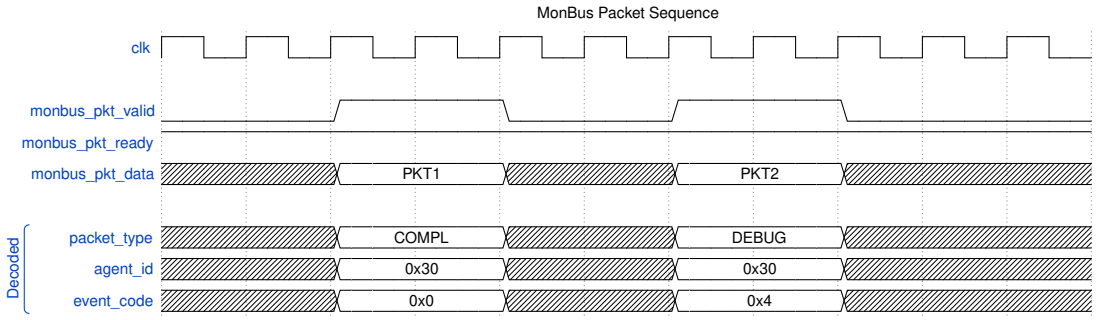
Event Code	Type	Description
0x0	COMPLETION	Descriptor complete
0x1	COMPLETION	Chain complete
0x2	ERROR	Timeout
0x3	ERROR	AXI error
0x4	DEBUG	State transition

The scheduler's and the descriptor engine's COMPLETION packets are reported only while the half's `SCHED_CONFIG.COMPL_EN` is set (reset value 1). With the bit clear the scheduler group drops them before its monitor-bus arbiter and still acknowledges the emitters, so a channel runs identically either way; ERROR packets are reported regardless of `ERR_EN`.

15.5.2 Descriptor Engine Events (Agent 0x10-0x17)

Descriptor Engine Event Codes

Event Code	Type	Description
0x0	COMPLETION	Descriptor fetched
0x1	ERROR	Address range error
0x2	ERROR	AXI read error
0x3	DEBUG	Fetch started



Timing Diagram

15.6 Event Data Encoding

15.6.1 Completion Event Data

```
[34:32] - Reserved
[31:0]  - Transfer length (beats completed)
```

15.6.2 Error Event Data

```
[34:32] - Error type
[31:16] - Reserved
[15:0]  - Error details
```

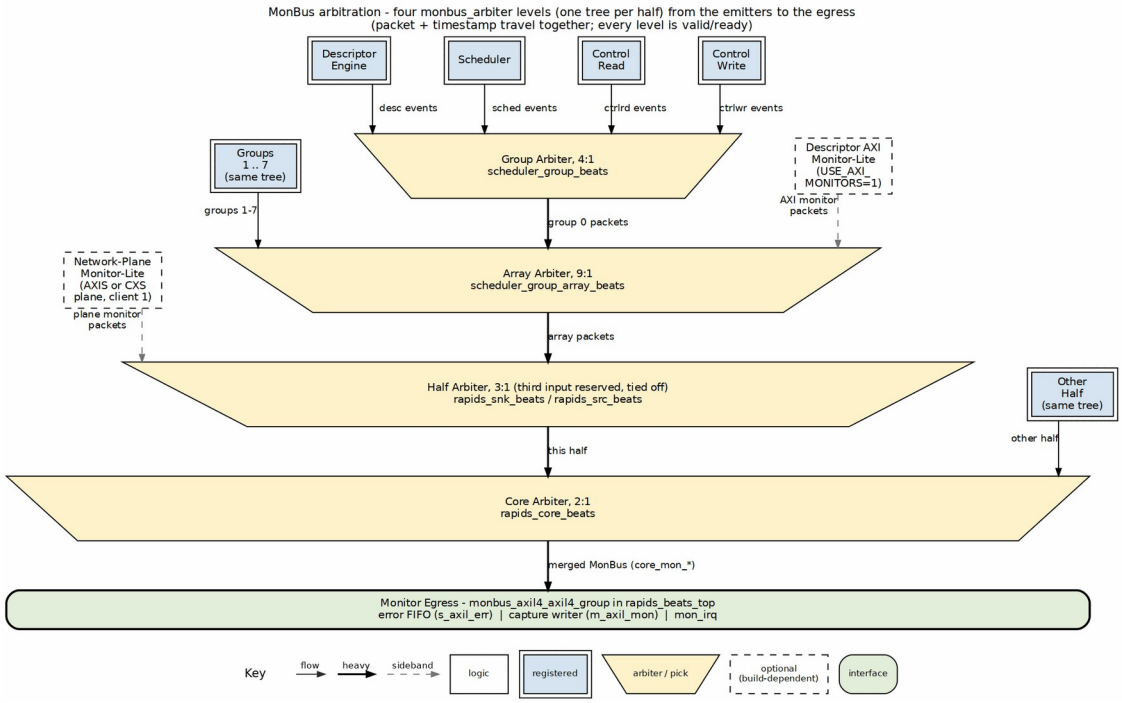
15.6.3 State Transition Event Data

```
[34:32] - Reserved
[31:28] - Previous state
[27:24] - New state
[23:0]  - Timestamp (lower bits)
```

15.7 MonBus Arbitration

Multiple RAPIDS modules generate MonBus packets. Internal arbitration ensures ordered delivery:

15.7.1 Figure 3.5.1: MonBus Arbitration Tree



MonBus Arbitration Tree

Source: ch03_monbus_arbitration.dot

15.7.2 Arbitration Policy

- Round-robin across modules
- No packet loss (backpressure if output blocked)
- Priority boosting for ERROR packets

15.8 Top-Level MonBus Delivery (monbus_axil4_axil4_group)

At the top level (`rapids_beats_top`, `rapids_beats_cxs_top`), the internal MonBus is not exposed as a raw 64-bit port. Instead, when `USE_AXI_MONITORS = 1`, the core's single merged stream (`core_mon_*`, which already carries the descriptor-monitor and network-plane packets) feeds a `monbus_axil4_axil4_group`. The data masters `m_axi_rd` and `m_axi_wr` carry no monitors. When `USE_AXI_MONITORS = 0`, the group is not built and its outputs are tied off (`mon_irq = 0`). The group presents three external interfaces:

Top-Level MonBus AXI-Lite Group

Interface	Signals	Purpose
Error-drain slave	`s_axil_err_ar*`, `s_axil_err_r*` (AXI-Lite, 32-bit read)	CPU reads captured error events from the error FIFO
Capture master	`m_axil_mon_aw*`, `m_axil_mon_w*` (64-bit `wdata`), `m_axil_mon_b*`	Bulk-writes the MonBus trace to system memory
Interrupt	`mon_irq`	Asserts on error/threshold events

The capture region and flush behavior are configured by `cfg_mon_base_addr`, `cfg_mon_limit_addr`, and `cfg_mon_flush_watermark`.

15.9 Integration Notes

15.9.1 Downstream Connection

Internally, MonBus output connects to:

1. **MonBus Arbiter** - Combines core + rd/wr monitor sources
2. **monbus_axil4_axil4_group** - Error-drain slave, capture master, and `mon_irq`
3. **Debug FIFO** - Buffered access for debug tools

15.9.2 Packet Rate

MonBus Packet Rates

Scenario	Approximate Rate
Idle	0 packets/cycle
Single channel active	~1 packet/100 cycles
All channels active	~1 packet/20 cycles
Error storm	Up to 1 packet/cycle

16 CXS Network Interface

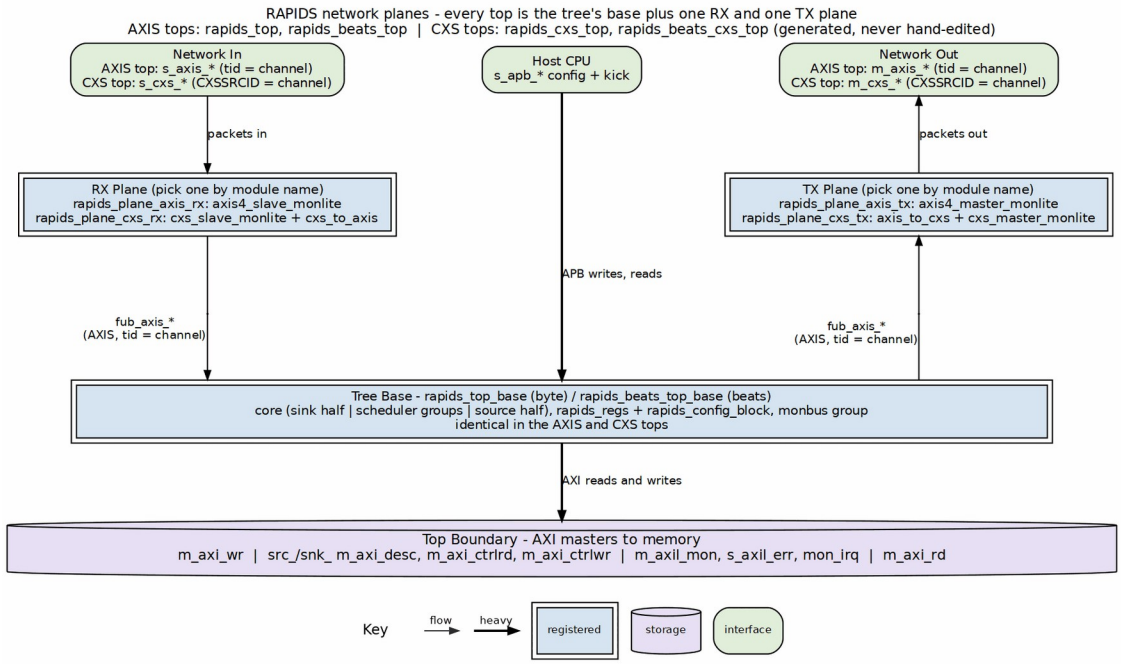
16.1 Overview

The network side of RAPIDS is a swappable **network plane**. Each top is the tree's base (core, registers, monitor group) plus one receive (RX) plane and one transmit (TX) plane, and the plane is chosen by the top's module name:

- `rapids_beats_top`, `rapids_top`: AXI-Stream planes; the `s_axis_*` and `m_axis_*` ports described in the AXIS Interface chapter.
- `rapids_beats_cxs_top`, `rapids_cxs_top`: CXS planes; the **sink port** (`s_cxs_*`) into the sink half and the **source port** (`m_cxs_*`) out of the source half, one CXS link each.

The core never sees CXS. Every plane presents the same AXIS stream (`tid` = channel) to the core, so the descriptor engines, schedulers, SRAM buffers and AXI engines are identical in the AXIS and CXS tops. All four tops are generated from the tree's base by `bin/gen_plane_tops.py`.

16.1.1 Figure 3.6.1: Network Planes



Network Planes

Source: ch03_network_planes.dot

16.2 Link Profile

The CXS planes use one fixed profile of Arm IHI 0079 (CXS) Issue D. Issue D is the first issue with the SRCID, TGTID and USER sidebands, which RAPIDS needs to carry the channel and the byte strobes.

CXS Link Profile

Property	Value
Links	One per direction: the sink port `s_cxs_*` into the sink half, the source port `m_cxs_*` out of the source half
Channel	`CXSSRCID` carries the RAPIDS channel (the AXIS `tid`)
`CXSTGTID`	The AXIS `tdest`
`CXSUSER`	`{tuser, tstrb}`; `tuser` is dropped when `DATA_WIDTH/8 + AXIS_USER_WIDTH > 128`
Packets per flit / start alignment	`CXSMAXPKTPERFLIT` = 2, 16-byte alignment; one AXIS packet is one CXS packet
Stream framing	Per stream, keyed on the channel bits of `CXSSRCID`: channels may interleave mid-packet
Link control	Explicit credit return, activation and deactivation; idle deactivation after `CXS_IDLE_CYCLES`
Parity	Off
Data width	`DATA_WIDTH` up to 1024 (`CXSUSER` is at most 128 bits and carries `tstrb`)

Channels interleave on one link. The byte-granular source arbitrates per drain block while a packet spans its whole descriptor, so with two or more source channels active the egress switches channel in the middle of a packet. That is legal CXS (streams may interleave, IHI 0079D 2.5.2) because every block on the link frames packets per stream rather than per link.

16.3 Credits

One credit permits one flit (IHI 0079D 2.1). Credits are a single pool for the whole link: a credit grant carries no ID, so it cannot be reserved for one channel.

CXS Top Parameters

Parameter	Default	Meaning
CXS_MAX_CREDIT	32	Credits the receiver grants. The grant-to-flit-to-free round trip is about 3 cycles, so 32 sustains one flit per cycle with margin for link pipelining or a clock crossing
CXS_RX_FIFO_DEPTH	CXS_MAX_CREDIT	The sink plane's receive FIFO; it is the credit pool, so every granted credit is backed by space
CXS_CREDIT_BEYOND_SPEC	CXS_MAX_CREDIT > 63	Above the spec's limit of 63 the link is built beyond spec, and the far end must be built the same way
CXS_IDLE_CYCLES	32	Idle cycles before the TX plane deactivates the link

The credit count is not tied to SRAM_DEPTH. SRAM_DEPTH is the beats each channel buffers in each direction; the credit pool only has to cover the link's round trip.

16.4 Signal List

The sink port (`s_cxs_*`) is a CXS receiver; the source port (`m_cxs_*`) is a CXS transmitter. Direction is given at the RAPIDS top.

CXS Link Signals

Signal	Sink port (`s_cxs_*`)	Source port (`m_cxs_*`)	Width	Description
`*_cxs_valid`	input	output	1	Flit valid
`*_cxs_data`	input	output	`DATA_WIDTH`	Flit data
`*_cxs_cntl`	input	output	`CXS_CNTL_PW`	Packet start/end control (CXSCNTL)
`*_cxs_srcid`	input	output	`AXIS_ID_WIDTH`	Source ID; the channel
`*_cxs_tgtid`	input	output	`AXIS_DEST_WIDTH`	Target ID; the AXIS `tdest`
`*_cxs_user`	input	output	`CXS_UW`	`{tuser, tstrb}`
`*_cxs_crdgnt`	output	input	1	Credit grant
`*_cxs_crdrtn`	input	output	1	Credit return
`*_cxs_activereq`	input	output	1	Link activation request
`*_cxs_activeack`	output	input	1	Link activation acknowledge
`*_cxs_deacthint`	output	input	1	Deactivation hint

CXS Status Signals

Signal	Direction	Description
`snk_cxs_link_state`, `src_cxs_link_state`	output [1:0]	`{ACTIVEREQ, ACTIVEACK}` of each link
`snk_cxs_protocol_error`, `src_cxs_protocol_error`	output	Sticky: the endpoint saw a CXS protocol violation
`snk_cxs_unsupported`	output	Sticky: a received flit that AXIS framing cannot represent

16.5 Flow Control and the Sink Contract

The byte-granular sink holds `s_axis_tready` low on an AXIS top until the channel's descriptor has been fetched. On a CXS top the same backpressure appears as credits: the receive FIFO fills and the sink plane stops granting credits until the core drains it.

16.6 Monitoring

Each CXS plane carries a `cxs_monitor_lite` in place of the AXIS monitor-lite. It is programmed from the same CSRs (`SNK.WRMON_*` for the sink plane, `SRC.RDMON_*` for the source plane) and reports with the same agent IDs (0x09 sink, 0x0A source). Its packets are tagged `PROTOCOL_CXS` and join the half's 3:1 `monbus_arbiter` as client 1, exactly where the AXIS plane's packets go. See the Monitor Bus Interface chapter.

16.7 Routing and Payload Type (Proposed)

A proposal (2026-10-16, not implemented) gives every packet a header carrying its destination coordinate, source coordinate, payload type (`cfg` / `prog` / `mem`), length and target address. The header belongs to the tile network, not to CXS, and is described once in the compute-engine tile's `TILE_SPEC.md` Section 11. CXS only transports it, as the first flit of the packet. Nothing on this page changes: `CXSSRCID` keeps the channel, so per-stream framing is the same. The descriptor fields that supply the header are in the Descriptor Format chapter.

17 Use Case: Network-to-Memory Transfer

17.1 Scenario

A network agent delivers a packet of `L` bytes on the sink AXI-Stream. RAPIDS writes exactly those bytes to system memory starting at the byte address `A` named by the channel's descriptor. The packet may start anywhere in a beat, and the bytes before `A` and after `A + L - 1` in the boundary beats are left untouched.

The examples use a 256-bit data path (32 byte lanes, `DATA_WIDTH` = 256, the Genesys 2 configuration). Byte lanes are numbered 0 to 31.

17.2 Sequence

Network-to-Memory Sequence

Step	Actor	Action
1	Software	Builds a descriptor: destination byte address `A`, length `L` in bytes, and the channel's sink direction enabled (Descriptor Format)
2	Software	Kicks the channel so the descriptor engine fetches the descriptor
3	Scheduler	Computes the beat count and hands the sink data path a packet record: `{bytes = L, offset = A mod 32}`
4	Network agent	Streams the packet on `s_axis_*` with `TID` = channel. `TREADY` rises once the record exists
5	Sink ingress	Shifts each packed stream beat up by the offset and writes memory-aligned beats, with byte enables, into the channel's SRAM partition
6	Sink AXI master	Drains the SRAM as AXI4 write bursts, never crossing 4 KB, with `WSTRB` from the stored byte enables
7	Sink ingress	At `TLAST`, compares the packet's byte count with `L`; a mismatch sets the channel's sticky write-error flag
8	Scheduler	On the last write response, completes the descriptor and reports it on the MonBus

17.2.1 Ordering Rule

Issue the descriptor before the packet arrives, or stream one channel at a time. `s_axis_tready` is one signal and it stays low for a beat whose channel has no packet record yet. Until that record exists the whole stream is held, including beats for other channels queued behind it. See AXIS Interface.

17.3 Worked Example 1: 77 Bytes at Offset 5

Destination `A` = `base + 5`, where `base` is beat-aligned, and `L` = 77.

Example 1 Quantities

Quantity	Value
Offset	5
Beats written	(5 + 77 + 31) >> 5 = 3
Stream beats	3: 32, 32 and 13 bytes
Last stream beat `TSTRB`	`0x00001FFF`

Example 1 Write Beats

Memory beat	Address	Lanes written	Bytes	`WSTRB`
0	`base`	5 to 31	27	`0xFFFFFFE0`
1	`base + 32`	0 to 31	32	`0xFFFFFFFF`
2	`base + 64`	0 to 17	18	`0x0003FFFF`

The three beats carry 27 + 32 + 18 = 77 bytes. Stream byte 0 lands in lane 5 of the first memory beat. The five bytes that do not fit in that beat are held and join the next one, so each memory beat is assembled from two neighboring stream beats. The 13 bytes of the last stream beat plus the five held bytes fill lanes 0 to 17 of the last memory beat, so no extra flush beat is needed.

The write is one AXI4 burst of three beats at `base`. `AxADDR` is the aligned address and `AxSIZE` is the full 32-byte beat.

17.4 Worked Example 2: 4035 Bytes Crossing a 4 KB Boundary

Destination A = 0x0CB (byte address within a 4 KB page), L = 4035.

Example 2 Quantities

Quantity	Value
Offset	0x0CB mod 32 = 11
Aligned start	0x0C0
Beats written	(11 + 4035 + 31) >> 5 = 127
Beats before the 4 KB boundary	(0x1000 - 0x0C0) / 32 = 122
Beats after the boundary	5
First beat WSTRB	lanes 11 to 31, 0xFFFFF800
Last beat	at 0x1080, lanes 0 to 13, WSTRB = 0x00003FFF

The 127 beats hold 21 + 125 x 32 + 14 = 4035 bytes. The sink write master cannot issue one 127-beat burst, because it would cross 0x1000. It splits at the boundary:

Example 2 Write Bursts

Burst	AWADDR	Beats
1	0x0C0	up to 122
2	0x1000	5

If the configured write burst limit (WR_XFER_BEATS) is below 122, burst 1 is itself split; the 4 KB cap and the configured limit are combined and the smaller wins. After the first burst the address continues from the aligned address plus the beats already written.

17.5 Errors

Network-to-Memory Error Cases

Condition	Result
Packet byte count differs from L	Channel's sticky write-error flag is set and stays set until reset (Error Handling)
Write response is not OKAY	AXI write error is reported through the same flag
Zero-length descriptor	No record, no packet expected, nothing written

17.6 Related Chapters

- Descriptor Chaining, shared with RAPIDS Beats
- Multi-Channel Operation, shared with RAPIDS Beats

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

18 Use Case: Memory-to-Network Transfer

18.1 Scenario

RAPIDS reads L bytes from system memory starting at the byte address A named by the channel's descriptor and emits them on the source AXI-Stream as one packed packet. The read may start anywhere in a beat. The stream carries only the requested bytes: the first stream beat starts at lane 0 with byte A, and TLAST marks the descriptor's last byte.

The examples use a 256-bit data path (32 byte lanes, the Genesys 2 configuration).

18.2 Sequence

Memory-to-Network Sequence

Step	Actor	Action
1	Software	Builds a descriptor: source byte address `A`, length `L` in bytes, the channel's source direction enabled (Descriptor Format)
2	Software	Kicks the channel
3	Scheduler	Computes the beat count and hands the source data path a packet record: `{bytes = L, offset = A mod 32}`
4	Source AXI master	Reads whole aligned beats as AXI4 read bursts, never crossing 4 KB, into the channel's SRAM partition
5	Source egress	Pops beats from the SRAM and shifts them down by the offset. The first pop only primes the shifter
6	Source egress	Emits packed stream beats with `TID` = `TDEST` = channel; the last beat carries a contiguous `TSTRB` and `TLAST`
7	Scheduler	When the packet has left, completes the descriptor and reports it on the MonBus

Unlike the sink, the source needs no ordering rule against the network: the record is created before any read data is requested, so the egress always knows the packet length.

18.3 Worked Example 1: 77 Bytes at Offset 5

Source A = `base + 5` with `base` beat-aligned, L = 77.

Example 1 Quantities

Quantity	Value
Offset	5
Memory beats read	(5 + 77 + 31) >> 5 = 3
Stream beats	3: 32, 32 and 13 bytes
Last stream beat `TSTRB`	`0x00001FFF`

The read master issues one burst of three beats at `base`. The egress then works as follows:

Example 1 Egress Steps

Memory beat popped	Output
M0	None. Bytes 5 to 31 are held: 27 bytes
M1	Stream beat 0: the 27 held bytes plus bytes 0 to 4 of M1, `TSTRB` = `0xFFFFFFFF`
M2	Stream beat 1: bytes 5 to 31 of M1 plus bytes 0 to 4 of M2, `TSTRB` = `0xFFFFFFFF`
(flush)	Stream beat 2: the remaining 13 bytes, `TSTRB` = `0x00001FFF`, `TLAST`

The stream carries 32 + 32 + 13 = 77 bytes as one packet.

18.4 Worked Example 2: 4035 Bytes Crossing a 4 KB Boundary

Source A = 0x0CB, L = 4035.

Example 2 Quantities

Quantity	Value
Offset	11
Memory beats read	127
Beats before the 4 KB boundary	122
Beats after the boundary	5
Stream beats	127: 126 full beats and a last beat of 3 bytes
Last stream beat TSTRB	0x00000007

The read master splits at 0x1000: one burst of up to 122 beats from 0x0C0 and one of 5 beats from 0x1000, further split if the configured read burst limit (RD_XFER_BEATS) is smaller. The egress removes the 11 leading bytes and delivers 4035 bytes as one packet with a single TLAST, however many drain blocks and read bursts the transfer spanned. RAPIDS Beats asserted TLAST at the end of every drain block; RAPIDS does not.

18.5 Errors

Memory-to-Network Error Cases

Condition	Result
Read response is not OKAY	Channel's read error is reported (Error Handling)
Zero-length descriptor	No record and no packet

18.6 Related Chapters

- Descriptor Chaining, shared with RAPIDS Beats
- Multi-Channel Operation, shared with RAPIDS Beats

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

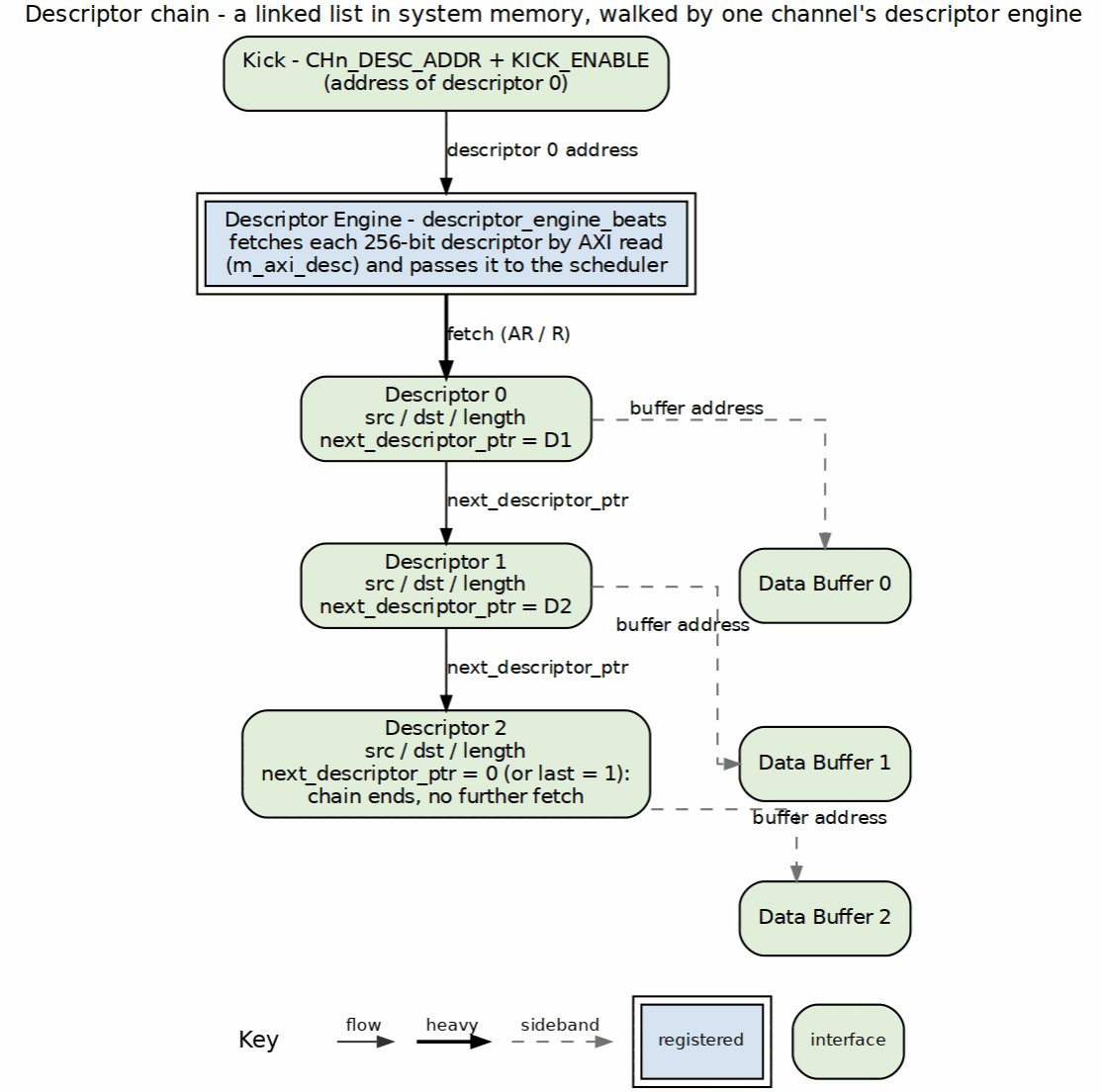
19 Use Case: Descriptor Chaining

19.1 Overview

Descriptor chaining enables multiple transfers to execute sequentially without software intervention. Each descriptor contains a pointer to the next descriptor, forming a linked list that RAPIDS processes automatically.

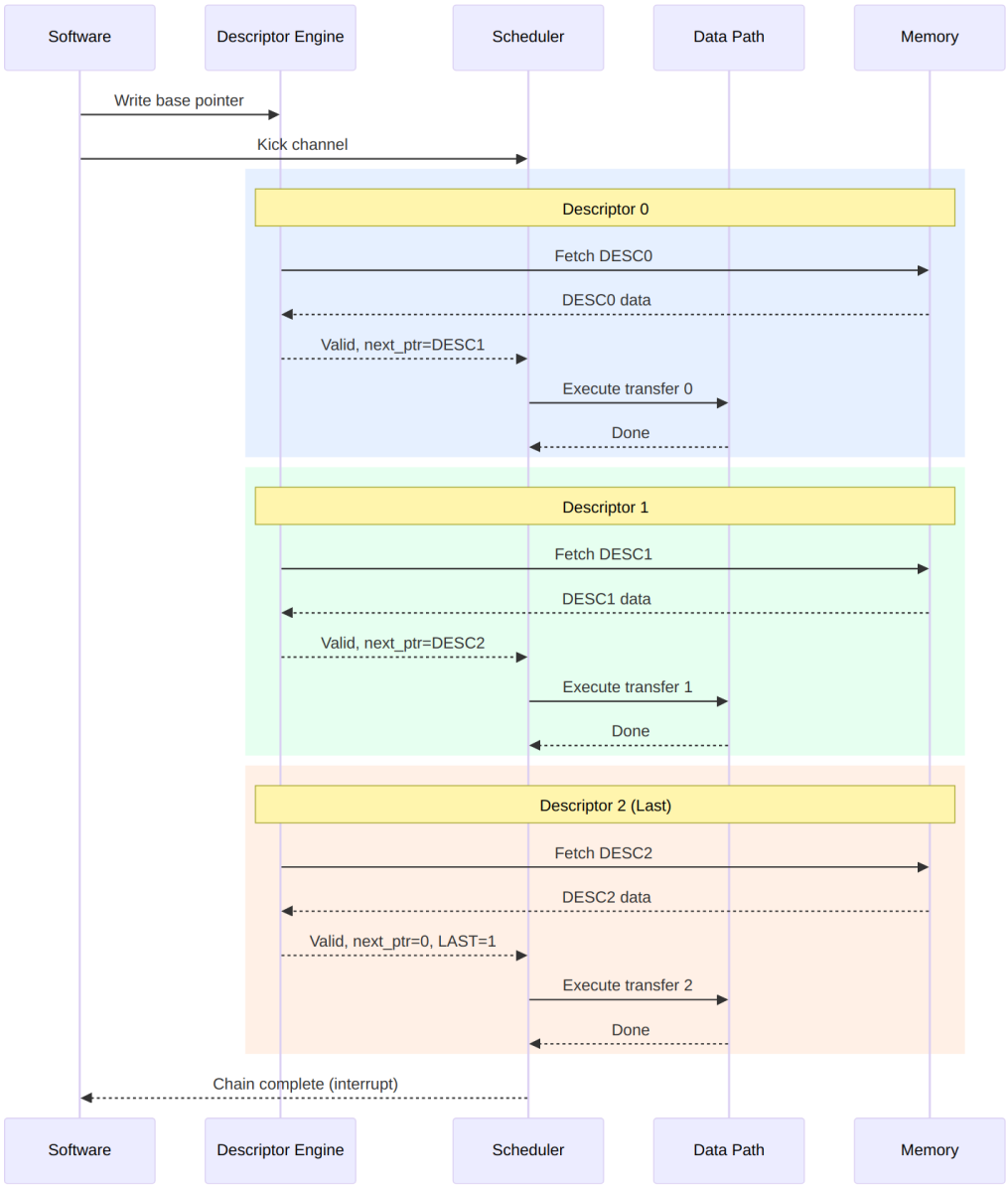
19.2 Chain Structure

19.2.1 Figure 4.3.1: Descriptor Chain Structure

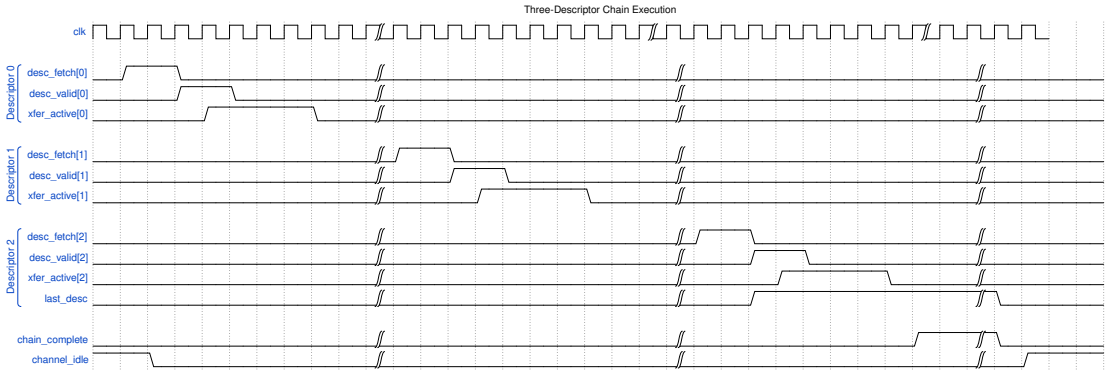


Descriptor Chain Structure

Source: ch04_chain_structure.dot



Chain Execution Flow



Timing Diagram

19.3 Descriptor Fields for Chaining

19.3.1 Relevant Fields

Chaining-Related Descriptor Fields

Field	Bits	Description
`next_ptr`	[191:128]	64-bit pointer to next descriptor
`last`	[255]	Last descriptor in chain flag
`irq_en`	[254]	Generate interrupt on completion

19.3.2 Chain Termination

A chain terminates when ANY of these conditions is true:

1. `next_ptr == 0` (null pointer)
2. `last == 1` (explicit last flag)
3. Error during transfer

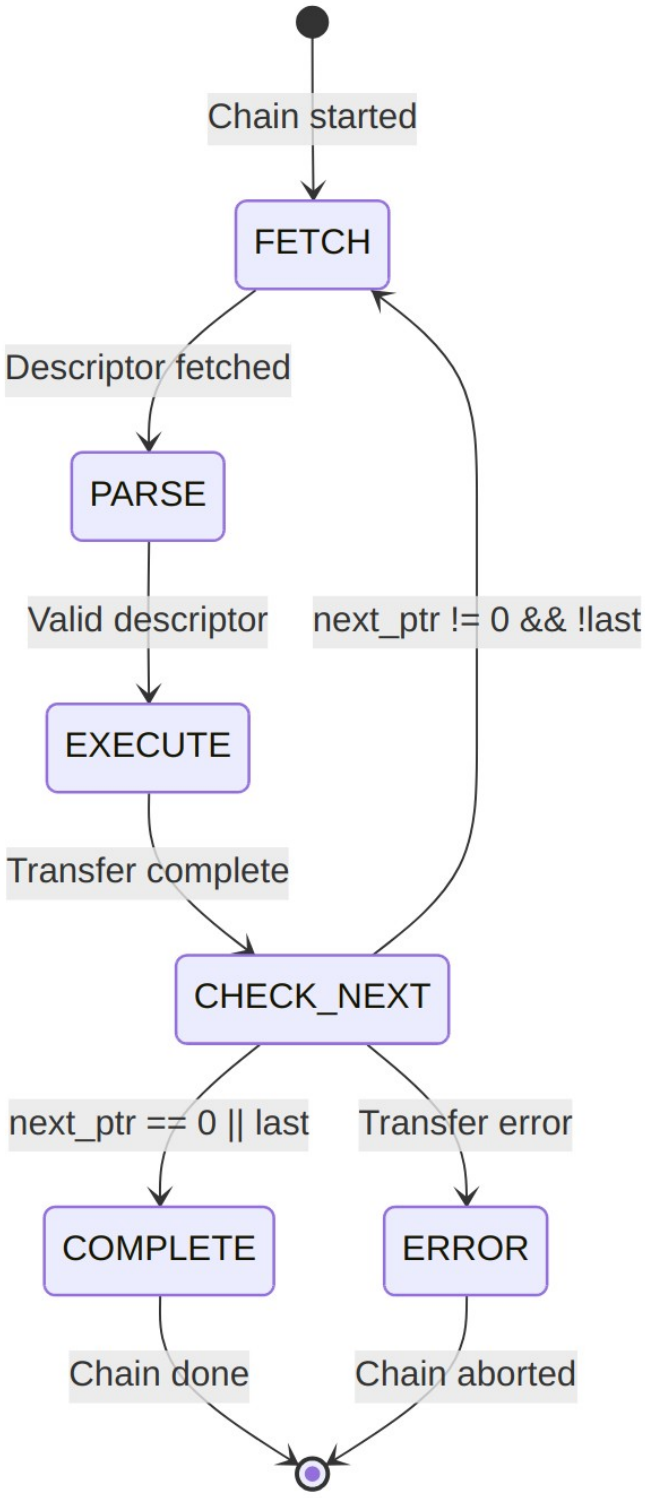
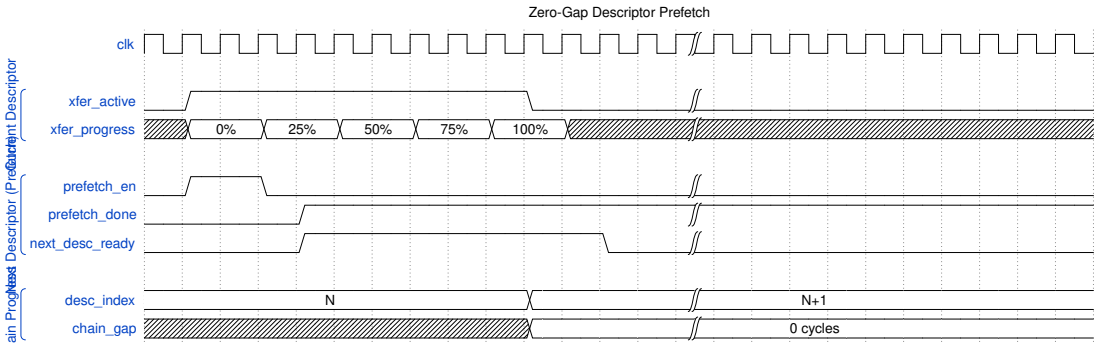

Chain started
FETCH
Descriptor fetched
PARSE
Valid descriptor
next_ptr != 0 && !last
EXECUTE
Transfer complete
CHECK_NEXT
next_ptr == 0 || last
Transfer error
COMPLETE
ERROR
Chain done
Chain aborted

Diagram 6

19.4 Prefetch Optimization

19.4.1 Descriptor Prefetching

RAPIDS can prefetch the next descriptor while the current transfer executes:



Waveform 7

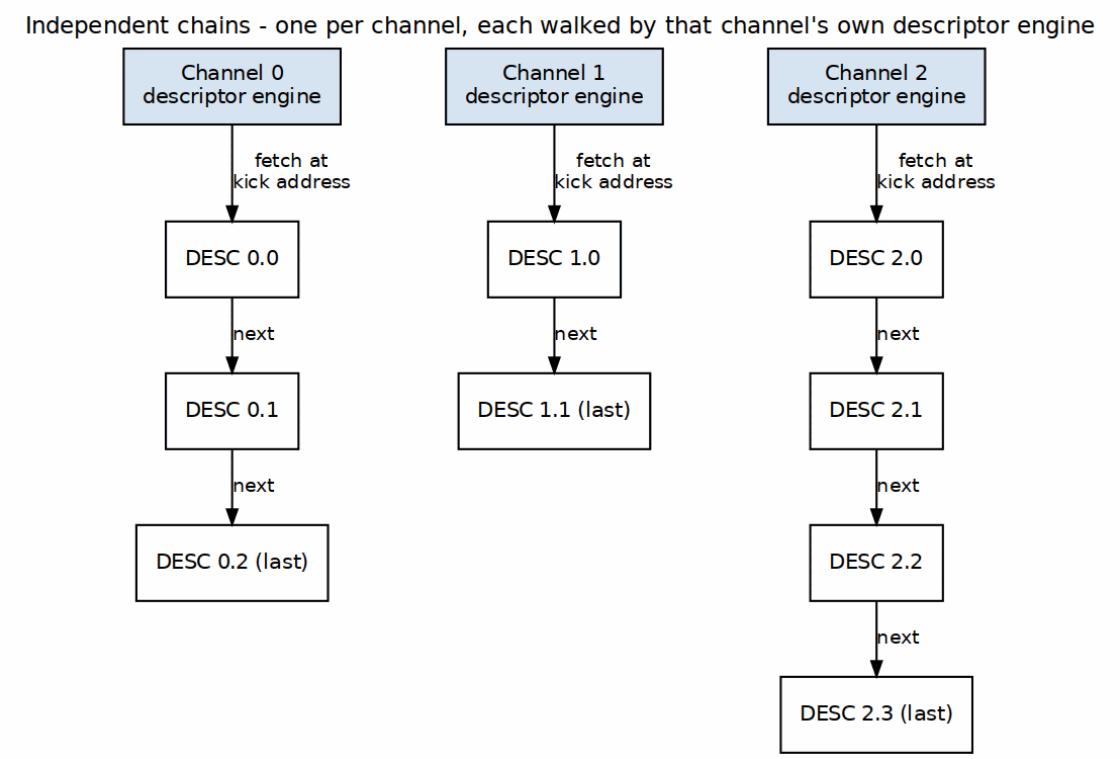
Benefit: Zero-cycle gap between chained transfers when prefetch completes before current transfer.

Prefetch depth is controlled by `DESCENG_CONFIG.PREFETCH_EN` and `DESCENG_CONFIG.FIFO_THRESH` (register block, offset 0x220). With prefetch disabled the descriptor engine fetches on demand (roughly one descriptor ahead of the scheduler); with prefetch enabled it buffers up to `FIFO_THRESH` descriptors ahead. A chain fetch that is throttled (output FIFO at the threshold) is deferred and reissued automatically once the FIFO drains, so no chain link is lost.

19.5 Multi-Channel Chaining

Each channel maintains independent chains:

19.5.1 Figure 4.3.2: Independent Per-Channel Chains



Independent Per-Channel Chains

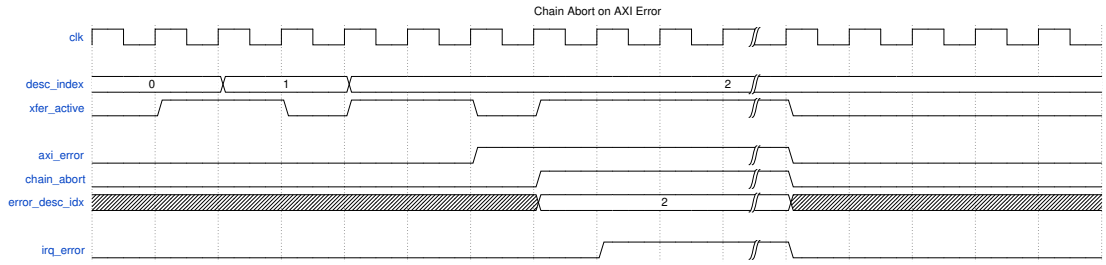
Source: ch04_multichannel_chains.dot

Channels execute their chains concurrently and independently.

19.6 Error During Chain

19.6.1 Error Handling

When an error occurs mid-chain:



Waveform 8

Error Response: 1. Current transfer aborts 2. Chain processing stops 3. Error status captures failing descriptor index 4. MonBus reports error event 5. Interrupt generated (if enabled)

19.6.2 Recovery

Software must: 1. Read error status to identify failing descriptor 2. Fix the issue (descriptor content, memory mapping, etc.) 3. Restart chain from failing descriptor or beginning

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

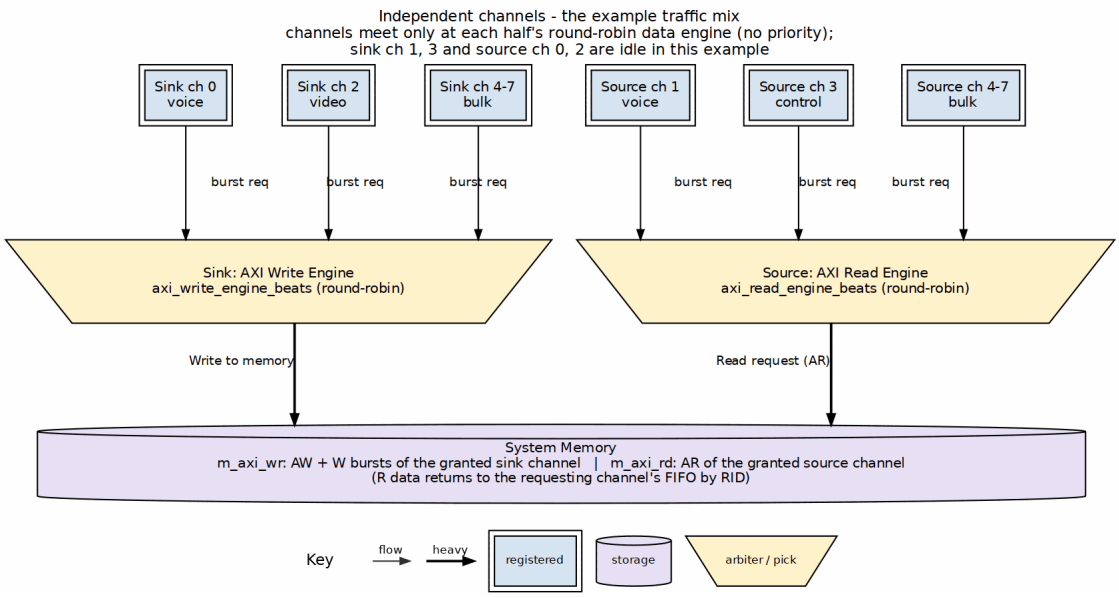
20 Use Case: Multi-Channel Operation

20.1 Overview

RAPIDS Beats supports 8 independent DMA channels that can operate concurrently. This enables parallel data transfers for multiple network connections, priority-based scheduling, and efficient resource utilization.

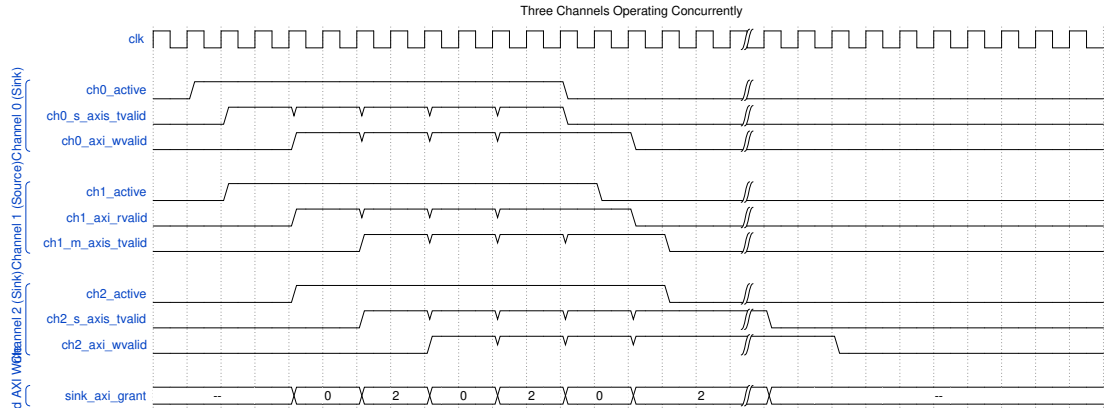
20.2 Channel Independence

20.2.1 Figure 4.4.1: Independent Channels



Independent Channels

Source: ch04_channel_independence.dot

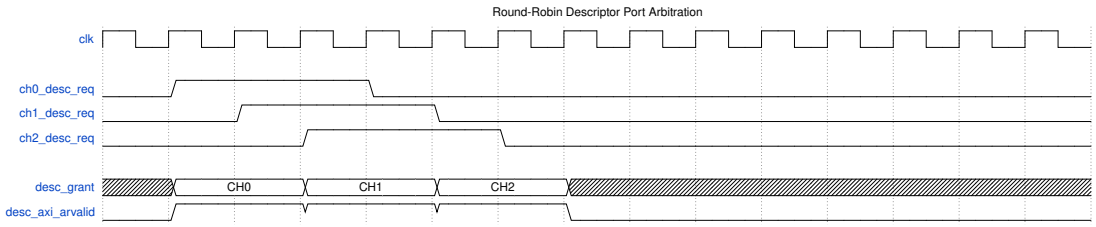


Concurrent Operation Timing

20.3 Resource Arbitration

20.3.1 Descriptor Port Arbitration

All channels share a single AXI port for descriptor fetches:

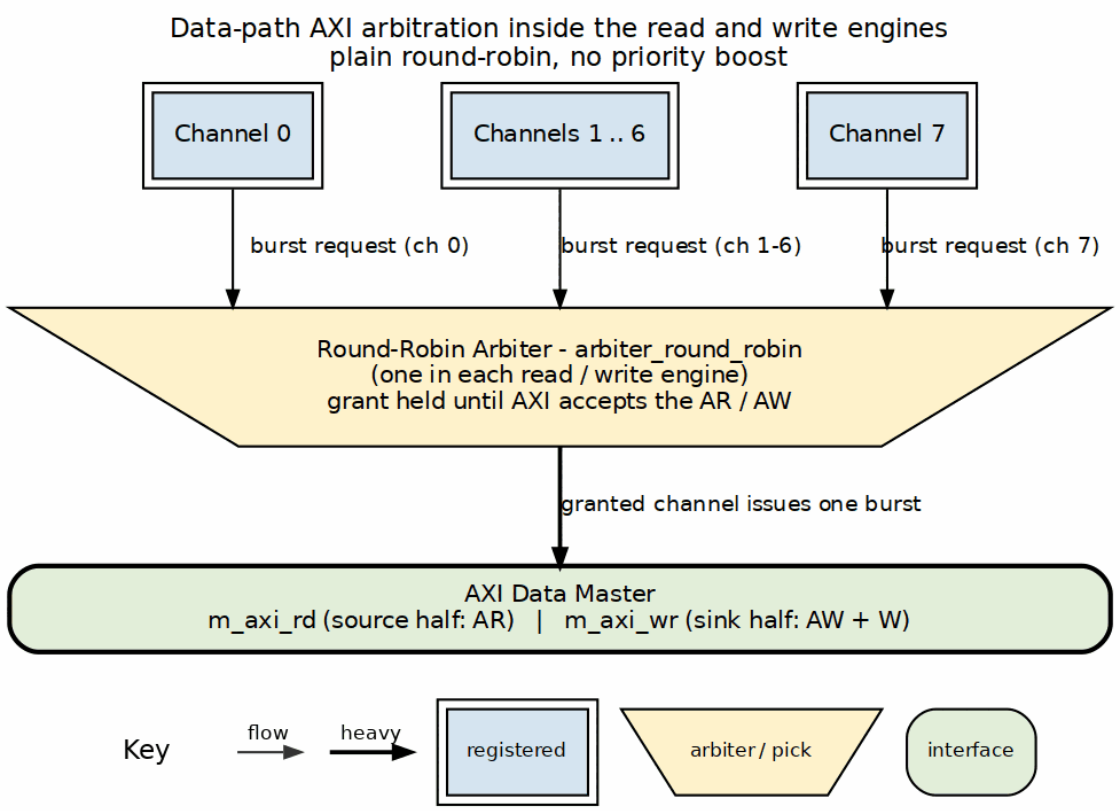


Waveform 10

20.3.2 Sink/Source Arbitration

Data path AXI ports use plain round-robin arbitration (`arbiter_round_robin` inside each engine). There is no priority boost: the descriptor's priority field is carried but not used for arbitration.

20.3.3 Figure 4.4.2: Data-Path AXI Arbitration



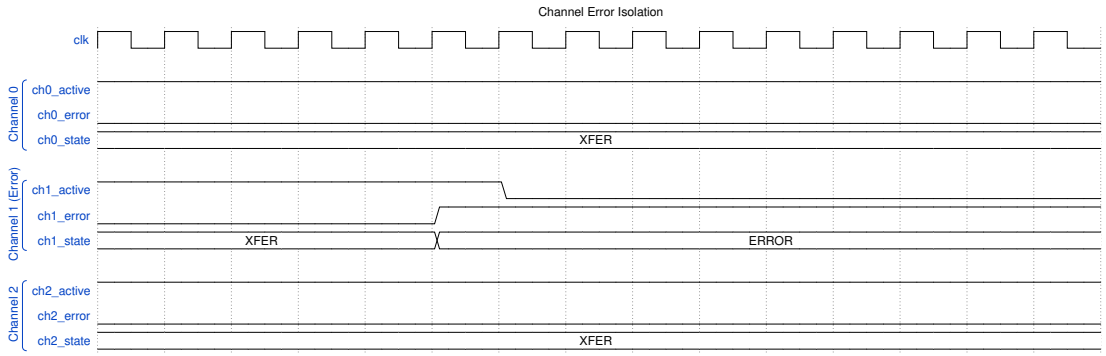
Data-Path AXI Arbitration

Source: ch04_datapath_arbitration.dot

20.4 Channel Isolation

20.4.1 Error Isolation

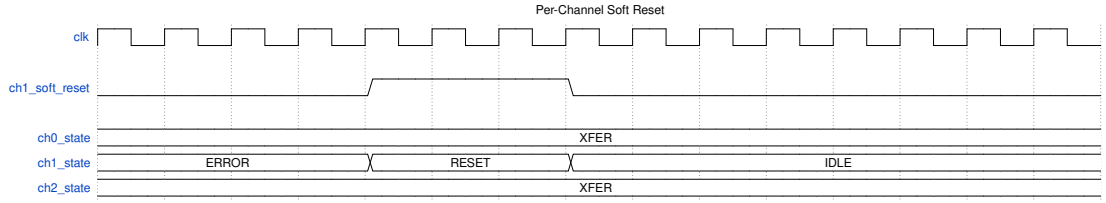
Errors in one channel do not affect other channels:



Waveform 11

20.4.2 Per-Channel Soft Reset

Individual channels can be reset without affecting others:



Waveform 12

20.5 Use Case Examples

20.5.1 Mixed Traffic Pattern

Typical network application with different traffic types:

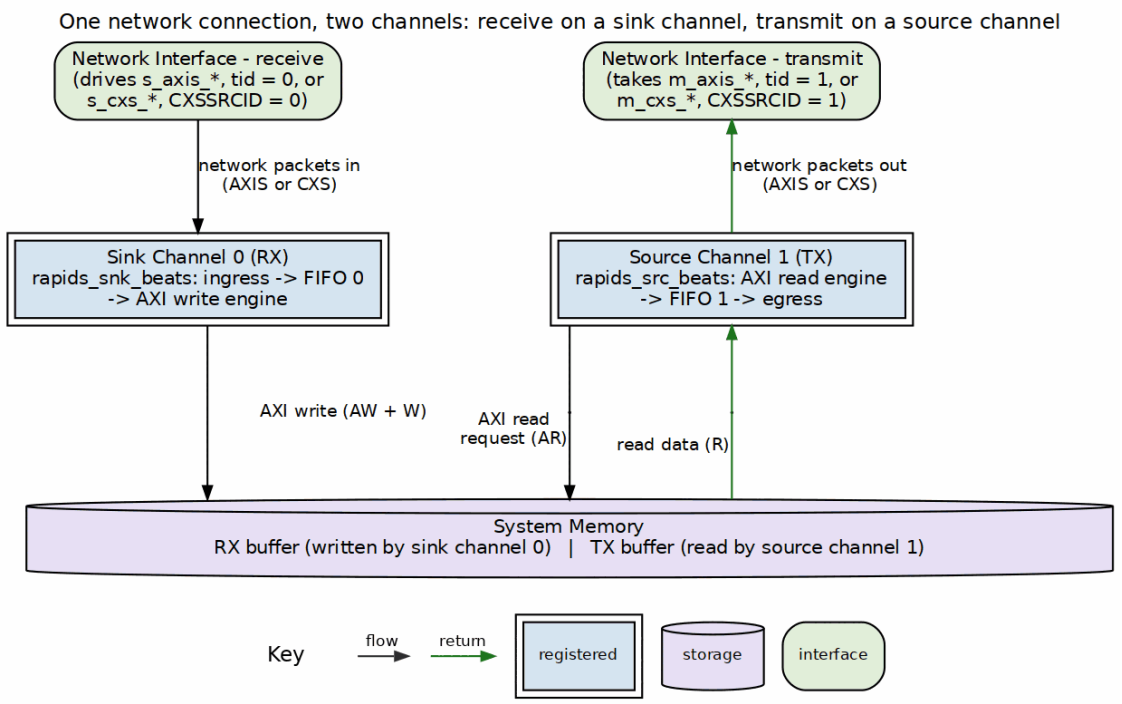
Mixed Traffic Channel Assignment

Channel	Traffic Type	Direction	Priority	Notes
0	Voice/RTP	Sink	High	Low latency required
1	Voice/RTP	Source	High	Low latency required
2	Video	Sink	Medium	High bandwidth
3	Control	Source	Medium	Small packets
4-7	Bulk data	Both	Low	Background transfers

20.5.2 Bidirectional Transfer

Single network connection using paired channels:

20.5.3 Figure 4.4.3: Bidirectional Transfer



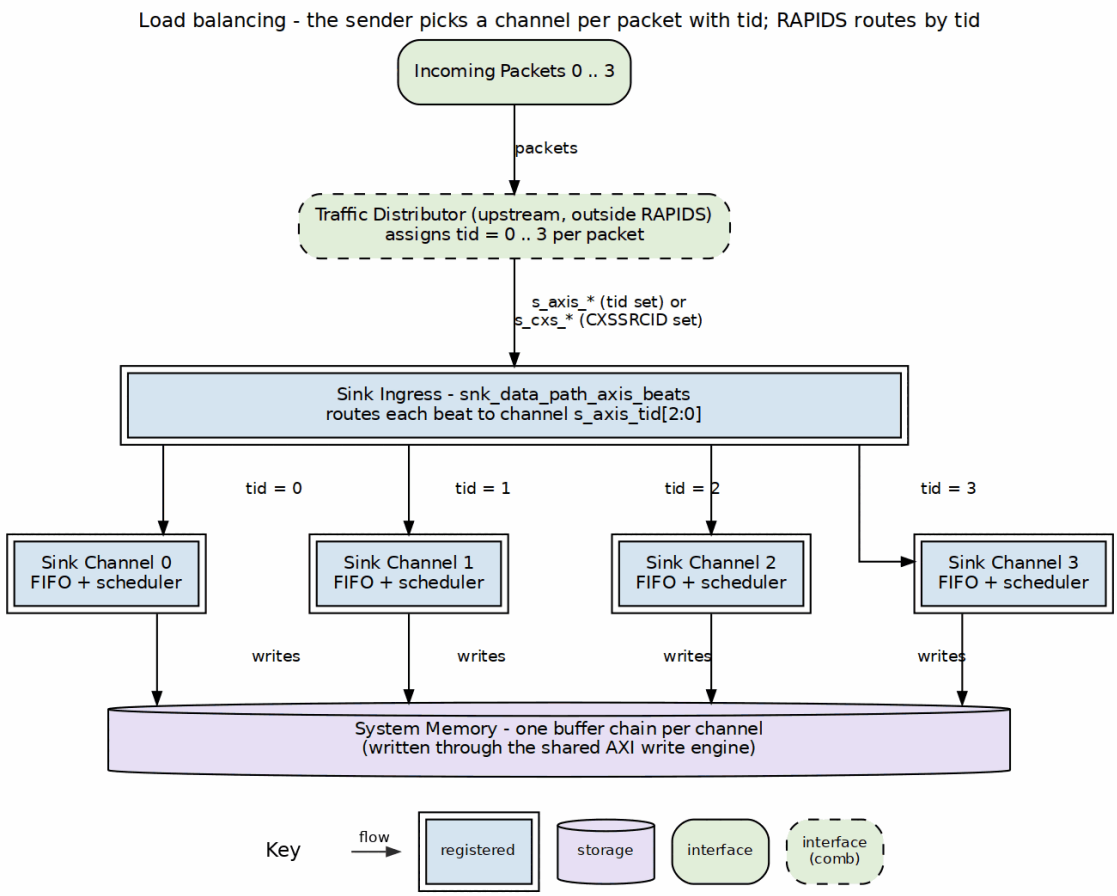
Bidirectional Transfer

Source: ch04_bidirectional.dot

20.5.4 Load Balancing

Multiple channels processing same traffic type:

20.5.5 Figure 4.4.4: Load Balancing by tid



Load Balancing by tid

Source: ch04_load_balancing.dot

20.6 Performance Considerations

20.6.1 Bandwidth Sharing

Bandwidth Distribution

Active Channels	Per-Channel BW (% of max)	Notes
1	100%	Full bandwidth available
2	~50% each	Round-robin sharing
4	~25% each	Fair distribution
8	~12.5% each	All channels active

Note: Actual distribution depends on transfer sizes and memory latency.

20.6.2 Latency Impact

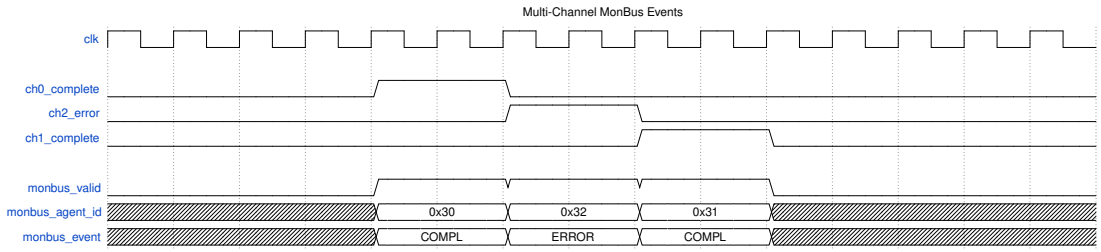
Multi-channel operation adds arbitration latency:

Arbitration Latency

Scenario	Additional Latency
Single channel	0 cycles
2 channels contending	1-2 cycles
8 channels contending	Up to 7 cycles

20.7 MonBus Events

Each channel generates independent MonBus events:



Waveform 13

Events are arbitrated and serialized on the shared MonBus output.

21 Descriptor Format

21.1 Overview

RAPIDS uses 256-bit (32-byte) descriptors to define DMA transfers. Descriptors are stored in system memory and fetched by the Descriptor Engine when a channel is kicked. The layout is the layout RAPIDS Beats uses, bit for bit. What changed is the meaning of two things: the `length` field counts BYTES, and `src_addr` / `dst_addr` are byte addresses with no alignment requirement.

A descriptor therefore describes a transfer the way software thinks of it. A packet of 37 bytes at address 0x1007 is a descriptor with `length` = 37 and `dst_addr` = 0x1007. The hardware turns that into beats, byte enables and a partial last beat; software never does the arithmetic.

21.2 Descriptor Layout

21.2.1 256-Bit Structure

Bit Range	Field Name	Width	Description
[255:213]	reserved	43	Reserved (write 0)
[212:210]	desc_type	3	0 = LEGACY, 1 = EXTENDED (see below)
[209:208]	opcode	2	0 = DATA, 1 = CTRL_READ, 2 = CTRL_WRITE
[207:200]	priority	8	Transfer priority (informational)
[199:196]	channel_id	4	Channel ID (informational)
[195]	error	1	Error flag (written by hardware)
[194]	last	1	Last descriptor in chain
[193]	gen_irq	1	Generate interrupt on completion
[192]	valid	1	Descriptor valid flag
[191:160]	next_descriptor_ptr	32	Address of next descriptor (0 = last)
[159:128]	length	32	Transfer length in BYTES
[127:64]	dst_addr	64	Destination byte address
[63:0]	src_addr	64	Source byte address

: 256-Bit Descriptor Layout (`rapids_pkg::descriptor_t`)

Tile header fields (`TILE_HDR` = 1, 2026-10-17): `ptype` at `[215:213]`, `op` (write / read) at `[216]` and, on SOURCE descriptors, a destination coordinate and target address in `dst_addr` (`[63:49]` the 15-bit coordinate `{ext, level, X, Y, Z}`, `[48:0]` the target), which is otherwise unused there. They supply the tile packet header defined in the compute-engine tile's `TILE_SPEC.md` Section 11, and are ignored when `TILE_HDR` = 0. The field table is in the RAPIDS Beats Descriptor Format chapter (the positions are shared; only this design implements them).

Tile credits (`TILE_CREDITS` = 1, 2026-10-18): a tile descriptor waits until the advertisement window after reset closes (`SRC.TILE_CREDIT_STATUS.ADV_DONE`). Its destination must then be legal (`SRC.TILE_LEGAL_DEST_LO` / `_HI`): a delta node's internal ID always is, and a cell or control core is if it advertised credits in the window. Otherwise the descriptor is refused as invalid and its channel errors. A packet to a cell or core leaves only when it has returned enough credits for every flit, header included; a packet to a node is never held. A read request (`op` = 1) to a node is refused: a node never replies.

Straps toward RAPIDS (`TILE_CREDITS` = 1, 2026-10-19): traffic into RAPIDS is not granted; each sender holds a fixed strap. A cell writes against `TILE_LAND_FLITS` (default 16) flits of landing space of its own, so a cell whose sink channel is full blocks only itself. A control core sends `desc` against `TILE_DESC_PER_CORE` (default 2) descriptor-intake entries. RAPIDS returns every flit and

entry it consumes, discards included, as a header-only `credit` packet on its own port (`m_tile_credit_*`), so a return never waits behind data. After a root failover, `SRC.TILE_RESYNC.GO` sends a `set` to every legal sender once the tile is quiescent (`tile_quiescent`); channel resets wait for it too. `SRC.TILE_STALL_CYCLES` arms two stall checks that report on MonBus: sink data that has not moved, and source reads that got no data. The straps are fixed at build time and reported in `TILE_STRAPS`, the last config word of each half (SRC 0x07FC): `LAND_FLITS` [15:0] and `DESC_PER_CORE` [31:16], 0 without `TILE_CREDITS`. The Hive reads it to set the cells' and cores' straps to match. The detail is in the MAS Tile Header chapter.

The authoritative definition is `rapids_pkg::descriptor_t` in `rtl/includes/rapids_pkg.sv`. Only bits `[255:0]` are fetched and used. Every field position is the same as in RAPIDS Beats, so a descriptor builder needs one change: the value written to `length`.

21.3 Field Descriptions

21.3.1 Control Bits [195:192]

Control Bits

Bit	Field	Description
192	`valid`	Must be 1; a descriptor with valid=0 puts the channel in the error state
193	`gen_irq`	Set to 1 to generate a completion interrupt
194	`last`	Set to 1 for the last descriptor in a chain
195	`error`	Error flag; written by hardware, write 0

21.3.2 Direction

There is no direction field in the descriptor. Direction is a property of the **half** that owns the channel. The SOURCE half (memory to AXIS) reads from `src_addr`. The SINK half (AXIS to memory) writes to `dst_addr`. A descriptor is delivered to one half's descriptor engine by the address it was staged at, so the same layout serves both. Each half looks only at its own address, so the two offsets never have to agree.

21.3.3 Transfer Length [159:128]

- 32-bit field specifying the transfer size in BYTES.
- The length is the packet length. On the sink it is the number of bytes the AXIS packet must carry. On the source it is the number of bytes in the one AXIS packet the descriptor produces.
- A length of 0 moves nothing and carries no packet record (see Alignment and Offsets).

21.3.4 Next Pointer [191:160]

- 32-bit address of the next descriptor, zero-extended to the engine's 64-bit address width.
- A value of 0 terminates the chain, as does `last` = 1.
- Autonomous chaining also requires the address to fall inside one of the two configured descriptor address ranges.

Chain traversal is unchanged from RAPIDS Beats. See the shared Descriptor Chaining chapter.

21.3.5 Address Fields [127:0]

For SINK transfers (network to memory):

Field	Usage
`dst_addr` [127:64]	Memory write destination, any byte address
`src_addr` [63:0]	Not used (write 0)

For SOURCE transfers (memory to network):

Address Field Usage

Field	Usage
`dst_addr` [127:64]	Not used (write 0)
`src_addr` [63:0]	Memory read source, any byte address

21.4 Alignment and Offsets

Let `BYTE_LANES` be the beat size in bytes (`DATA_WIDTH / 8`, 32 for the 256-bit build) and `OFF_W` its log2 (5 for 256 bits). For a linear DATA descriptor:

```
offset      = addr[OFF_W-1:0]                     (the low bits of the
byte address)
beats_total = 0                                    if length == 0
            = (offset + length + BYTE_LANES - 1) >> OFF_W   otherwise
```

`beats_total` is the number of whole memory beats the transfer touches. The sum is computed with a 33-bit intermediate, so a length near 4 GB does not overflow. Each half applies the formula to its own address: the source uses `src_addr`, the sink uses `dst_addr`.

Beat Count Examples (BYTE_LANES = 32)

Length (bytes)	Offset	beats_total	Useful bytes / (beats x 32)
0	any	0	no transfer
1	0	1	0.031
2	31	2	0.031
32	0	1	1.000
32	1	2	0.500
37	0	2	0.578
60	5	3	0.625
100	7	4	0.781
203	3	7	0.906

The last column is the fraction of the bus the transfer uses. A transfer that is a whole number of beats and starts on a beat boundary is 1.000. Everything else pays for the beats it touches, not the bytes it carries.

21.4.1 How the offset reaches the bus

- **AXI address.** `AxADDR` is always issued beat-aligned. The offset never appears in the address. Address progression inside a transfer is the aligned-down start plus a beat count times `BYTE_LANES`.
- **Sink writes.** The first beat's `WSTRB` leaves lanes `0` to `offset-1` clear, and the last beat's `WSTRB` covers only the bytes that remain. Bytes outside the packet are never written, so a sub-beat transfer does not disturb its neighbors in memory.

- **Source reads.** The read master fetches whole beats from the aligned address. The egress re-packer drops the leading `offset` bytes and packs the rest, so the AXIS packet starts at byte lane 0.
- **AXIS.** The stream is packed. Only the last beat is partial: `TSTRB` is contiguous from lane 0 and `TLAST` is on that beat. See the AXIS Interface chapter.

21.4.2 4 KB boundaries

A burst never crosses a 4 KB boundary. The engines cut a burst at the boundary, so a transfer that straddles one is issued as more than one burst. This is a protocol rule handled in hardware, not something software arranges. Example: 203 bytes at an address whose low 12 bits are 0xFC3 has offset 3 and 7 beats. The first two beats (aligned addresses 0xFC0 and 0xFE0) end at the boundary. The remaining five beats start at the next 4 KB page.

21.4.3 Zero-length descriptors

A descriptor with `length` = 0 moves nothing. It issues no read or write request and it carries no packet record, whatever its addresses are. It still occupies its place in the chain, so `last`, `next_descriptor_ptr` and `gen_irq` are honored. A zero-length DATA descriptor is therefore a legal way to report a completion event or end a chain without moving data.

21.4.4 Packet record

Every non-zero DATA descriptor produces one packet record, `{bytes, offset}`, for each enabled direction. The data path uses the record to place bytes on the sink or to frame the packet on the source. The record is pulsed once, when the scheduler leaves the fetch state to start the transfer.

- Each channel's record queue is `PQ_DEPTH` deep (4). A descriptor whose record does not fit waits. This is backpressure and not an error.
- The sink ingress does not accept the first beat of a packet until the channel's record exists. Issue the descriptor before the packet arrives, or send them concurrently.
- `s_axis_tready` is one signal, qualified by the `tid` of the beat on the bus. A beat for a channel with no record stalls the whole stream, including beats for other channels queued behind it. The per-channel record queues and ingress holds keep the data intact, so the effect is delay and not corruption. A stream that must not head-of-line block should have every channel's descriptor issued before the stream starts.

21.5 Descriptor Opcodes (DATA / CTRL_READ / CTRL_WRITE)

The 2-bit opcode selects one of three behaviors. Only `DATA` moves payload, and only `DATA` is affected by the byte-granular changes.

Descriptor Opcodes

Opcode	Value	Behavior
`DATA`	`2'b00`	Payload transfer; `length` in bytes, byte addresses
`CTRL_READ`	`2'b01`	Consumer gate: poll a memory location until `(read & mask) == expected`, then release the chain
`CTRL_WRITE`	`2'b10`	Producer doorbell: one 32-bit write to a memory location, then continue

The CTRL_READ and CTRL_WRITE field layouts are unchanged from RAPIDS Beats, and they carry no packet record. See the RAPIDS Beats descriptor chapter for the field tables and the retry limit.

21.6 Extended Descriptors (row/col-major addressing)

desc_type = 1 (EXTENDED) selects strided, 2-D or circular addressing instead of linear accumulation. It is enabled at build time by USE_ROW_COL_MAJOR_ADDRESSING, which defaults to 1. An extended descriptor is 512 bits in two 256-bit chunks. Chunk 0 is the layout above with desc_type = 1. Chunk 1 is fetched by a second single-beat read at descriptor_addr + 0x20, so the two chunks must be contiguous in memory.

Chunk 1 (at descriptor_addr + 0x20)

Bit Range	Field Name	Width	Description
[255:192]	reserved_hi	64	Reserved (future 3rd dimension)
[191:188]	wr_reserved	4	Reserved
[187:182]	wr_wrap1_log2	6	Write outer wrap window, log2 (0 = off)
[181:176]	wr_wrap0_log2	6	Write inner wrap window, log2 (0 = off)
[175:160]	wr_inner_count	16	Write beats per contiguous run
[159:128]	wr_stride_1	32	Write outer stride, SIGNED bytes
[127:96]	wr_stride_0	32	Write inner stride, SIGNED bytes
[95:92]	rd_reserved	4	Reserved
[91:86]	rd_wrap1_log2	6	Read outer wrap window, log2 (0 = off)
[85:80]	rd_wrap0_log2	6	Read inner wrap window, log2 (0 = off)
[79:64]	rd_inner_count	16	Read beats per contiguous run
[63:32]	rd_stride_1	32	Read outer stride, SIGNED bytes
[31:0]	rd_stride_0	32	Read inner stride, SIGNED bytes

: Extended Descriptor Chunk 1 (rapids_pkg::descriptor_ext_t)

The mode selection, the address equation and the signed strides are unchanged from RAPIDS Beats. The layout is byte-compatible with STREAM's descriptor_ext_t.

21.6.1 EXT descriptors stay beat-aligned (permanent limitation)

TYPE=EXT is not byte-granular, and it is not going to be. This is a deliberate, permanent limitation of RAPIDS. An extended descriptor must have:

- src_addr and dst_addr aligned to BYTE_LANES (32 bytes for the 256-bit build).
- A length that is a multiple of BYTE_LANES.

Row and column striding are counted in whole beats, so a run of inner_count beats is inner_count x BYTE_LANES bytes, and the strides step between beat-aligned rows. An EXT descriptor with an unaligned address or a length that is not a beat multiple is outside the contract and is not checked. Use a linear descriptor when the transfer starts or ends mid-beat. The byte-

granular path (offsets, byte enables, partial last beat, packet records) applies to linear descriptors only.

21.7 Descriptor Fetch

Fetch is unchanged from RAPIDS Beats. The Descriptor Engine issues a single-beat AXI4 read (`arlen` = 0) at the descriptor address, and again at `+0x20` for an EXTENDED descriptor. The parsed descriptor is handed to the scheduler, which converts `length` and the address to `beats_total`, the record and the address working registers as described above. The descriptor address and `next_descriptor_ptr` are 32-byte aligned; that is the only alignment rule that remains for linear descriptors.

21.8 Alignment Requirements

Address and Length Alignment

Field	Alignment	Notes
Descriptor address	32-byte	One descriptor is 32 bytes; an EXTENDED descriptor's chunk 1 is fetched at `+0x20`
`next_ptr` [191:160]	32-byte	32-bit descriptor address, zero-extended by the engine
`dst_addr` [127:64], linear	none	Any byte address
`src_addr` [63:0], linear	none	Any byte address
`dst_addr` / `src_addr`, EXT	`DATA_WIDTH/8`	Permanent: EXT is beat-aligned
`length`, linear	none	Any byte count; 0 moves nothing
`length`, EXT	multiple of `DATA_WIDTH/8`	Permanent: EXT counts whole beats

21.9 What Changed from RAPIDS Beats

Descriptor Differences, RAPIDS Beats to RAPIDS

Aspect	RAPIDS Beats	RAPIDS
Descriptor size and bit layout	256 bits	Same
`length` unit	beats	bytes
`length` = 0	Reserved	Legal: moves nothing, no record, chain continues
`src_addr` / `dst_addr` alignment (linear)	`DATA_WIDTH/8`	None
Beats moved	`length`	`(offset + length + BYTE_LANES - 1) >> OFF_W`
Sink write strobes	All lanes	`WSTRB` marks the bytes written
Source packet	Full beats	One packed packet per descriptor, partial last beat
Per-descriptor packet record	None	`{bytes, offset}` per enabled direction
Sink accepts a packet	Before the descriptor	After the channel's record exists
CTRL_READ / CTRL_WRITE	Defined	Unchanged
TYPE=EXT	Beat rows	Beat rows, aligned addresses and lengths (permanent)
Chaining, `next_ptr`, `last`, `gen_irq`	Defined	Unchanged

21.10 Descriptor Examples

21.10.1 Sink Descriptor (Network to Memory)

A 100-byte sink transfer to `0x1_0000_0007`, chained to a descriptor at `0x2000`, raising an interrupt on completion. The destination is 7 bytes into a beat, so `offset` = 7 and `beats_total` = (7 + 100 + 31) >> 5 = 4. The first beat writes lanes 7 to 31, the last beat writes 4 bytes.

```
64-bit words, LSB word first:
[63:0]    = 0x0000_0000_0000_0000  // src_addr  (unused by the SINK
half)
[127:64]  = 0x0000_0001_0000_0007  // dst_addr  = 0x1_0000_0007
[191:128] = 0x0000_2000_0000_0064  // next_ptr[191:160]=0x2000,
length[159:128]=100
[255:192] = 0x0000_0000_0000_0003  // valid=1, gen_irq=1, last=0
```

21.10.2 Source Descriptor (Memory to Network)

A 60-byte source transfer from `0x2_0000_0005`, terminating the chain. `offset` = 5 and `beats_total` = (5 + 60 + 31) >> 5 = 3 memory beats. The source emits one AXIS packet of two stream beats: 32 bytes, then 28 bytes with `TLAST`.

```
64-bit words, LSB word first:
[63:0]    = 0x0000_0002_0000_0005  // src_addr = 0x2_0000_0005
[127:64]  = 0x0000_0000_0000_0000  // dst_addr (unused by the SOURCE
half)
[191:128] = 0x0000_0000_0000_003C  // next_ptr = 0, length = 60
[255:192] = 0x0000_0000_0000_0007  // valid=1, gen_irq=1, last=1
```

21.11 Software Construction

21.11.1 C Structure Example

```
typedef struct __attribute__((packed, aligned(32))) {
    uint64_t src_addr;      // [63:0]     source byte address
    uint64_t dst_addr;      // [127:64]   destination byte address
    uint32_t length;        // [159:128]  transfer length in BYTES
    uint32_t next_ptr;      // [191:160]  32-bit, 0 terminates the
chain
    uint32_t flags;         // [223:192]  see macros below
    uint32_t reserved[2];   // [255:224]
} rapids_descriptor_t;

// flags bit positions are relative to descriptor bit 192
#define DESC_VALID          (1u << 0)   // [192]
#define DESC_GEN_IRQ        (1u << 1)   // [193]
#define DESC_LAST           (1u << 2)   // [194]
#define DESC_ERROR          (1u << 3)   // [195] hardware-written
#define DESC_CHANNEL_ID(n)  (((n) & 0xFu) << 4)    // [199:196]
#define DESC_PRIORITY(n)    (((n) & 0xFFu) << 8)   // [207:200]
#define DESC_OPCODE(n)      (((n) & 0x3u) << 16)   // [209:208]
#define DESC_TYPE(n)        (((n) & 0x7u) << 18)   // [212:210]

#define DESC_OP_DATA        0x0
#define DESC_OP_CTRL_READ   0x1
#define DESC_OP_CTRL_WRITE  0x2

#define DESC_TYPE_LEGACY    0x0
#define DESC_TYPE_EXT       0x1

// Beats a linear DATA descriptor touches (BYTE_LANES = 32 for 256-
bit)
static inline uint32_t rapids_beats_total(uint64_t addr, uint32_t
length)
{
    const uint64_t lanes = 32;
    if (length == 0)
        return 0;
    return (uint32_t)(((addr & (lanes - 1)) + length + lanes - 1) /
lanes);
}
```

The helper is for software that wants to size buffers or predict burst counts. The hardware does not need it.

21.11.2 Descriptor Ring Setup

```
// Descriptor ring: descriptors must be 32-byte aligned
rapids_descriptor_t *ring = aligned_alloc(32, NUM_DESC *
sizeof(*ring));

for (int i = 0; i < NUM_DESC; i++) {
    memset(&ring[i], 0, sizeof(ring[i]));
    ring[i].dst_addr = dst[i];                // any byte address
    ring[i].length   = len[i];                // bytes
    ring[i].flags    = DESC_VALID | DESC_GEN_IRQ;
    if (i < NUM_DESC - 1) {
        ring[i].next_ptr = (uint32_t)(uintptr_t)&ring[i + 1];
    } else {
        ring[i].next_ptr = 0;                 // terminates the chain
        ring[i].flags   |= DESC_LAST;
    }
}
```

The descriptor buffers themselves stay 32-byte aligned. The payload addresses do not have to be.

---

22 Register Map

22.1 Overview

RAPIDS Beats provides a memory-mapped register interface for configuration and status. All registers are accessed through the single top-level APB slave (`s_apb_*`) and are implemented by the PeakRDL-generated `rapids_regs` register block. One addrmap holds two independent halves of the same layout (`rapids_engine_regs`): **SRC at 0x0000** and **SNK at 0x1000**, 4 KB each. Within a half, the staged descriptor addresses occupy +0x000-0x03F, the launch register `KICK_ENABLE` sits at +0x040, configuration and status at +0x100-0x3FF, and the nested monitor regfile (`rapids_mon_regs`) at +0x800. Every offset below is relative to its half; add 0x1000 for the SNK copy.

Because the SNK half lives at 0x1000, the APB address bus must be at least 13 bits wide; `paddr[12]` selects the half and `paddr[11]` the monitor window.

22.2 Address Space Layout

Range (per half)	Target	Purpose
+0x000-0x03F	rapids_engine_regs	Per-channel staged descriptor addresses
+0x040	rapids_engine_regs	KICK_ENABLE per-channel launch bits
+0x100-0x3FF	rapids_engine_regs	Configuration and status
+0x800-0x9FF	rapids_mon_regs	AXI-monitor config and performance
0x0000 / 0x1000	half base	SRC half / SNK half

The three *_PKT_MASK registers are masks: bit[type] = 1 masks (drops) that packet type, at the monitor-lite and again at the monbus group. They reset to all-masked (0xFFFF), so nothing reaches the group until the host clears the bits of the classes it wants to see. (rapids BUG-008: the description used to read “1 = enable”; the polarity and the reset are as intended.) *_TIMEOUT counts MICROSECONDS in its low 16 bits.

On a build with USE_MON_REGS = 0 (the default when USE_AXI_MONITORS = 0) the two monitor windows keep their addresses but answer every access with PSLVERR and zero read data: the config blocks drive nothing from them, and a read-back that succeeded would tell the host a monitor is configured that does not exist.

: Address Space Layout

22.3 Base Registers (0x100-0x3FF)

Base Registers

Offset	Name	Access	Description
0x100	GLOBAL_CTRL	RW	GLOBAL_EN[0], GLOBAL_RST[1] (level; write 0 to release)
0x104	GLOBAL_STATUS	RO	SYSTEM_IDLE[0]
0x108	VERSION	RO	MINOR[7:0], MAJOR[15:8], NUM_CHANNELS[23:16]=8
0x120	CHANNEL_ENABLE	RW	CH_EN[7:0] per-channel enable
0x124	CHANNEL_RESET	RW	CH_RST[7:0] per-channel reset (level; write 0 to release)
0x140	CHANNEL_IDLE	RO	CH_IDLE[7:0] – not driven at the top level, reads 0; use SCHEDULER_IDLE
0x144	DESC_ENGINE_IDLE	RO	Per-channel descriptor-engine idle
0x148	SCHEDULER_IDLE	RO	Per-channel scheduler idle (excludes CH_ERROR)
0x150-0x16C	CH_STATE[0..7]	RO	Per-channel FSM state (stride 0x4)
0x170	SCHED_ERROR	RO	Per-channel sticky scheduler error
0x174	AXI_RD_COMPLETE	RO	Per-channel read-complete (SRC half only; reads 0 in SNK)
0x178	AXI_WR_COMPLETE	RO	Per-channel write-complete – not driven, reads 0
0x200	SCHED_TIMEOUT_CYCLES	RW	Write-progress timeout window (cycles, reset 1000)
0x204	SCHED_CONFIG	RW	SCHED_EN, TIMEOUT_EN, ERR_EN, COMPL_EN, PERF_EN

Offset	Name	Access	Description
0x208	SCHED_TIMEOUT_LIMIT	RW	LIMIT[7:0] escalation limit (reset 4, 0 = never)
0x220	DESCENG_CONFIG	RW	DESCENG_EN[0], PREFETCH_EN[1], FIFO_THRESH[5:2] (reset 8)
0x224	DESCENG_ADDR0_BASE	RW	Descriptor address range 0 base [31:0]
0x228	DESCENG_ADDR0_LIMIT	RW	Descriptor address range 0 limit [31:0]
0x22C	DESCENG_ADDR1_BASE	RW	Descriptor address range 1 base [31:0]
0x230	DESCENG_ADDR1_LIMIT	RW	Descriptor address range 1 limit [31:0]
0x240	CTRL_CONFIG	RW	CTRLRD_MAX_TRY[8:0] control-read poll retry budget (0-511, reset 16)
0x2A0	AXI_XFER_CONFIG	RW	AXI transfer sizing configuration
0x2B0	PERF_CONFIG	RW	Performance profiler configuration
0x2C0-0x2CC	OBS_CTRL/ OBS_FLAGS/ OBS_DATA0/ OBS_DATA1	RW/RO	Observation mux
0x35C	PERF_CH_SEL	RW	Performance channel select
0x378-0x380	HIST_SEL/ HIST_DATA/ HIST_TOTAL	RW/RO	Latency histogram

22.4 Monitor Registers (+0x800 within each half)

Monitor Registers (+0x800 within each half; SRC at 0x0800, SNK at 0x1800)

Offset	Name	Access	Description
+0x800	MON_FIFO_STATUS	RO	MonBus capture/error FIFO status
+0x804	MON_FIFO_COUNT	RO	MonBus FIFO occupancy
+0x8C0-0x8DC	DAXMON_*	RW	Descriptor-monitor config (enable/timeout/latency/masks)
+0x8E0-0x8FC	RDMON_*	RW	Source-egress AXIS monitor-lite config (same layout; live in the SRC half, rapids TASK-015)
+0x900-0x91C	WRMON_*	RW	Sink-ingress AXIS monitor-lite config (same layout; live in the SNK half, rapids TASK-015; the SRC copy's COMPRESS_EN drives the group)
+0x950-0x978	DAXMON_PERF_*	RO/RW	Descriptor-monitor performance counters
+0x980-0x9A8	RDMON_PERF_*	RO/RW	Read-monitor performance counters
+0x9B0-0x9D8	WRMON_PERF_*	RO/RW	Write-monitor performance counters
+0x9E0-0x9F4	{RD,WR}MON_PERF_CH_*	RO	Per-channel producer/backpressure/starve/idle/overflow

22.5 Key Register Fields

22.5.1 GLOBAL_CTRL (0x100) - Read/Write

GLOBAL_CTRL Register

Bits	Field	Reset	Description
[0]	GLOBAL_EN	0	Master enable for the entire engine
[1]	GLOBAL_RST	0	Global reset: holds every channel of the half in reset while 1. Not self-clearing – write 0 to release
[31:2]	Reserved	0	Reserved

22.5.2 SCHED_TIMEOUT_LIMIT (0x208) - Read/Write

SCHED_TIMEOUT_LIMIT Register

Bits	Field	Reset	Description
[7:0]	LIMIT	4	Consecutive write-progress timeout windows before escalating to a fatal, sticky CH_ERROR. 0 = never escalate. Total escalation time ~= LIMIT x SCHED_TIMEOUT_CYCLES.

22.5.3 DESCENG_CONFIG (0x220) - Read/Write

DESCENG_CONFIG Register

Bits	Field	Reset	Description
[0]	DESCENG_EN	1	Descriptor engine master enable
[1]	PREFETCH_EN	0	Prefetch chaining (0 = on-demand, ~1 ahead)
[5:2]	FIFO_THRESH	8	Descriptors buffered ahead when prefetch enabled
[31:6]	Reserved	0	Reserved

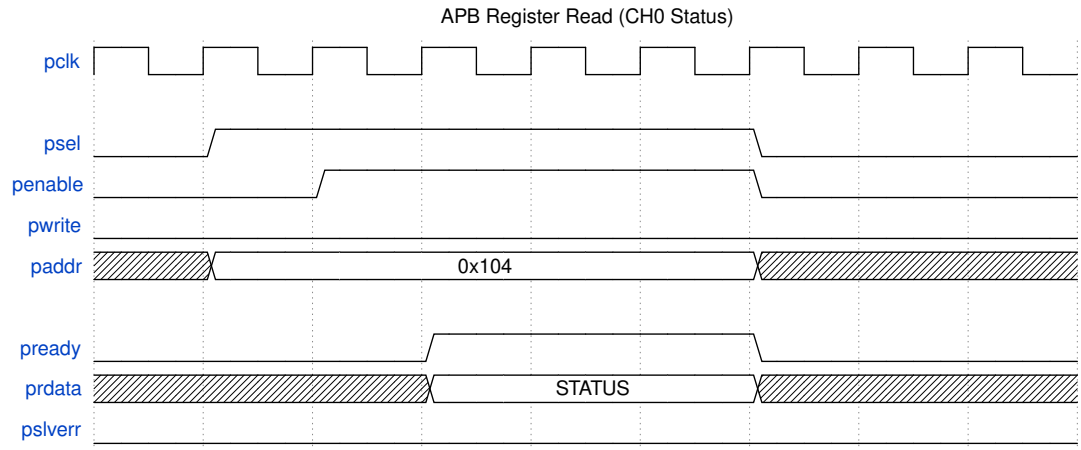
22.5.4 Tile Header Status (0x2D0-0x2D8) - byte-granular RAPIDS with TILE_HDR = 1

These registers exist in both halves and read zero in RAPIDS Beats and in a TILE_HDR = 0 build. The sticky bits are write-1-to-clear.

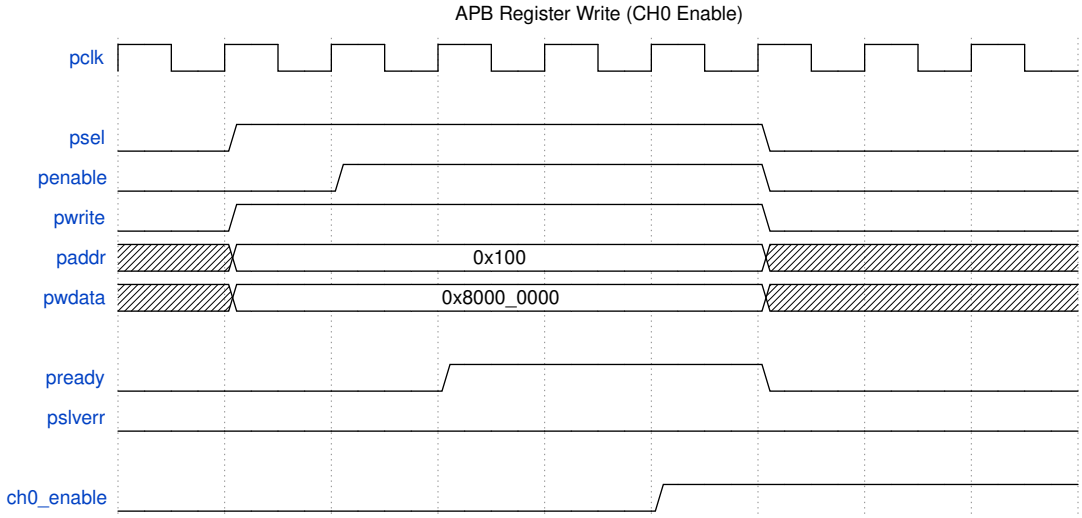
Tile Header Status Registers

Register	Half	Fields
TILE_STATUS (0x2D0)	SNK	UNSUPPORTED [0]: the sink port consumed a packet it could not act on. TAG_MISMATCH [1]: a data packet's tag differed from its stream ID
TILE_DESC_DROPPED (0x2D4)	SRC	DROPn [n]: channel n dropped a cell-issued descriptor (no free slot, EXT, or during channel reset)
TILE_COUNTS (0x2D8)	SNK	DESC_COUNT [15:0]: desc packets passed to the source half. CREDIT_COUNT [31:16]: credit packets received. Both wrap

A drop does not error the channel; TILE_DESC_DROPPED is the only record of it. See the compute-engine tile's TILE_SPEC Section 11.12.



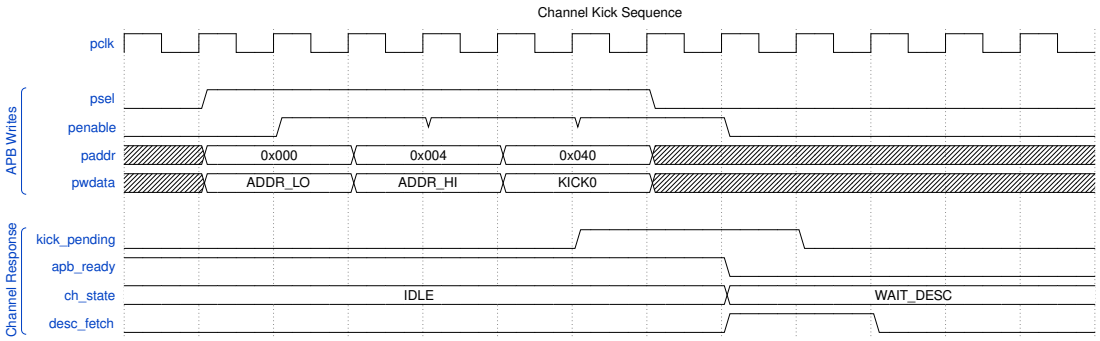
Register Access Timing



Waveform 15

22.6 Channel Kick Sequence

Staging a channel's descriptor address, then writing that channel's bit in `KICK_ENABLE`, triggers descriptor processing:



Waveform 16

22.7 Address Decoding

Registers are global (not per-channel windows); per-channel fields are packed as bit lanes within a single 32-bit register (e.g. `CHANNEL_ENABLE.CH_EN[7:0]`, `SCHEDULER_IDLE.SCHED_IDLE[7:0]`). The only per-channel array is `CH_STATE[0..7]` at 0x150-0x16C (stride 0x4). Descriptor kick-off is part of this register block: staged addresses at 0x000-0x03F (`CH*_DESC_ADDR_{LOW,HIGH}`, stride 0x8) and the `KICK_ENABLE` launch register at 0x040.

```
+0x000 - 0x03F: Staged descriptor addresses (CH*_DESC_ADDR_{LOW,HIGH})
+0x040        : KICK_ENABLE (per-channel launch, singlepulse)
+0x100 - 0x3FF: Base configuration / status registers
+0x800 - 0x9FF: Monitor configuration / performance registers
SRC half at 0x0000, SNK half at 0x1000 (paddr[12]); PSLVERR on the
monitor
window when USE_MON_REGS = 0.
```

23 16-Channel Register Map

RAPIDS comes in two register-map sizes. The shared Register Map chapter describes the 8-channel map, which `rapids_top`, `rapids_cxs_top`, `rapids_8ch_top` and `rapids_8ch_cxs_top` use. `rapids_16ch_top` and `rapids_16ch_cxs_top` use the 16-channel map. That map is the 8-channel map with only the channel-count-dependent registers changed:

Register Differences, 8- and 16-Channel Maps

Register	8-channel map	16-channel map
`CHn_DESC_ADDR_LOW` / `_HIGH`	n = 0..7 at 0x000-0x03C	n = 0..15 at 0x000-0x07C
`KICK_ENABLE`	0x040, `KICK0`..`KICK7`	0x080, `KICK0`..`KICK15`
`CHANNEL_ENABLE`, `CHANNEL_RESET`, `CHANNEL_IDLE`, `DESC_ENGINE_IDLE`, `SCHEDULER_IDLE`, `SCHED_ERROR`, `AXI_RD_COMPLETE`, `AXI_WR_COMPLETE`	one bit per channel, [7:0]	[15:0]
`CH_STATE[n]`	n = 0..7 at 0x150	n = 0..15 at 0x180
`VERSION.NUM_CHANNELS`	8	16
`PERF_CH_SEL.CH_SEL`	[2:0]	[3:0]
`RDMON_PERF_CH_OVERFLOW_HI`, `WRMON_PERF_CH_OVERFLOW_HI`	absent	MON +0x1F8 / +0x1FC: channels 8..15 (4 bits each)

Every other register keeps the name, offset and fields it has in the 8-channel map. Software that addresses registers by name (the generated `rtl/rapids_regmap.py` and `rtl/rapids_regmap_16ch.py`) needs no change except loading the map for its build. Software that hard-codes `KICK_ENABLE` at 0x040 must move it to 0x080.

The 16-channel RDL is derived from the 8-channel one by `bin/gen_rapids_regs_16ch.py`, never edited by hand. The same script writes the per-size glue that the shared `rapids_top_base` includes.

24 Initialization Sequence

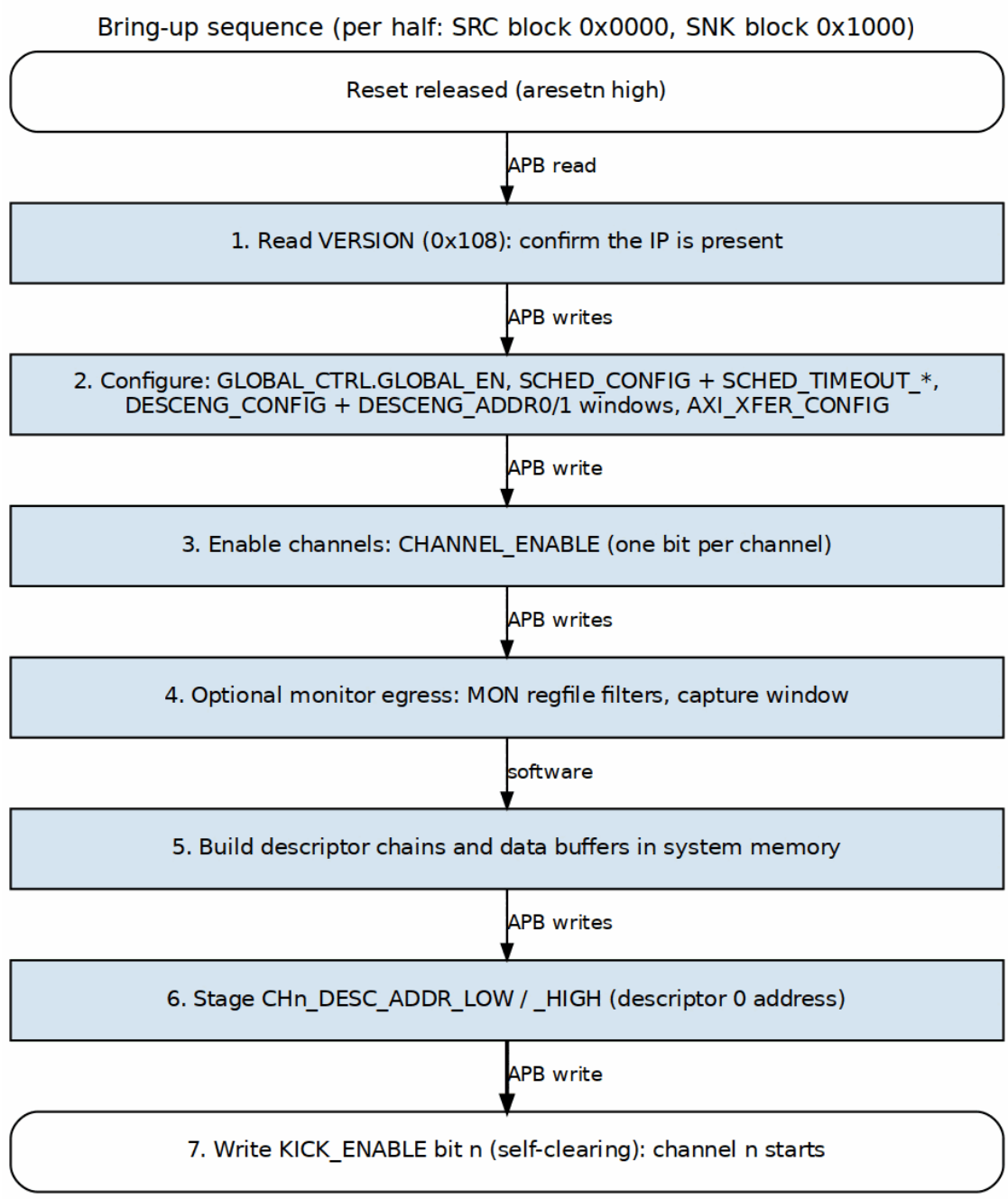
24.1 Overview

This section describes how software brings up RAPIDS Beats and starts a transfer. Every register named here is a register of the PeakRDL `rapids_engine_regs` register file (`rtl/macro_beats/rapids_engine_regs.rdl`), using the names in the generated per-instance map `rapids_regmap.py`. The two halves share one layout: **SRC at 0x0000** and **SNK at 0x1000**. Each half is initialized on its own, and the offsets in the Register Map are relative to the half.

The code below uses `reg_wr(half, NAME, value)` and `reg_rd(half, NAME)` as shorthand for an APB access to register `NAME` in the SRC or SNK half. Resolve the names through the generated map instead of hard-coding offsets.

24.2 Boot Sequence

24.2.1 Figure 5.3.1: Bring-Up Sequence



Bring-Up Sequence

Source: ch05_boot_sequence.dot

24.3 What the Hardware Requires

A channel runs a descriptor only when every one of these terms is true. They are ANDed in `rapids_config_block`, so a missing one leaves the channel idle without raising an error:

Conditions for a Channel to Run

Term	Register.field	Reset	Effect when 0
Global enable	`GLOBAL_CTRL.GLOBAL_EN`	0	Masks `CH_EN`, `SCHED_EN`, `DESCENG_EN` and every monitor enable in the half
Channel enable	`CHANNEL_ENABLE.CH_EN[n]`	0	The scheduler stays in `CH_IDLE` with a descriptor waiting
Scheduler enable	`SCHED_CONFIG.SCHED_EN`	1	All schedulers in the half are disabled
Descriptor engine enable	`DESCENG_CONFIG.DESCENG_EN`	1	No descriptor is fetched
Descriptor window	`DESCENG_ADDR0_BASE/_LIMIT`, `DESCENG_ADDR1_BASE/_LIMIT`	0 / 0xFFFFFFFF	A descriptor address outside both windows is refused and the channel errors
Kick	`KICK_ENABLE.KICKn`	–	Nothing is fetched until the bit is written

`GLOBAL_EN` and `CH_EN` both reset to 0. The other enables reset to 1, and both descriptor windows reset to the full 32-bit range, so a minimal bring-up only has to write `CHANNEL_ENABLE`, `GLOBAL_CTRL`, the staged address and `KICK_ENABLE`.

24.4 Detailed Steps

24.4.1 Step 1: Verify Hardware Presence

```
// VERSION (0x108): MINOR[7:0], MAJOR[15:8], NUM_CHANNELS[23:16]
uint32_t version = reg_rd(SRC, VERSION);
uint8_t minor    = (version >>  0) & 0xFF;   // 0x5A (90) in this
release
uint8_t major    = (version >>  8) & 0xFF;   // 0x00: pre-release,
version 0.90
uint8_t num_ch   = (version >> 16) & 0xFF;   // 8

if (num_ch == 0) {
    printf("ERROR: RAPIDS not detected\n");
    return -1;
}
```

There is no separate configuration register. The channel count is the only build parameter that software can read back. Data width and SRAM depth are not readable from the register file.

24.4.2 Step 2: Optional Reset

`GLOBAL_CTRL.GLOBAL_RST` and `CHANNEL_RESET.CH_RST[n]` are ordinary read/write bits. They do **not** self-clear. A channel is held in reset (scheduler forced to `CH_IDLE`, descriptor engine reset) for as long as its bit, or `GLOBAL_RST`, reads 1. Software must write the bit back to 0 to release it.

```
// Pulse the global reset: set, then clear. Keep GLOBAL_EN at 0
meanwhile.
reg_wr(half, GLOBAL_CTRL, GLOBAL_CTRL_GLOBAL_RST);   // bit 1
reg_wr(half, GLOBAL_CTRL, 0);

// Every scheduler should now report idle (SYSTEM_IDLE = AND of
SCHED_IDLE).
while (!(reg_rd(half, GLOBAL_STATUS) & 1)) { }
```

24.4.3 Step 3: Configure the Engine

```
// Scheduler: SCHED_EN[0] TIMEOUT_EN[1] ERR_EN[2] COMPL_EN[3]
PERF_EN[4]
reg_wr(half, SCHED_TIMEOUT_CYCLES, 1000);   // write-progress window,
in clocks
reg_wr(half, SCHED_TIMEOUT_LIMIT,  4);      // fatal after 4 stalled
windows; 0 = never
reg_wr(half, SCHED_CONFIG, SCHED_EN | TIMEOUT_EN | ERR_EN | COMPL_EN);

// Descriptor engine: DESCENG_EN[0] PREFETCH_EN[1] FIFO_THRESH[5:2]
reg_wr(half, DESCENG_CONFIG, DESCENG_EN | PREFETCH_EN | (4 << 2));
```

```
// Descriptor address windows (lower 32 bits). A fetch address outside
// both windows is refused and the channel enters CH_ERROR.
reg_wr(half, DESCENG_ADDR0_BASE,  desc_region_base);
reg_wr(half, DESCENG_ADDR0_LIMIT, desc_region_base + desc_region_size
- 1);

// Burst sizing: RD_XFER_BEATS[7:0] WR_XFER_BEATS[15:8]
//               ALLOC_SIZE[23:16]  DRAIN_SIZE[31:24]
reg_wr(half, AXI_XFER_CONFIG, (16 << 24) | (16 << 16) | (8 << 8) | 8);
```

The SRC half uses the read-side fields and the SNK half uses the write-side fields. Writing the whole word in both halves does no harm.

24.4.4 Step 4: Optional Monitor and Interrupt Egress

The only interrupt output is `mon_irq`. It comes from the top-level `monbus_axil4_axil4_group` and is asserted while that group's error FIFO is not empty. The scheduler error and completion packets travel on the CORE protocol. For both halves, the group takes its CORE filter from the **SRC** half's `WRMON_*` registers:

```
// CORE packets: PKT_MASK bit[type] = 1 DROPS that type; reset is
0xFFFF.
reg_wr(SRC, WRMON_PKT_MASK, ~((1u << 0) | (1u << 1)) & 0xFFFF);  //
pass Error, Completion

// ERR_SELECT[3:0] bit[type] = 1 routes that type to the error FIFO (-
> mon_irq).
// Bit 0 = Error.  Bit 1 = Completion, which includes the gen_irq
descriptor event.
reg_wr(SRC, WRMON_ERR_CFG, (0xFF << 8) | (1u << 0));             //
keep ERR_MASK reset value
```

Packets that are passed but not selected for the error FIFO go to the capture path (`m_axil_mon_*`). The group, `mon_irq` and the `s_axil_err_*` drain exist only when `USE_AXI_MONITORS = 1`. With `USE_AXI_MONITORS = 0`, `mon_irq` is tied to 0, and software must poll `SCHED_ERROR` and `SCHEDULER_IDLE` instead.

24.4.5 Step 5: Build Descriptors

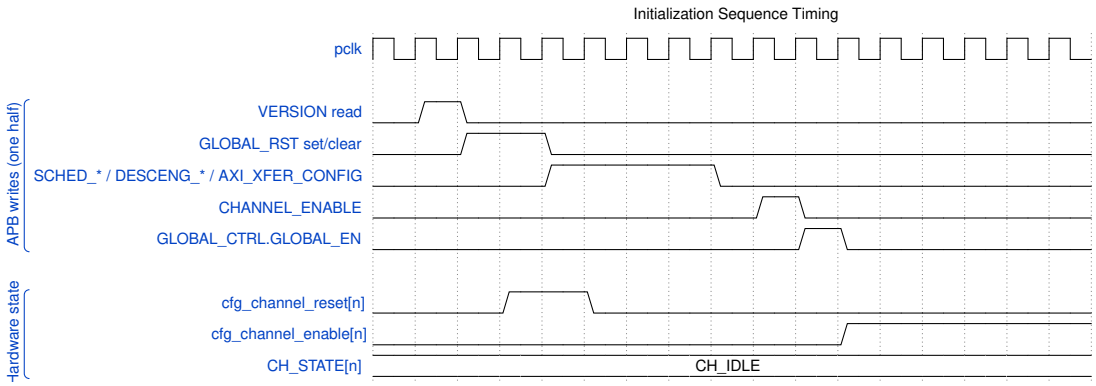
```
// rapids_pkg::descriptor_t, 256 bits (see Descriptor Format)
for (int i = 0; i < NUM_RX_DESC; i++) {
    desc[i].src_addr           = 0;                          // unused
by a sink transfer
    desc[i].dst_addr           = rx_buffer + i * FRAME_SIZE;
    desc[i].length             = FRAME_LEN;   // BEATS in RAPIDS
Beats, BYTES in byte-granular RAPIDS
    desc[i].next_descriptor_ptr = (i < NUM_RX_DESC - 1) ?
(uint32_t)&desc[i + 1] : 0;
    desc[i].valid              = 1;                          // 0 ->
```

```
CH_ERROR
    desc[i].last                = (i == NUM_RX_DESC - 1);
    desc[i].gen_irq             = 1;                          //
completion event RAPIDS_EVENT_IRQ
}
__sync_synchronize();   // descriptors visible before the kick
```

The descriptors must lie inside a `DESCENG_ADDR0`/`DESCENG_ADDR1` window.

24.4.6 Step 6: Enable

```
reg_wr(half, CHANNEL_ENABLE, 1u << ch);          // CH_EN[n]
reg_wr(half, GLOBAL_CTRL,    GLOBAL_CTRL_GLOBAL_EN);   // bit 0;
GLOBAL_RST stays 0
```



Initialization Timing

`cfg_channel_enable[n]` is `CH_EN[n] & GLOBAL_EN`, so it asserts only after the last of the two writes.

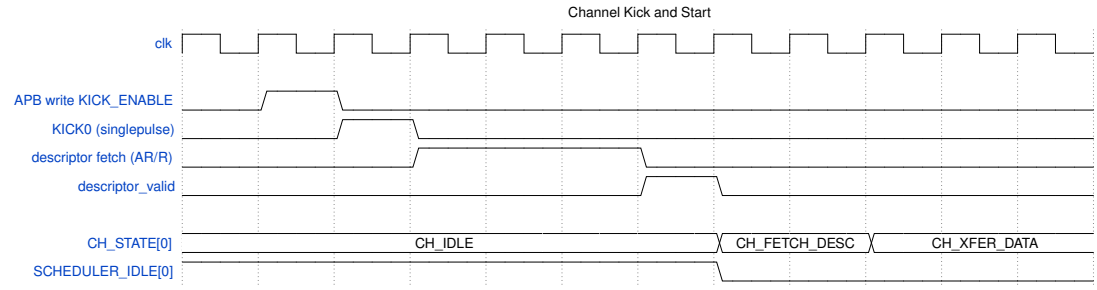
24.5 Starting Transfers

24.5.1 Kick a Channel

Stage the descriptor address, then launch it with a single write to KICK_ENABLE. The address registers only store the value, so writing them does not start anything. Each KICKn bit is a singlepulse field and clears itself. One 32-bit write can launch several channels on the same clock.

```
reg_wr(half, CH0_DESC_ADDR_LOW,  (uint32_t)((uint64_t)&desc[0]));
reg_wr(half, CH0_DESC_ADDR_HIGH, (uint32_t)((uint64_t)&desc[0] >>
32));
reg_wr(half, KICK_ENABLE, 1u << 0);                 // KICK0
```

The APB write completes as soon as the request is raised. The RTL holds the request until the descriptor engine accepts it, so a completed write does not mean the engine has accepted the kick.



Waveform 18

24.6 Completion and Interrupt Handling

There is no interrupt status register and no per-channel interrupt clear. Software sees completion in one of two ways:

- **Polling.** `SCHEDULER_IDLE.SCHED_IDLE[n]` returns to 1 when the scheduler is back in `CH_IDLE` after the last descriptor of a chain. A channel in `CH_ERROR` never reports idle, and `SCHED_ERROR.SCHED_ERR[n]` reads 1 for it instead. Poll both registers, or the loop will hang on a faulted channel. Do not poll `CHANNEL_IDLE`, `AXI_WR_COMPLETE`, or the SNK half's `AXI_RD_COMPLETE`. Those fields are not driven at the top level and always read 0 (see the Register Map).
- **mon_irq.** If Step 4 routed Completion packets to the error FIFO, every descriptor completion lands there, and so does every error. A completion is reported as `RAPIDS_EVENT_DESC_COMPLETE`, or as `RAPIDS_EVENT_IRQ` when the descriptor's `gen_irq` is set. The descriptor engine's descriptor-loaded completions land there too. `mon_irq` stays high until the records are drained over `s_axil_err_*`.

```
void rapids_irq_handler(void) {
    // mon_irq == error FIFO not empty: drain every record
    while (mon_irq_pending()) {
        mon_record_t r = err_fifo_read();        // s_axil_err_* reads
        if (r.pkt_type == PKT_TYPE_ERROR) {
            handle_channel_error(r.half, r.channel_id);   // see Error
Handling
        } else if (r.pkt_type == PKT_TYPE_COMPLETION) {
            process_completed_buffer(r.half, r.channel_id,
r.event_data);
        }
    }
}
```

To run the channel again, stage the next chain's first descriptor address and write its `KICK_ENABLE` bit. No re-enable is needed, because `CH_EN` stays set.

24.7 Reset and Recovery

24.7.1 Per-Channel Reset

```
void reset_channel(int half, int ch) {
    // Assert, then release: CH_RST is NOT self-clearing.
    reg_wr(half, CHANNEL_RESET, 1u << ch);
    reg_wr(half, CHANNEL_RESET, 0);

    while (!(reg_rd(half, SCHEDULER_IDLE) & (1u << ch))) { }
    // SCHED_ERROR[ch] is the CH_ERROR state, so it reads 0 again
here.
}
```

The scheduler's error state is sticky. The only ways out of `CH_ERROR` are a channel reset (`CH_RST[n]`) or a global reset (`GLOBAL_RST`). The reset leaves `CH_EN[n]` and the staged descriptor address unchanged. After the reset, re-stage the address if it should change, then kick.

24.7.2 Global Reset

```
void reset_rapids_half(int half) {
    reg_wr(half, GLOBAL_CTRL, GLOBAL_CTRL_GLOBAL_RST);   // GLOBAL_EN
= 0, RST = 1
    reg_wr(half, GLOBAL_CTRL, 0);                        // release
    while (!(reg_rd(half, GLOBAL_STATUS) & 1)) { }       //
SYSTEM_IDLE
    rapids_init_half(half);                              // Steps 3-6
again
}
```

`GLOBAL_RST` drives every channel's reset in that half only. The other half, and the shared monitor group's error FIFO, are not affected.

25 Error Handling

25.1 Overview

RAPIDS reports errors per channel and per direction. The error classes are the ones RAPIDS Beats already has, and the byte-granular design adds one: the packet on the sink stream does not carry the number of bytes the descriptor asked for. This chapter lists every condition that stops a channel, how software sees it, and how to recover. It also lists three conditions that look like errors and are not.

Two facts shape the whole chapter:

1. **A fatal error parks the channel.** The scheduler enters its error state and stays there. Only a channel reset or a global reset leaves it.
2. **A channel reset recovers every class.** The scheduler, the descriptor and control engines, both data engines and both data paths all take the per-channel reset. No error class needs the hardware reset `aresetn`, and resetting one channel does not disturb the others.

25.2 Error Classes

Error Classes

Class	Detected by	Fatal to the channel	Sticky until
Invalid descriptor (`valid` = 0)	Scheduler, on `CH_FETCH_DESC`	Yes	Channel reset
Descriptor address outside both configured ranges	Descriptor engine, before any fetch	Yes	Channel reset
Descriptor address of zero or otherwise invalid over APB	Descriptor engine	Yes	Channel reset
Descriptor fetch returns an AXI error (`RRESP` not OKAY)	Descriptor engine	Yes	Channel reset
Data read	AXI read engine	Yes	Channel reset

Class	Detected by	Fatal to the channel	Sticky until
returns an AXI error (`RRESP` not OKAY)			
Data write returns an AXI error (`BRESP` not OKAY)	AXI write engine	Yes	Channel reset
Sink packet length mismatch (new)	Sink ingress, at `tlast`	Yes	Channel reset
Control read retry exhausted	Control read engine	Yes	Channel reset
Control write returns an AXI error	Control write engine	Yes	Channel reset
Write-progress timeout, first window	Scheduler timeout counter	No	Any write progress

Class	Detected by	Fatal to the channel	Sticky until
s			
Write-progress timeout, escalated	Scheduler timeout counter	Yes	Channel reset

The first four rows and the control-engine rows behave exactly as in RAPIDS Beats. The data-path rows are the same conditions. In RAPIDS the channel reset also clears them; the “Recovery” section lists what it does to work still in flight.

25.3 The Byte-Granular Error: Packet Length Mismatch

25.3.1 Contract

A sink descriptor of `length` bytes on a channel promises one AXI-Stream packet on that channel. The packet is delimited by `tlast`. The number of valid bytes in the packet, counted from `tstrb`, must equal `length`.

The sink ingress counts the valid bytes of every accepted beat. On the beat that carries `tlast` it compares the running total, including that beat, with the byte count in the channel's packet record. If the two differ, the ingress sets a sticky per-channel bit and raises it on the channel's `sched_wr_error` line.

Length Mismatch Cases

Situation	Result
Packet shorter than `length` (early `tlast`)	Error
Packet longer than `length` (late `tlast`)	Error
Packet equals `length`	No error
Descriptor `length` = 0	No packet record, no packet expected

The hardware does not pad a short packet and does not drop the extra bytes of a long one. Nothing is corrected silently. The bytes that were accepted are still written, so memory that the descriptor covers must not be trusted after this error.

25.3.2 Exposure

The mismatch is visible in three places.

Where the Length Mismatch Appears

Where	What software sees
`SNK.SCHED_ERROR.SCHED_ERR`	Bit for the channel set. This is the same per-channel bit that reports every fatal scheduler error, so the register does not say which class occurred.
MonBus	One error packet per error episode from the channel's scheduler. Its data field carries the sticky write-error flag.
Scheduler state	The channel is in its error state, so `SCHEDULER_IDLE.SCHED_IDLE` does not show it idle.

The error is raised on the sink half only. The source half has no counterpart: the egress re-packer builds the packet from the descriptor, so the packet it emits always has `length` bytes.

25.3.3 How it Clears

The per-channel bit is cleared by the channel reset (and therefore by the global reset, which asserts every channel's reset). It stays clear afterwards; the next packet is checked against its own record. See "Recovery" below for what the reset does to a packet that is part-way through the stream.

25.3.4 Prevention

- Issue the sink descriptor for a packet before, or together with, the first beat of that packet. The stream waits for the descriptor (see "Not Errors" below), so a late descriptor costs time and nothing else.
- Set `length` to the exact byte count of the packet. If the length is only known at the end of the packet, write the descriptor after the length is known.
- Do not split one packet across two descriptors and do not put two packets under one descriptor. One descriptor maps to one `tlast` delimited packet.

25.4 Shared Error Classes

25.4.1 Invalid or Unfetchable Descriptor

The scheduler leaves `CH_FETCH_DESC` only for a descriptor whose `valid` bit is set. A descriptor with `valid` = 0 sends the channel to its error state.

The descriptor engine rejects a descriptor fetch whose address falls outside the two configured address ranges. The rejection happens before any AXI read is issued. It also reports an error for a fetch address of zero or otherwise invalid on the APB path, and for an AXI error response on the fetch itself.

A descriptor whose `length` is zero is not invalid. See “Not Errors”.

25.4.2 Data-Path AXI Errors

The AXI read engine sets a sticky per-channel flag when a data read returns a response other than OKAY. The AXI write engine does the same for a write response. Both flags drive the channel’s error state.

25.4.3 Control Engine Errors

A control read that does not match after the configured number of retries reports an error. Retry exhaustion is what stops a control read that can never match from hanging the channel. A control write that returns an AXI error reports an error. Both are fatal to the descriptor that issued them.

25.4.4 Write-Progress Timeout

The scheduler counts the cycles in which it holds a write request that the write engine has not accepted. A write completion or a write commit resets the count. Its configuration is shared with RAPIDS Beats:

Timeout Configuration

Register field	Meaning	Default
`SCHED_TIMEOUT_CYCLES.TIMEOUT_CYCLES`	Length of one window in cycles	0x3E8
`SCHED_TIMEOUT_LIMIT.LIMIT`	Consecutive expired windows that escalate to a fatal error. Zero never escalates.	4

A window that expires is recoverable. Any write progress clears the strike count. Only the configured number of consecutive expired windows makes the timeout fatal. A LIMIT of zero keeps the timeout purely advisory. The timeout is unchanged by the byte-granular design.

25.5 Not Errors

Three conditions look like faults and are normal operation.

25.5.1 Packet Record Queue Full

Each channel keeps a queue of packet records, one per non-zero data descriptor per enabled direction. The queue holds four records. When it is full, the scheduler waits in `CH_FETCH_DESC` until a record is consumed. Software sees the channel busy and nothing else.

No error bit is set and no MonBus error is sent. The timeout counter runs only while the scheduler holds a write request the write engine has not accepted, so waiting for a record does not count toward it. Descriptor chains longer than the queue therefore run at the pace the data paths consume records.

25.5.2 Sink Beat With No Packet Record

The sink ingress accepts a stream beat for a channel only when that channel's packet record exists. Until then `s_axis_tready` stays low.

`s_axis_tready` is a single signal. It is qualified by the `tid` of the beat that is on the bus. A beat for a channel with no record therefore stalls the whole stream, including beats for other channels that are queued behind it in the sender. The queues and holds inside RAPIDS are per channel, so no data is lost or mixed. The cost is delay, and it lands on every channel that shares the stream.

Two rules keep this delay short:

- Post the sink descriptor for a channel before the sender drives the first beat for that channel.
- A sender that interleaves channels on one stream must not assume that a waiting channel leaves the others unaffected at the AXI-Stream interface.

25.5.3 The 4 KB Burst Cap

An AXI burst must not cross a 4 KB boundary. The scheduler splits a transfer at each boundary. A descriptor that spans several pages issues several bursts, and every burst starts beat-aligned. The split is invisible to software, and it neither sets a flag nor counts against any timeout. The cost appears only as throughput, described in the throughput chapter.

25.6 Error Reporting

25.6.1 Registers

Software finds the failing channel from these fields. All names come from the shared register map and are the same as in RAPIDS Beats.

Error Registers

Register field	Use
`SRC.SCHED_ERROR.SCHED_ERR`	One bit per channel, source half
`SNK.SCHED_ERROR.SCHED_ERR`	One bit per channel, sink half
`SCHEDULER_IDLE.SCHED_IDLE`	One bit per channel, set when the scheduler is idle
`CHANNEL_RESET.CH_RST`	One bit per channel, resets the scheduler and its engines
`GLOBAL_CTRL.GLOBAL_RST`	Resets every channel
`GLOBAL_CTRL.GLOBAL_EN`	Global enable

The per-channel error bit is the summary of every fatal class in the “Error Classes” table. It does not name the class. To name the class, read the MonBus error packet or use the scheduler debug outputs.

25.6.2 MonBus

The scheduler sends one error packet the first time a channel enters its error state. It does not repeat the packet on every cycle. The packet carries the sticky write-error flag and the sticky read-error flag in its data field. MonBus decoding is described in the shared monitor bus chapter.

25.7 Recovery

25.7.1 Channel Reset

Writing the channel's bit in `CHANNEL_RESET.CH_RST` of the half that reported the error, `SNK` or `SRC` (or `GLOBAL_CTRL.GLOBAL_RST` for every channel of that half), returns the channel to idle. The reset reaches the scheduler, the descriptor and control engines, the AXI read and write engines, and the sink and source data paths. Every sticky error flag in that chain clears, including the data read and write response flags and the sink packet length flag. Software should:

1. Set the channel's `CH_RST` bit for at least one clock, then clear it. A single-cycle pulse and a held level both work. To reset both directions of a channel, write both halves.
2. Poll the `SCHED_ERR` bit and the `SCHED_IDLE` bit for the channel. The error bit stays clear and the idle bit sets.
3. Rebuild the descriptor chain from a known-good state. Bytes that the failed chain already wrote stay in memory.
4. Start the channel again as in the initialization sequence.

This recovers every error class in the "Error Classes" table. No hardware reset is needed, and the other channels keep running through it.

25.7.2 What the Reset Does to Work in Flight

A channel reset can arrive with beats, bursts and packets still moving. The hardware settles them as follows, so the channel is clean when the reset releases.

Channel Reset: Handling of In-Flight Work

Where	In-flight work	Effect
Sink stream, packet already started	Beats of the packet still to come	The ingress accepts and discards them up to and including `tlast`, so the tail of the old packet cannot be taken as the start of the next. The stream does not stall.
Sink stream, no packet started	None	Nothing to discard. A beat for the

Where	In-flight work	Effect
		channel is held off while the reset is asserted.
Sink write path	An AXI write burst already open	The write engine finishes the burst with null beats (`WSTRB` = 0), so the AXI protocol stays legal and the null beats write nothing. A beat already presented on the write data channel when the reset hit is completed unchanged. No new burst opens for the channel until the open bursts and their responses have finished. Its `AWADDR` value is unchanged.
Sink write path	Write responses still due	They are consumed and ignored. A `BRESP` error that arrives after the reset does not set the error flag.
Source read path	Read bursts already issued	The read engine issues no further bursts, then drains and discards the

Where	In-flight work	Effect
		returning read data until none is outstanding. A `RRESP` error in that data does not set the error flag.
Source stream, packet in progress	Beats not yet sent	The egress stops. A beat already on `m_axis` completes. The remaining beats are not sent (see below).
Other channels	Everything	Not affected. The data paths mask only the channel in reset.

The sink ingress discards only the tail of a packet of which at least one beat had been accepted when the reset hit. The sender must still send that packet's `tlast`; the beat is taken and dropped, and the next beat for the channel is checked against the next packet record as usual. While the reset is asserted the channel's beats are held off (`s_axis_tready` low).

The source stream is left unterminated. If the reset lands between the first and last beat of a source packet, RAPIDS stops sending it and does not emit a `tlast` (and no null terminator). The receiver sees a partial packet. A receiver that must resynchronize after a channel reset should treat the reset as the end of the packet, or flush its own per-channel state when it resets the channel. A packet that had not started, or that had already sent its `tlast`, is unaffected.

Head-of-line blocking is unchanged. A sink beat for a channel with no packet record still stalls the shared `s_axis_tready`, as described under "Not Errors". A channel reset does not remove this. After the reset, post the next sink descriptor before the sender starts the next packet.

25.8 Software Checklist

Software Checklist

Step	Action
Before the run	Program `SCHED_TIMEOUT_CYCLES` and `SCHED_TIMEOUT_LIMIT`. Post the sink descriptor before the first beat of its packet.
During the run	Poll `SCHED_ERR` for the active channels, or watch MonBus for error packets.
On an error	Note the channel and read the MonBus packet. Decide which class from the descriptor state and the stream.
Recovery	Channel reset for every error class, including a length mismatch or an AXI response error.
After recovery	Re-post descriptors. Re-send the whole packet, not the tail. A source receiver discards the partial packet of a reset channel.

25.9 Related Chapters

- Descriptor Format: the byte `length`, the packet record and the zero-length rule.
- Register Map: the shared register layout.
- Monitor Bus Interface: the error packet format.

26 Throughput

RAPIDS moves bytes. The datapath moves beats. This chapter separates the two figures because they differ for small and unaligned transfers, and it labels every measured number with the build that produced it.

The sink path moves an AXI-Stream ingress stream into memory. The source path reads memory and drives an AXI-Stream egress stream. The two paths are independent engines with their own buffers, schedulers and AXI masters, so their bandwidths add.

26.1 Theoretical Maximum

A beat is `DATA_WIDTH / 8` bytes. The board build uses `DATA_WIDTH = 256`, so a beat is 32 bytes. Per-direction bandwidth is the beat size times the clock.

Theoretical Maximum (256-bit Build)

Item	Value
Beat size	32 bytes
Clock on the board	100 MHz
Line rate per direction	3.2 GB/s
Sink plus source together	6.4 GB/s

The line rate scales linearly with the clock and with the data width. The width does not change the per-beat behavior of the engines, so every utilization figure below holds at any width and only the GB/s scale changes.

26.2 Byte Efficiency

The byte-granular design accepts any start address and any length. The memory side still moves whole beats. A transfer of `length` bytes that starts at byte `offset` inside a beat occupies

```
beats = (offset + length + BYTE_LANES - 1) >> OFF_W
```

beats on that side, with `BYTE_LANES = 32` and `OFF_W = 5` for 256 bits. The useful fraction of those beats is

```
efficiency = useful_bytes / (beats * BYTE_LANES)
```

Here `useful_bytes` is `length`. Efficiency is 1.0 only when both the start and the end land on beat boundaries. The bus meters count a beat with partial byte enables as a productive beat, so their utilization figures measure the beat stream. The byte throughput is the beat throughput multiplied by this efficiency.

Beats and Efficiency for Small and Unaligned Transfers

Length	Offset	Beats	Efficiency
1	0	1	0.031
2	31	2	0.031
32	0	1	1.000
32	1	2	0.500
37	0	2	0.578
60	5	3	0.625
100	7	4	0.781
203	3	7	0.906

The loss is at most two partial beats per descriptor: one at the head and one at the tail. It shrinks as the length grows. A 4 KB aligned transfer is 128 full beats and loses nothing. Software that needs the full line rate should align buffers to the beat size and use lengths that are multiples of it. Software that cannot align pays at most two beats per descriptor.

The offset applies to each half separately. The source half uses the source address and the sink half uses the destination address. A copy whose source and destination have different offsets can have a different beat count on each half.

26.2.1 4 KB Boundaries

AXI bursts do not cross a 4 KB boundary, so a transfer is split at every boundary it crosses. A page holds 128 beats at 32 bytes. The split adds one burst per extra page, and each burst starts beat-aligned. It costs burst-start overhead and no data. For large transfers the overhead amortizes in

the same way as any other burst boundary. The burst-length sweep below gives the measured cost of short bursts.

26.2.2 Head-of-Line Delay on the Sink Stream

The sink stream has one `s_axis_tready`. It is qualified by the `tid` of the beat on the bus. A beat for a channel with no packet record yet stalls the whole stream, including beats for other channels behind it. The queues inside RAPIDS are per channel, so the cost is delay only. A sender that posts each sink descriptor before its first beat never sees this stall. The error chapter describes the rule.

26.3 Measured Results

Three sets of results exist. They answer different questions and must not be mixed.

Sources of Measured Figures

Set	Build	Question it answers
Byte campaign	Byte-granular RAPIDS on the Genesys 2	Are byte lengths and offsets correct on the board?
Beats performance report	RAPIDS Beats on the Genesys 2	How fast does the beat datapath run?
Byte-build performance	None yet	How fast does the byte build run in beat mode?

The byte checkers on the board build hold `ready` while they feed a transfer. A performance row measured with them therefore does not show the beat-mode backpressure behavior that the beats performance report was designed to show. The beat-mode rows in the beats performance report come from the beats build and are quoted as such below. A performance characterization of the byte build needs a build whose checkers run in beat mode, and that build does not exist yet.

26.3.1 Byte Campaign (Correctness)

Source: `projects/fpga-systems/Genesys2/dma-ip/rapids/reports/board`, files `rapids_byte_bytes_20260930_072516.json` and `rapids_byte_smoke_20260930_072435.json`. Two channels were active, one descriptor per channel.

Byte Campaign on the Board (7 of 7 Pass)

Packet bytes	Offset	Sink beats per channel	Result
1	1	1	Pass
2	31	2	Pass
32	0	1	Pass
37	0	2	Pass
100	7	4	Pass
96	17	4	Pass
203	3 (address	7	Pass

Packet bytes	Offset	Sink beats per channel	Result
	4035, crosses a 4 KB boundary)		

The smoke run, four beats on two channels, also passes. The beat count the sink write master issued equals the formula in every row. Golden data matched in every row on both halves. These rows prove correctness. They are runs of one to seven beats, so their utilization figures are dominated by launch and are not throughput.

26.3.2 Shared Engines on the Board

Source: the two Genesys 2 bitstreams of the Resource Estimates chapter, `rapids_byte_pce.bit` (`SHARED_ENGINES = 0`) and `rapids_byte.bit` (`SHARED_ENGINES = 1`). Both were run back to back on 2026-10-17 on the same board, with the same host program. The records are `projects/fpga-systems/Genesys2/dma-ip/rapids/reports/perf/json/rapids_byte_perf_20261017_shared_engines0.json` and `..._shared_engines1.json`. Each run is the 117-point byte performance profile (`--byte-perf --profile standard`, 8 channels, byte-wise checkers) and then every directed sequence (`--byte-seq all`).

Board Results, SHARED_ENGINES 0 and 1

Measure	Per-channel engines	Shared engines
Byte performance points passed	117 of 117	117 of 117
Directed sequences passed	5 of 5 (378 checks)	5 of 5 (378 checks)
Highest sink rate	2560 MB/s	2560 MB/s
Highest source rate	356 MB/s	356 MB/s

Every point is compared with itself across the two bitstreams:

Per-Point Comparison

Half	Points compared	Geometric mean, shared / per-channel	Within 2 %	Shared slower by more than 2 %	Shared faster by more than 2 %
S	109	1.000	99	6 (at most -2.4 %)	4 (at most +5.8

Half	Points compared	Geometric mean, shared / per-channel	Within 2 %	Shared slower by more than 2 %	Shared faster by more than 2 %
ink					%)
Source	97	1.002	95	1 (-2.1 %)	1 (+22.2 %)

The source rates are capped by the harness: the byte-wise checker takes 9 cycles per 32-byte beat, so 356 MB/s is 3200 / 9. The points that moved by more than 2 % are small transfers whose time is dominated by launch and descriptor fetch:

- The six slower sink points are all 8-channel, 33-byte, single-descriptor transfers (129.4 to 126.3 MB/s). All eight channels kick together, and the likely cause is that their first descriptor fetches now take turns at one AR issuer instead of eight. This has not been confirmed with a capture.
- The faster points are single-channel 203-byte sink transfers (up to +5.8 %) and a chain of 16 one-byte descriptors on 4 channels at the source (+22.2 %, 38.8 to 47.4 MB/s). These are consistent with the shared engine's R path, which writes each descriptor straight into its channel's FIFO, but the cause has not been isolated.

On sustained transfers the two builds measure the same. Sharing the engines costs no measurable throughput on this board.

26.3.3 RAPIDS Beats Performance Report

Source: `projects/fpga-systems/Genesys2/dma-ip/rapids_beats/reports/perf/README.md`, version 2.2. Build: RAPIDS Beats, 8 channels, 256 bits, 4 KB of buffer per channel, 100 MHz. Every row is golden-CRC checked. Utilization is engaged utilization: productive cycles over productive, backpressure and starvation cycles.

RAPIDS Beats Build, 8 Channels at 128 KB per Channel

Path	Interface	Engaged utilization	Effective bandwidth
Sink	AXIS in	99.7 %	3.19 GB/s
Sink	AXI4 write	100.0 %	3.20 GB/s
Source	AXI4 read	100.0 %	3.20 GB/s
Source	AXIS out	100.0 %	3.20 GB/s

The 0.3 point shortfall on the sink stream is a one-time 82-cycle fill stall of the 4 KB per-channel buffer. It does not repeat inside a run.

RAPIDS Beats Build, Transfer Size Sweep at 8 Channels

Beats per channel	AXIS in	AXI4 write	AXI4 read	AXIS out
1	88.9 %	38.1 %	27.6 %	27.6 %
4	97.0 %	84.2 %	69.6 %	69.6 %
16	99.2 %	95.5 %	90.1 %	90.1 %
64	99.8 %	98.5 %	97.3 %	97.3 %
256	96.1 %	99.7 %	99.3 %	99.3 %
1024	99.0 %	99.9 %	99.8 %	99.8 %
4096	99.7 %	100.0 %	100.0 %	100.0 %

RAPIDS Beats Build, Channel Scaling at 4096 Beats per Channel

Active channels	AXIS in	AXI4 write	AXI4 read	AXIS out
1	100.0 %	99.9 %	99.7 %	99.7 %
2	100.0 %	99.9 %	99.8 %	99.8 %
4	100.0 %	100.0 %	99.9 %	99.9 %
8	99.7 %	100.0 %	100.0 %	100.0 %

RAPIDS Beats Build, Burst Length Sweep at 8 Channels and 1024 Beats

Beats per burst	AXIS in	AXI4 write	AXI4 read	AXIS out
1	35.9 %	35.6 %	99.8 %	99.8 %
2	71.0 %	71.0 %	99.8 %	99.8 %
4	99.0 %	99.9 %	99.8 %	99.8 %
8 or more	99.0 %	99.9 %	99.8 %	99.8 %

The beats report also holds the memory-latency sweeps and the buffer-depth comparison. Those results belong to the beat datapath and are not repeated here.

26.3.4 Using the Beats Numbers

The byte build was not measured in beat mode, so the rows above are reference points and not measurements of it. The memory-side figures count beats. For a byte transfer, multiply the beat figure by the efficiency in the byte efficiency table to estimate the byte rate. Treat the result as an estimate until the byte build has its own performance run.

26.4 Throughput Limiting Factors

Throughput Limiting Factors

Factor	Effect	Mitigation
Partial head and tail beats	At most two beats per descriptor carry fewer than 32 useful bytes	Align buffers to the beat size and use beat-multiple lengths
4 KB boundaries	One extra burst per page crossed	Place large buffers on page boundaries
Short bursts	Below 4 beats the write side loses efficiency in the beats measurement	Keep the burst configuration at 4 beats or more
Buffer fill at run start	A one-time 82-cycle stall on the sink stream for the 4 KB build	A fixed cost per run, so its share shrinks as transfers grow
Head-of-line stall on the sink stream	Delay on all channels when one channel lacks a packet record	Post each sink descriptor before its first beat
Record queue full	The scheduler waits until a record is consumed; delay only	Chains longer than four descriptors run at the pace the data paths consume records
Memory latency	Reduces small-transfer efficiency	Use the beats report's latency sweeps to size outstanding depth
TYPE=EXT descriptors	Beat rows only, so no head or tail loss but no unaligned addressing	Keep EXT buffers aligned, which they must be anyway

Control descriptors move no payload. A control read gates on a memory location and a control write issues a single write. Budget them as latency, not bandwidth.

26.5 Related Chapters

- Latency Characteristics: shared with RAPIDS Beats.
- Descriptor Format: the byte length, the offset and the beat-count formula.
- Error Handling: the length mismatch and the head-of-line rule.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

27 Latency Characteristics

Latency is dominated by the one-time pipeline fill from a channel kick to the first data beat. Once running, both paths sustain the throughput of Section 6.1; control descriptors add a bounded synchronization delay.

27.1 Latency Components

27.1.1 Kick-to-First-Beat Breakdown

Stage	Typical Cycles (100 MHz)	Notes
Kick decode (APB write to `KICK_ENABLE`)	TBD – re-measure	Address is staged beforehand; the launch request is then held until the descriptor engine accepts it
Descriptor fetch (AXI read, 256-bit)	AXI read latency + 1	Overlapped by prefetch on chains
Descriptor parse	1-2	Opcode + fields decoded
SRAM fill (sink) / AXI read issue (source)	AXI/AXIS latency	First beat into the buffer
First beat out	1-2	Drain to memory (sink) / AXIS (source)

27.1.2 Total Latency Formula

```
L_first_beat  =  L_kick + L_desc_fetch + L_parse + L_pipeline_fill
```

> **Note:** `L_kick` predates the staged-address/`KICK_ENABLE` kick and has not been re-measured. The previous figure came from a path that stalled APB until the descriptor engine accepted the kick; acceptance is now asynchronous, so the term is no longer bounded by an APB stall.

`L_desc_fetch` is hidden on chained descriptors when the descriptor engine has prefetched the next descriptor (`DESCENG_CONFIG.PREFETCH_EN`).

27.2 First-Beat Latency

The dominant term is the memory/network round trip to fill the pipeline. For a sink transfer the first AXIS beat must arrive and be buffered before the first AXI write can drain; for a source transfer the first AXI read must return before the first AXIS egress beat. After the first beat the pipeline is full and throughput is bandwidth-bound, not latency-bound.

27.3 Control-Descriptor Latency

Control descriptors are latency, not throughput (Section 6.1):

Opcode	Latency
`CTRL_WRITE` (doorbell)	One single-beat AXI write: AW->W->B round trip
`CTRL_READ` (gate)	One poll per retry (AXI read + compare); retries paced by `tick_1us`, up to `CTRL_CONFIG.CTRLRD_MAX_TRY` (default 16, max 511)

A `CTRL_READ` gate blocks its descriptor (and the chain) until the polled value matches. The retry budget bounds the worst case: an unsatisfied gate cannot hang the channel – after `CTRLRD_MAX_TRY` polls the engine raises `ctrlrd_error` instead of waiting forever.

27.4 Latency Variability

27.4.1 Sources of Variability

- Memory read latency (descriptor fetch, source reads) and write-response timing.
- AXIS backpressure (`s_axis_tready` / `m_axis_tready`) on the network end.
- Multi-channel arbitration for the shared AXI master.
- `CTRL_READ` retry count when a gate is not yet satisfied.

27.4.2 Worst-Case Latency

Bounded by: memory timeout (`SCHED_TIMEOUT_CYCLES` / `SCHED_TIMEOUT_LIMIT` escalation on the scheduler side) and the control-read retry budget (`CTRL_CONFIG.CTRLRD_MAX_TRY`). Both convert an otherwise-unbounded wait into a reported error so a channel is never permanently stalled.

27.5 Latency Optimization

27.5.1 Software Recommendations

- Chain descriptors so the descriptor engine prefetches the next while the current transfers (hides `L_desc_fetch`).
- Keep the AXIS consumer/producer ready to avoid backpressure stalls.
- Size `CTRL_READ` gates so the expected value is already (or soon) present to minimize poll retries.

27.5.2 Hardware Configuration

- Adequate `SRAM_DEPTH` so the buffer covers memory/network latency without stalling.
- Tune `AXI_XFER_CONFIG` (RD/WR beats, ALLOC/DRAIN) so bursts amortize AXI setup.

Last Updated: 2026-07-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

28 Resource Estimates

The figures in this chapter are measured. They come from the placed and routed board build of the byte-granular RAPIDS on the Genesys 2, not from an estimate. The chapter states the configuration first because every number depends on it.

28.1 Build Configuration

Board Build Configuration

Item	Value
Top level	`rapids_byte_genesys2_top` (RAPIDS plus the characterization harness)
Device	`xc7k325tffg900-2` (Kintex-7 325T)
Tool	Vivado 2025.1, design state Physopt postRoute
Clock	100 MHz
Channels	8, on both the sink and the source half
Data width	256 bits (32-byte beats)
Buffer depth	128 entries per channel, 4 KB per channel
Interface observers	On
AXI monitors	Off

The measured design includes the characterization harness: the UART bridge, the pattern generators, the byte checkers and the bus meters. The figures are therefore an upper bound for RAPIDS alone. The harness is not separated in the reports.

28.2 FPGA Resource Summary

Source: `projects/fpga-systems/Genesys2/dma-ip/rapids/reports/perf/json/rapids_byte_build_bytecrc.json` and `rapids_byte_build_wordcrc.json`, extracted from the post-route reports by `reports/extract_build_metrics.py`.

The harness has two checker builds. `BYTE_CRC=1` is the standard bitstream: the byte-wise CRC checkers take 9 cycles per 32-byte beat and verify every byte and strobe. `BYTE_CRC=0` is a measurement bitstream with word-wide checkers that take one beat per cycle, so the harness does not cap the measured rate.

Post-Route Utilization, 8 Channels, 256 Bits

Resource	Standard (BYTE_CRC=1)	Word-wide (BYTE_CRC=0)	Available
Slice LUTs	92,874 (45.57 %)	86,065 (42.23 %)	203,800
LUT as logic	81,352	75,695	203,800
LUT as distributed RAM	11,522	10,370	64,000
Slice registers	82,379 (20.21 %)	79,527 (19.51 %)	407,600
F7 muxes	4,709	4,430	101,900
F8 muxes	106	107	50,950
Block RAM tiles	68 (15.28 %)	52 (11.69 %)	445
DSP	0	0	0

All block RAM tiles are RAMB36E1. No RAMB18 is used, and no register is a latch. DSP use is zero because RAPIDS moves data and performs no multiply-accumulate. The 16 tiles and about 6,800 LUTs that the word-wide build saves are the byte checkers' golden storage and CRC logic, which belong to the harness and not to RAPIDS.

28.3 Timing

Source: the same two build JSONs.

Timing at 100 MHz

Metric	Standard (BYTE_CRC=1)	Word-wide (BYTE_CRC=0)
Worst negative slack, setup	+0.301 ns	+0.077 ns
Total negative slack	0.000 ns	0.000 ns
Worst hold slack	+0.023 ns	+0.013 ns
Failing endpoints	0 of 320,415	0 of 301,738

All user timing constraints are met in both builds. The slack is small and positive; treat 100 MHz as the design point and not as a margin.

28.4 Board Result

On the standard bitstream (sha256 `cfad34c3...`) the byte-wise campaign passes 117 of 117 points and the directed sequences 4 of 4. On the word-wide bitstream (sha256 `48e1282c...`) the beat-aligned profile passes 28 of 28 points and reaches 3182 MB/s on the sink and 3199 MB/s on the source at 8 channels and 4096 beats, against a theoretical peak of 3200 MB/s (100 MHz x 32 bytes). The word-wide checker CRCs only slice 0 of each beat and ignores strobes, so it is compared with the beats golden; byte-level integrity comes from the byte-wise campaign. Both runs are in the throughput chapter and the characterization report (`reports/perf/README.md`, v0.2).

28.5 Shared Engines: Before and After

Source: two Genesys 2 builds made on 2026-10-17 from the same RTL, differing only in `SHARED_ENGINES`. The bitstreams are `rapids_byte_pce.bit` (`SHARED_ENGINES = 0`, per-channel engines) and `rapids_byte.bit` (`SHARED_ENGINES = 1`, the default). Both builds use 8 channels per half, 256 bits, 128 entries per channel, `BYTE_CRC = 1`, AXI monitors off and observer taps off, at 100 MHz with Vivado 2025.1. The RAPIDS rows are the `u_dut` instance of the hierarchical post-route report, so they exclude the harness. These builds come from newer RTL and harness than the 2026-09-30 build above, so the two sets of numbers do not compare directly.

Post-Route Utilization, SHARED_ENGINES 0 and 1

Resource	Per-channel engines	Shared engines	Change
Whole design, Slice LUTs	79,724	68,520	-11,204 (-14.1 %)
Whole design, Slice registers	58,107	41,497	-16,610 (-28.6 %)
RAPIDS (`u_dut`), LUTs	55,152	44,128	-11,024 (-20.0 %)
RAPIDS (`u_dut`), flip-flops	44,650	28,167	-16,483 (-36.9 %)
`rapids_core`, LUTs / flip-flops	46,350 / 37,984	35,595 / 21,507	-10,755 / -16,477
Sink half, LUTs / flip-flops	23,892 / 19,559	18,626 / 11,320	-5,266 / -8,239
Source half, LUTs / flip-flops	22,458 / 18,425	16,969 / 10,187	-5,489 / -8,238
Block RAM tiles	68	68	0
DSP	0	0	0
Worst slack,	+0.251 ns	+0.016 ns	

Resource	Per-channel engines	Shared engines	Change
setup and recovery (WNS)			
Worst synchronous setup slack (clock group)	+0.259 ns	+0.593 ns	
Worst hold slack	+0.040 ns	+0.009 ns	

Almost all of the saving is in the engines. In the per-channel build the 16 descriptor engines (8 per half) take 11,009 LUTs and 7,360 flip-flops, the 16 control-read engines 2,388 and 7,038, and the 16 control-write engines 1,886 and 4,910. The shared build has one of each per half. The descriptor FIFOs stay per channel, which is why the descriptor saving is smaller than the control-engine saving. Block RAM and DSP are unchanged because none of the engines uses either.

Both builds meet timing at 100 MHz with no failing endpoint. The shared build's worst slack is thinner (+0.016 ns), but that path is not in a shared engine: it is the asynchronous reset recovery path from the reset synchronizer to the sink write engine's data registers. On synchronous setup paths the shared build has more margin than the per-channel build, +0.593 ns against +0.259 ns.

28.6 Comparison with the Beats Build

The RAPIDS Beats performance report lists the resources of its 256-bit build in its version 2.0 text. Both builds run 8 channels at 100 MHz on the same device with a buffer of 128 entries per channel.

Beats and Byte-Granular Builds

Item	RAPIDS Beats, 256-bit, v2.0	RAPIDS, byte-granular (standard build)
Slice LUTs	75,679	92,874
Block RAM tiles	44	68
Worst negative slack	+0.608 ns	+0.301 ns

The columns are not a like-for-like comparison of the two designs. The two bitstreams come from different versions of the RTL and the harness. The byte checkers and the byte-wise golden logic are in the byte build and not in the beats build. The beats build has since been rebuilt, and the v2.2 beats report header gives a slack of +0.286 ns for it and no new resource counts. Use the table as a size comparison of two board images, and not as the cost of byte granularity.

28.7 What Depends on What

Apart from the engine breakdown above, the reports used here do not break the design down by block, and this chapter does not invent a breakdown. The relationships that follow from the design are these.

Scaling Notes

Change	Effect
Channels 8 to fewer, or 16	Scheduler and buffer resources scale with channel count. With shared engines only the per-channel engine state scales; with per-channel engines every engine does
`SHARED_ENGINES` 1 to 0	Adds 11,024 LUTs and 16,483 flip-flops at 8 channels (above)
Buffer depth	Block RAM scales with entries per channel, per direction, and with the entry width
Data width	Datapath logic and buffer width scale with it. The line rate scales with it.
Interface observers off	Removes the observer logic and its histograms
AXI monitors on	Adds monitor logic. They are off in this build.

The sink buffer entry is `DATA_WIDTH + DATA_WIDTH / 8` bits wide, because each entry stores the byte enables beside the data. The source buffer entry stays `DATA_WIDTH` bits. The byte-granular sink therefore stores a wider entry than the beats sink for the same depth and width.

The byte-granular design adds per-channel state that the beats design lacks: a packet-record queue four deep, a sink spill register, an ingress packet byte counter and, on the source half, the re-pack hold register. The reports do not isolate their cost.

28.8 Related Chapters

- Throughput: the measured campaign and the beats report.
- Error Handling: the reset behavior of the sticky flags.